

ETERNAL SUNSHINE: AN AI-POWERED BOARD-SPECIFIC LEARNING, ASSESSMENT AND RESOURCE PLATFORM FOR CLASSES 9–12



**Anum Sheraz
(FA22-BSE-007)**

**Munazza Javed
(FA22-BSE-019)**

**Aniqa Saghir
(FA22-BSE-031)**

Session 2022–2026

**Department of Software Engineering
Faculty of Engineering & Technology
Mirpur University of Science and Technology (MUST), Mirpur
AJK Pakistan**

ETERNAL SUNSHINE: AN AI-POWERED BOARD-SPECIFIC LEARNING, ASSESSMENT AND RESOURCE PLATFORM FOR CLASSES 9–12



By

**Anum Sheraz
(FA22-BSE-007)**

**Munazza Javed
(FA22-BSE-019)**

**Aniqa Saghir
(FA22-BSE-031)**

A Project Report is submitted in partial fulfillment of
the requirement for the degree of

**Bachelor of Science
In
SOFTWARE ENGINEERING**

Session 2022–2026

**Department of Software Engineering
Faculty of Engineering & Technology**

**Mirpur University of Science and Technology (MUST), Mirpur
(AJK) Pakistan**

CERTIFICATION

We hereby undertake that this final year project is an original one and no part of this final year project falls under plagiarism. If found otherwise, at any stage, we will be responsible for the consequences.

Student's Name: Anum Sheraz	Signature: _____
Registration No.: FA22-BSE-007	Date: _____
Student's Name: Munazza Javed	Signature: _____
Registration No.: FA22-BSE-019	Date: _____
Student's Name: Aniq Saghir	Signature: _____
Registration No.: FA22-BSE-031	Date: _____

Certified that the contents and form of final year project entitled

“Eternal Sunshine: An AI-Powered Board-Specific Learning, Assessment and Resource Platform for Classes 9–12”

submitted by

Anum Sheraz, Munazza Javed, Aniq Saghir

have been found satisfactory for the requirement of the degree.

Supervisor: _____

Co-Supervisor (If any): _____

External Examiner: _____

Chairperson: _____

DEDICATION

To our parents, whose patience and quiet sacrifice made every late night possible;
to our teachers in the Department of Software Engineering, who taught us to ask
better questions before writing a single line of code; and to the students of classes
nine to twelve across Pakistan who study without a tutor, a library or a second
chance — this work is for you.

CONTENTS

	Page No.
LIST OF FIGURES.....	xiii
LIST OF TABLES.....	xvi
LIST OF ABBREVIATIONS.....	xix
ACKNOWLEDGMENTS.....	xx
ABSTRACT.....	xxi
1. INTRODUCTION.....	1
1.1 OVERVIEW.....	1
1.2 BACKGROUND AND MOTIVATION.....	2
1.2.1 The examination-centred reality of Pakistani schooling.....	2
1.2.2 Why general chatbots are not enough.....	3
1.2.3 Evidence from the requirement survey.....	4
1.3 PROBLEM STATEMENT.....	5
1.4 AIM AND OBJECTIVES.....	6
1.4.1 Aim.....	6
1.4.2 Objectives.....	6
1.5 SCOPE OF THE PROJECT.....	7
1.5.1 In scope.....	7
1.5.2 Out of scope.....	7
1.6 SIGNIFICANCE AND CONTRIBUTIONS.....	8
1.7 ALIGNMENT WITH THE SUSTAINABLE DEVELOPMENT GOALS.....	9
1.8 COMPLEX ENGINEERING PROBLEM ATTRIBUTES.....	10
1.9 REPORT ORGANISATION.....	10
2. LITERATURE REVIEW.....	12
2.1 INTRODUCTION.....	12
2.2 ARTIFICIAL INTELLIGENCE IN EDUCATION.....	12
2.3 INTELLIGENT TUTORING SYSTEMS.....	14
2.4 CONVERSATIONAL AGENTS IN EDUCATION.....	14
2.5 LARGE LANGUAGE MODELS AND THEIR LIMITATIONS.....	15
2.6 RETRIEVAL-AUGMENTED GENERATION.....	16
2.7 VECTOR DATABASES AND SEMANTIC SEARCH.....	18
2.8 AUTOMATED ASSESSMENT OF FREE-TEXT ANSWERS.....	19
2.9 EVALUATING RETRIEVAL-AUGMENTED SYSTEMS.....	20
2.10 DOCUMENT PROCESSING FOR LOW-RESOURCE EDUCATIONAL CORPORA.....	21
2.11 THE EDUCATIONAL TECHNOLOGY LANDSCAPE IN PAKISTAN.....	22
2.12 COMPARATIVE ANALYSIS.....	23

2.13	SUMMARY OF THE REVIEWED LITERATURE.....	24
2.14	RESEARCH GAP AND POSITIONING.....	25
2.15	SUMMARY.....	26
3.	REQUIREMENT ANALYSIS AND METHODOLOGY.....	27
3.1	INTRODUCTION.....	27
3.2	DEVELOPMENT METHODOLOGY.....	27
3.2.1	Selection of an Agile process.....	27
3.2.2	Sprint structure and outcomes.....	28
3.2.3	Version control and collaboration.....	29
3.3	REQUIREMENT ELICITATION.....	29
3.4	SURVEY RESULTS AND ANALYSIS.....	30
3.4.1	Respondent profile.....	30
3.4.2	Study environment and habits.....	31
3.4.3	Existing tool use and its problems.....	32
3.4.4	What students find difficult.....	33
3.4.5	Demand for board-specific support.....	34
3.4.6	Features students asked for.....	36
3.4.7	Requirements derived from the survey.....	37
3.5	STAKEHOLDER AND USER CLASS ANALYSIS.....	38
3.5.1	User classes.....	38
3.5.2	Representative personas.....	39
3.6	FUNCTIONAL REQUIREMENTS.....	39
3.6.1	Authentication and profile management.....	39
3.6.2	Academic selection and curriculum scoping.....	40
3.6.3	Conversational tutoring.....	40
3.6.4	Practice, assessment and evaluation.....	41
3.6.5	Resources, analytics and administration.....	41
3.7	NON-FUNCTIONAL REQUIREMENTS.....	42
3.7.1	Performance.....	42
3.7.2	Security.....	43
3.7.3	Usability, reliability and maintainability.....	43
3.7.4	Safety.....	44
3.7.5	Legal and ethical constraints.....	44
3.8	BEHAVIOURAL MODELS.....	44
3.8.1	Use case model.....	44
3.8.2	Use case specifications.....	45
3.8.3	State machine model.....	47
3.8.4	Interaction sequence model.....	48
3.9	FEASIBILITY STUDY.....	49
3.9.1	Technical feasibility.....	49

3.9.2 Economic feasibility.....	50
3.9.3 Operational feasibility.....	51
3.9.4 Schedule feasibility.....	51
3.9.5 Risk assessment.....	51
3.10 REQUIREMENT TRACEABILITY MATRIX.....	52
3.11 TOOLS AND TECHNOLOGIES.....	52
3.12 SUMMARY.....	54
4. SYSTEM ARCHITECTURE.....	55
4.1 INTRODUCTION.....	55
4.2 ARCHITECTURAL DESIGN APPROACH.....	55
4.2.1 Architectural principles.....	56
4.3 HIGH-LEVEL ARCHITECTURE.....	57
4.4 SUBSYSTEM ARCHITECTURE.....	58
4.4.1 Presentation subsystem.....	58
4.4.2 Application subsystem.....	59
4.4.3 Artificial-intelligence subsystem.....	59
4.4.4 Data subsystem.....	59
4.5 DATA FLOW.....	60
4.5.1 Context diagram.....	60
4.5.2 Level 1 decomposition.....	61
4.6 ARTIFICIAL-INTELLIGENCE SUBSYSTEM DESIGN.....	62
4.7 KNOWLEDGE INGESTION PIPELINE.....	63
4.7.1 Stage 1 — acquisition and metadata by convention.....	64
4.7.2 Stage 2 — optical character recognition.....	64
4.7.3 Stage 3 — cleaning and chunking.....	65
4.7.4 Stage 4 — embedding and indexing.....	65
4.8 RETRIEVAL AND GENERATION AT REQUEST TIME.....	66
4.8.1 Query construction.....	66
4.8.2 Embedding with a cache.....	67
4.8.3 Filtered vector search.....	67
4.8.4 Combined retrieval for question generation.....	67
4.8.5 Context assembly and generation.....	68
4.9 CONVERSATION MEMORY ARCHITECTURE.....	68
4.10 INTERFACE CONTRACT BETWEEN TIERS.....	69
4.10.1 Application API surface.....	70
4.11 SECURITY ARCHITECTURE.....	71
4.12 DESIGN PATTERNS APPLIED.....	72
4.13 DEPLOYMENT ARCHITECTURE.....	73
4.14 SUMMARY.....	74
5. DETAILED SYSTEM DESIGN.....	75

5.1	INTRODUCTION.....	75
5.2	DATABASE DESIGN.....	75
5.2.1	Design objectives and normalisation.....	75
5.2.2	Entity-relationship model.....	76
5.2.3	Data dictionary.....	77
5.2.4	Enumerations and value normalisation.....	80
5.3	CLASS DESIGN.....	81
5.4	COMPONENT SPECIFICATIONS.....	82
5.4.1	Authentication and Profile Component.....	82
5.4.2	Retrieval Component.....	83
5.4.3	Teacher Agent Component.....	84
5.4.4	Tester and Evaluator Component.....	84
5.4.5	Session Evaluation Component.....	85
5.4.6	Conversation Memory Component.....	85
5.4.7	Assessment Orchestration Component.....	86
5.4.8	Content Management Component.....	87
5.4.9	Analytics Component.....	87
5.4.10	Ingestion Pipeline Component.....	88
5.5	USER INTERFACE DESIGN.....	88
5.5.1	Design principles.....	88
5.5.2	Navigation model.....	89
5.5.3	Screen inventory.....	89
5.6	PROMPT DESIGN.....	90
5.6.1	Teacher prompt.....	90
5.6.2	Tester prompt.....	91
5.6.3	Evaluator prompt.....	91
5.6.4	Session-evaluation prompt.....	92
5.7	ALGORITHM DESIGN.....	92
5.7.1	Overlapping chunk segmentation.....	92
5.7.2	Filtered retrieval.....	93
5.7.3	Conversation memory compression.....	93
5.7.4	Score parsing, clamping and session aggregation.....	94
5.7.5	Bounded activity feed.....	94
5.8	SUMMARY.....	95
6.	IMPLEMENTATION.....	96
6.1	INTRODUCTION.....	96
6.1.1	Implementation statistics.....	96
6.2	DEVELOPMENT ENVIRONMENT AND REPOSITORY STRUCTURE.....	97
6.2.1	Environment.....	97
6.2.2	Repository layout.....	97

6.2.3 Repository and version history.....	98
6.3 MODULE IMPLEMENTATION.....	100
6.3.1 Curricular vocabulary.....	100
6.3.2 Authentication.....	101
6.3.3 Academic selection and profile.....	102
6.3.4 The AI service boundary.....	103
6.3.5 Retrieval.....	104
6.3.6 The teacher agent and the chat module.....	105
6.3.7 The practice and assessment module.....	108
6.3.8 Progress analytics.....	111
6.3.9 Resource library and administration.....	112
6.3.10 Cost instrumentation.....	114
6.4 INTEGRATION BETWEEN THE TWO RUNTIMES.....	114
6.5 ENGINEERING CHALLENGES AND RESOLUTIONS.....	115
6.5.1 Refusing to answer because of a retrieval gap.....	115
6.5.2 Cost growth with conversation length.....	116
6.5.3 Optical character recognition noise.....	116
6.5.4 Memory exhaustion during ingestion.....	117
6.5.5 Losing an ingestion run to a network failure.....	117
6.5.6 Impossible marks.....	117
6.5.7 Inconsistent stored vocabulary.....	118
6.5.8 Cross-site request forgery in asynchronous calls.....	118
6.5.9 Unbounded activity growth.....	118
6.6 SUMMARY.....	119
7. SYSTEM TESTING.....	120
7.1 INTRODUCTION.....	120
7.2 TESTING STRATEGY.....	120
7.3 AUTOMATED TEST SUITE.....	121
7.3.1 A defect found and closed by the suite.....	122
7.4 FUNCTIONAL TEST CASES.....	123
7.4.1 Authentication and account management.....	123
7.4.2 Academic selection and scoping.....	124
7.4.3 Conversational tutoring.....	125
7.4.4 Practice and assessment.....	125
7.4.5 Resources, analytics and administration.....	126
7.5 NON-FUNCTIONAL TEST CASES.....	127
7.6 EVALUATION OF THE RETRIEVAL AND LANGUAGE-MODEL COMPONENTS.....	128
7.6.1 Test design for retrieval.....	128
7.6.2 Test design for the language-model agents.....	129

7.6.3 Threats to validity.....	130
7.7 INTEGRATION AND SYSTEM TESTING.....	130
7.7.1 Service contract testing.....	130
7.7.2 End-to-end journeys.....	131
7.8 DEFECT LOG.....	131
7.9 TEST SUMMARY AND COVERAGE.....	133
7.9.1 What was not tested.....	133
7.10 SUMMARY.....	134
8. RESULTS AND EVALUATION.....	135
8.1 INTRODUCTION.....	135
8.2 DELIVERED SYSTEM.....	135
8.3 THE SYSTEM IN OPERATION.....	136
8.4 EVALUATION OF THE ARTIFICIAL-INTELLIGENCE SUBSYSTEM.....	138
8.4.1 The indexed corpus.....	138
8.4.2 Retrieval quality.....	140
8.4.3 Does the curricular filter actually hold?.....	142
8.4.4 Grounding: answers with and without retrieval.....	142
8.4.5 Marking accuracy against a gold set.....	144
8.4.6 Validity of generated question sets.....	146
8.4.7 Conversation memory and bounded cost.....	147
8.4.8 Volume of system-generated data.....	148
8.4.9 Marking behaviour in live sessions.....	149
8.4.10 Marking quality in live sessions.....	149
8.4.11 Explanation quality.....	150
8.4.12 A rolling summary in operation.....	151
8.5 RESPONSIVENESS.....	152
8.6 COST ANALYSIS.....	152
8.7 USER ACCEPTANCE TESTING.....	155
8.8 COMPARISON WITH EXISTING PLATFORMS.....	156
8.9 ASSESSMENT AGAINST OBJECTIVES.....	157
8.10 DISCUSSION AND LIMITATIONS.....	158
8.10.1 What the evidence supports.....	158
8.10.2 Limitations of the system.....	158
8.10.3 Limitations of this evaluation.....	159
8.11 SUMMARY.....	160
9. CONCLUSION AND FUTURE WORK.....	161
9.1 CONCLUSION.....	161
9.2 CONTRIBUTIONS.....	163
9.3 STRENGTHS.....	164
9.4 LIMITATIONS.....	164

9.5 COST POSITION.....	165
9.6 FUTURE WORK.....	166
9.6.1 Immediate remediation.....	166
9.6.2 Short-term enhancements.....	166
9.6.3 Medium-term extensions.....	167
9.6.4 Longer-term research directions.....	168
9.7 REFLECTION ON THE DESIGN PROCESS.....	168
REFERENCES.....	170
APPENDIX A: GLOSSARY.....	175
APPENDIX B: REQUIREMENT ELICITATION SURVEY.....	177
B.1 THE INSTRUMENT.....	177
B.2 RESULTS OF THE CLOSED QUESTIONS.....	178
B.3 CODING OF THE OPEN QUESTIONS.....	180
B.4 CHARTS OF THE PRINCIPAL RESULTS.....	181
B.5 LIMITATIONS OF THE SURVEY.....	185
APPENDIX C: SELECTED CODE LISTINGS.....	186
C.1 PROMPT TEMPLATE — TEACHER AGENT.....	186
C.2 PROMPT TEMPLATE — EVALUATOR.....	186
C.3 PROMPT TEMPLATE — TESTER AGENT.....	187
C.4 PAST-PAPER CLEANING RULES.....	188
C.5 RESUMABLE EMBEDDING AND UPSERT.....	188
APPENDIX D: USER MANUAL.....	190
D.1 FOR STUDENTS.....	190
D.2 FOR ADMINISTRATORS.....	190
D.3 FOR THE SYSTEM OPERATOR.....	191
APPENDIX E: SIMILARITY REPORT.....	193

LIST OF FIGURES

Figure No.....	Page No.
1.1: Layered architecture of the Eternal Sunshine platform.....	2
1.2: Reported mismatch between board content and online material.....	5
2.1: The Retrieval-Augmented Generation request path implemented in.....	18
2.2: The four-stage offline ingestion pipeline.....	22
3.1: The Agile process applied to the project.....	28
3.2: Class of survey respondents.....	30
3.3: Where respondents study most.....	31
3.4: What students do when a topic is unclear.....	32
3.5: Use of online study tools.....	32
3.6: Problems reported with general-purpose assistants.....	33
3.7: What students need more help with.....	34
3.8: Biggest difficulty reported while studying.....	34
3.9: Reported mismatch between board and online material.....	35
3.10: Value of an application tied to the board syllabus.....	35
3.11: Ease of finding past papers and textbooks online.....	36
3.12: Features students asked for in a learning application.....	36
3.13: Use case diagram for the Eternal Sunshine platform.....	45
3.14: State transition diagram for a user session.....	48
3.15: Sequence diagram.....	49
4.1: Complete layered architecture with the components of each tier.....	57
4.2: Component diagram.....	60
4.3: Context diagram.....	61
4.4: Level 1 Data Flow Diagram.....	62
4.5: The four-stage offline ingestion pipeline.....	64
4.6: Request-time Retrieval-Augmented Generation flow.....	66
4.7: Conversation memory compression.....	68
4.8: Authentication and authorisation flow.....	72
4.9: Deployment diagram.....	73
5.1: Entity-relationship diagram of the MySQL schema.....	76
5.2: Class diagram of the core application classes.....	81
5.3: Navigation map.....	89
6.1: Size of the delivered codebase.....	97
6.2: Commits to the project repository by month.....	99
6.3: The sign-in screen.....	101

6.4: Two-factor authentication set-up on the account security page.....	102
6.5: Academic selection.....	103
6.6: The student profile page.....	103
6.7: The AI tutor interface.....	107
6.8: A grounded explanation produced by the teacher agent.....	108
6.9: Practice set-up.....	108
6.10: Question-type selection between multiple-choice, short and long.....	109
6.11: A generated multiple-choice question with its past-paper.....	109
6.12: Per-answer feedback from the evaluator.....	110
6.13: The session result.....	111
6.14: The student dashboard.....	112
6.15: The progress page.....	112
6.16: The student resource library with board, class, subject and.....	113
6.17: The administrative content console.....	113
6.18: The administrative user console with search, role and status.....	113
6.19: The administrative overview.....	113
7.1: The four levels of testing applied to the project and the.....	120
7.2: Output of the automated Pest suite.....	123
8.1: The public landing page presenting the product proposition and.....	136
8.2: The student dashboard after several completed practice sessions.....	136
8.3: A multi-turn tutoring session.....	137
8.4: The practice module in progress.....	137
8.5: The session result produced by the session evaluator.....	137
8.6: The progress page with subject-wise performance and the.....	137
8.7: The resource library with board, class and subject filters.....	138
8.8: The administrative console for content management.....	138
8.9: The interface at a 390-pixel mobile viewport.....	138
8.10: Indexed chunks by class.....	139
8.11: Indexed chunks by subject.....	140
8.12: Mean top-1 similarity by subject.....	141
8.13: Judged answer quality with and without retrieval.....	143
8.14: Mean unsupported claims per answer.....	144
8.15: Expected mark against awarded mark across the gold set.....	145
8.16: Prompt size across a ten-turn tutoring session.....	147
8.17: Mean tokens per call, by endpoint.....	153
B.1: Class of respondents.....	181
B.2: Where respondents study most.....	181

B.3: What students do when a topic is unclear.....	182
B.4: Use of online study tools.....	182
B.5: Problems reported with general-purpose assistants.....	182
B.6: What students need more help with.....	183
B.7: Biggest difficulty reported while studying.....	183
B.8: Value of an application tied to the board syllabus.....	183
B.9: Value of a chatbot that tests, marks and explains.....	184
B.10: Ease of finding past papers and textbooks online.....	184
B.11: Mismatch between board content and online material.....	184
B.12: Features asked for in a learning application.....	185
E.1: Similarity report summary page.....	193

LIST OF TABLES

Table No.....	Page No.
1.1 Project objectives and where each is addressed.....	6
1.2 Sustainable Development Goals and the features that meet them.....	9
1.3 Complex engineering problem attributes shown by the project.....	10
2.1 Comparison of existing platforms against the capability set.....	23
2.2 Principal reviewed literature and its bearing on this project.....	24
3.1 Sprint plan and outcomes.....	28
3.2 From survey findings to system requirements.....	37
3.3 Stakeholder analysis.....	38
3.4 User personas used to guide interface decisions.....	39
3.5 Functional requirements.....	39
3.6 Functional requirements.....	40
3.7 Functional requirements.....	40
3.8 Functional requirements.....	41
3.9 Functional requirements.....	41
3.10 Non-functional requirements.....	42
3.11 Non-functional requirements.....	43
3.12 Non-functional requirements.....	43
3.13 Use case UC-01.....	45
3.14 Use case UC-02.....	46
3.15 Use case UC-03.....	46
3.16 Use case UC-04.....	47
3.17 Development-phase cost position.....	50
3.18 Principal project risks and mitigations.....	51
3.19 Requirement traceability matrix.....	52
3.20 Tools and technologies with selection rationale.....	53
4.1 The four agents of the artificial-intelligence subsystem.....	62
4.2 Contract between the Laravel application and the FastAPI AI.....	69
4.3 HTTP API exposed by the Laravel application.....	70
4.4 Security controls by architectural band.....	71
4.5 Design patterns used and where they appear.....	72
5.1 Data dictionary.....	77
5.2 Data dictionary.....	78
5.3 Data dictionary.....	78
5.4 Data dictionary.....	78

5.5 Data dictionary.....	79
5.6 Data dictionary.....	79
5.7 Principal classes and their responsibilities.....	81
5.8 Component specification.....	82
5.9 Component specification.....	83
5.10 Component specification.....	84
5.11 Component specification.....	84
5.12 Component specification.....	85
5.13 Component specification.....	85
5.14 Component specification.....	86
5.15 Component specification.....	87
5.16 Component specification.....	87
5.17 Component specification.....	88
5.18 Screen inventory.....	89
5.19 Summary of the four prompt templates.....	92
6.1 Size of the delivered codebase.....	96
6.2 Development environment.....	97
6.3 Repository at the time of submission.....	98
7.1 Automated test suite composition and result.....	121
7.2 Test cases — authentication and account management.....	123
7.3 Test cases — academic selection and curricular scoping.....	124
7.4 Test cases — AI tutoring.....	125
7.5 Test cases — practice, marking and session evaluation.....	125
7.6 Test cases — resources, analytics and administration.....	126
7.7 Test cases — performance, security and compatibility.....	127
7.8 The evaluation harness.....	128
7.9 System test journeys.....	131
7.10 Defect log.....	131
7.11 Coverage summary by requirement area.....	133
8.1 Delivered feature set.....	135
8.2 Composition of the indexed corpus.....	139
8.3 Retrieval results over thirty curriculum questions.....	140
8.4 Curricular isolation probes.....	142
8.5 Judged answer quality, retrieval enabled against disabled.....	143
8.6 Marking accuracy of the evaluator agent.....	145
8.7 Structural validity of generated question sets.....	146
8.8 Records produced by the system during development and.....	148

8.9	Scored practice sessions by question type.....	149
8.10	Verbatim evaluator outputs with independent assessment.....	149
8.11	Verbatim tutor outputs.....	150
8.12	A rolling conversation summary produced by the memory manager.....	151
8.13	Observed response-time behaviour.....	152
8.14	Measured token use and cost per call, by endpoint.....	153
8.15	Unit rates used by the system's own cost accounting.....	154
8.16	Modelled cost per interaction.....	154
8.17	User acceptance testing.....	155
8.18	Delivered capability against existing platforms.....	156
8.19	Objective-by-objective assessment.....	157
A.1	Glossary of technical terms used in this report.....	175
B.1	Survey instrument as administered.....	177
B.2	Q1.....	178
B.3	Q3.....	178
B.4	Q4.....	178
B.5	Q6.....	178
B.6	Q8.....	178
B.7	Q9.....	179
B.8	Q10.....	179
B.9	Q13.....	179
B.10	Q14.....	179
B.11	Q7.....	180
B.12	Q11.....	180
B.13	Q12.....	180
B.14	Q5.....	181

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AJK	Azad Jammu and Kashmir
API	Application Programming Interface
BISE	Board of Intermediate and Secondary Education
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
FBISE	Federal Board of Intermediate and Secondary Education
HTTP / HTTPS	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JSONL	JSON Lines (one JSON object per line)
LLM	Large Language Model
LRU	Least Recently Used (cache eviction policy)
MAE	Mean Absolute Error
MCQ	Multiple Choice Question
MVC	Model-View-Controller
OAuth	Open Authorisation
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SDG	Sustainable Development Goal
SPA	Single Page Application
SQL	Structured Query Language
SRS	Software Requirements Specification
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
UAT	User Acceptance Testing
UI / UX	User Interface / User Experience
UML	Unified Modeling Language

ACKNOWLEDGMENTS

All praise is due to Almighty Allah, who gave us the health, the patience and the clear mind needed to finish this work.

We are deeply grateful to our supervisor, Engr. Areeb Ahmed Mir, of the Department of Software Engineering, for his guidance through both phases of this project. He insisted that we test our assumptions on real students before fixing a design. That single instruction changed the direction of the project for the better. His review of our architecture also saved us from several costly mistakes.

We thank the Chairperson and the faculty of the Department of Software Engineering for the academic environment, the laboratory access and the review feedback that shaped this report. The comments we received at the proposal defence and at the Software Requirements Specification review were carried directly into the requirements and the architecture presented here.

We thank the forty-eight students of classes nine to twelve who answered our requirement survey, and the volunteers who took part in the usability sessions. They told us what they actually struggle with, rather than what we assumed they struggle with. That is the reason the platform looks the way it does.

Finally, we thank our families and friends for their encouragement and their patience during the long months of development, testing and writing.

Anum Sheraz
Munazza Javed
Aniqa Saghir

ABSTRACT

School education in Pakistan is organised around examination boards, each with its own syllabus, prescribed textbooks and papers. The digital help available to students is not organised that way: a search engine, a video site or a general chatbot answers every student alike, whatever their board. In a survey of forty-eight students run for this project, 79.2 per cent said online material differs from what their board examines, and 52.1 per cent said past papers are hard to find.

This report presents Eternal Sunshine, a web platform that ties learning, testing and study resources to a named board, class and subject. It has three tiers: a React interface, a Laravel server and a separate FastAPI service for the artificial-intelligence work. Its core is Retrieval-Augmented Generation. Board textbooks, keybooks and past papers are converted to text by optical character recognition, cleaned, chunked and stored in a vector index of 8,057 passages, and every request to the model is grounded in passages fetched under a strict filter on board, class and subject. Three agents share it: a teacher that explains, a tester that sets labelled questions, and an evaluator that marks written answers.

Thirty curriculum questions all returned relevant material, and all sixty checked passages matched the requested board, class and subject, a filter precision of 1.00. Judged by an independent model with retrieval on and off, grounding rose from 1.92 to 2.75 out of 5, and retrieval never scored worse. The evaluator matched the expected mark exactly on 73.3 per cent of a fifteen-answer gold set and was within one mark on every item. Generated question sets passed all twenty-one structural checks, and the memory manager held prompt size flat, saving 37 per cent of tokens by turn ten. Built-in accounting places a tutoring exchange well under one US cent.

Grounding a small, inexpensive model in a curated, board-filtered corpus is therefore a practical route to trustworthy educational artificial intelligence in a low-resource setting. Wider board coverage, stricter grounding and a teacher dashboard are the main future work.

Keywords: educational technology; Retrieval-Augmented Generation; large language models; automated assessment; Pakistani board curriculum.

INTRODUCTION

This chapter introduces the project. It explains the problem that Pakistani school students face when they study for board examinations, gives the evidence collected from students themselves, states the aim and objectives of the work, sets the scope, and describes how the rest of the report is organised.

1.1 OVERVIEW

School education in Pakistan is built around examination boards. A student in Mirpur takes a paper set by the Azad Jammu and Kashmir Board of Intermediate and Secondary Education. A student in Islamabad takes a paper set by the Federal Board of Intermediate and Secondary Education. The two boards prescribe different textbooks. They order chapters differently, give different weight to topics, and ask questions in different styles. For the four years that shape a young person's future, classes nine to twelve, the board is not a small administrative detail (Ministry of Federal Education 2022; Federal Board of Intermediate and Secondary Education 2024). It is the single most important factor in what a student must learn and how they will be marked.

The digital help available to those students is not organised in the same way. A search engine returns pages written for the Punjab syllabus, an Indian syllabus, or no syllabus at all. Video sites host lectures of unknown origin. Since 2023, a growing number of students have turned to general chatbots such as ChatGPT. These tools answer fluently, but they answer from the whole internet rather than from the book the student will be examined on. They cannot say which questions have appeared in a particular board's papers. They write long, flowing prose where the marking scheme rewards three short sentences. They also state wrong facts with complete confidence. The research literature calls this hallucination. It is especially harmful when the reader is a fifteen-year-old who has no way to detect it.

This project attacks that gap directly. Eternal Sunshine is a web platform for learning, testing and study resources. Its central design promise is simple. Every sentence the artificial intelligence produces for a student is grounded in the real

textbooks, keybooks and past papers of that student’s own board, class and subject. The promise is kept by the architecture, not by an instruction in a prompt. The model is never asked a question until it has first been handed passages retrieved from a vector index that is filtered, at query time, on the student’s board, class and subject. The same grounded pipeline drives three abilities: explanation, question generation and answer marking. A student can therefore learn a topic, be tested on it in the style of their own board, and receive a mark with useful feedback, all from one curated set of books.

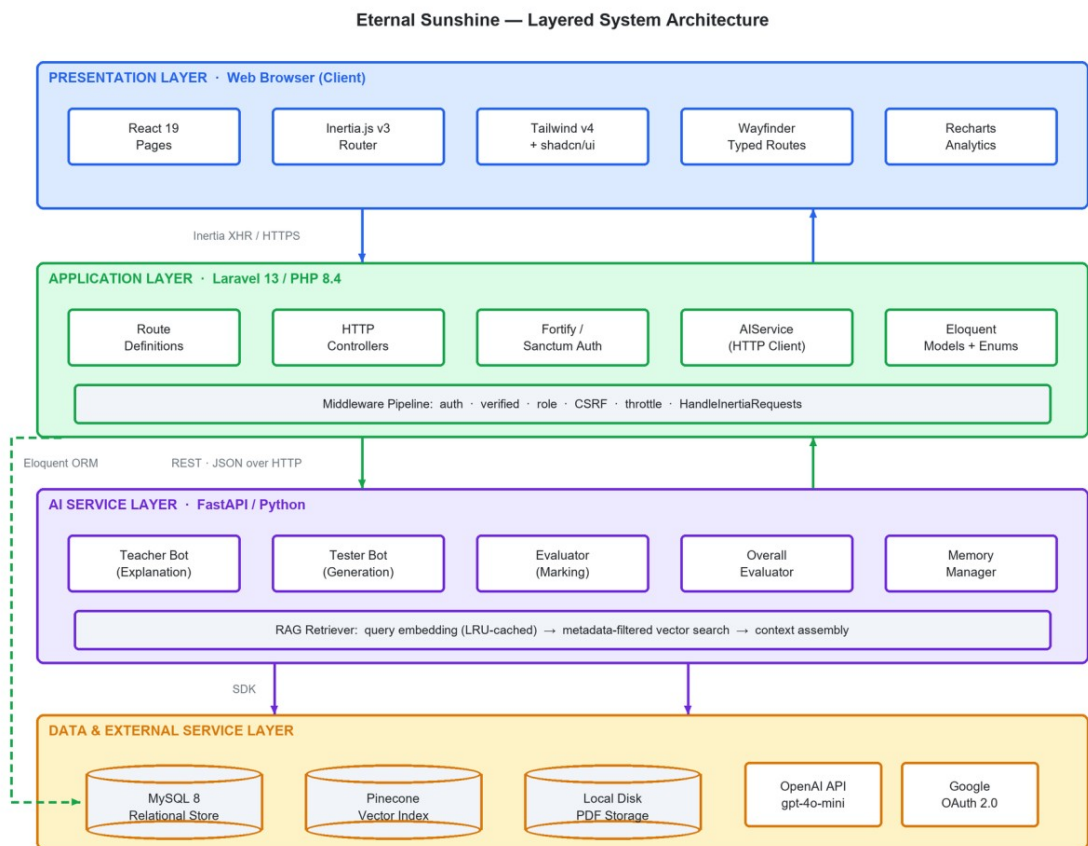


Figure 1.1: Layered architecture of the Eternal Sunshine platform.

1.2 BACKGROUND AND MOTIVATION

1.2.1 The examination-centred reality of Pakistani schooling

In practice, Pakistani secondary education is an examination-preparation exercise. Entry to a professional degree in medicine, engineering or computer science is decided almost entirely by the marks obtained in the Class 10 and Class 12 board examinations. This puts heavy pressure on a narrow set of skills. A

student must reproduce textbook definitions accurately, organise a long answer under the headings an examiner expects, and recognise the question patterns that a particular board repeats. Students who can afford a tuition academy buy exactly this. They get a tutor who knows the board, knows the paper and drills the pattern. Students who cannot afford tuition are left with a textbook and a guess paper.

The result is a problem of fairness rather than a problem of content. The textbooks exist. The past papers exist. What is scarce and expensive is the guidance that joins them together. Someone has to explain a concept in the words the book uses, say which questions matter, and mark an attempt honestly. That guidance layer is exactly what a well-grounded language model can supply, at a running cost measured in fractions of a cent.

1.2.2 Why general chatbots are not enough

During requirement gathering, students who said they used ChatGPT for study were asked what frustrated them about it. Twenty-six of the forty-five students who answered this question reported a concrete problem. Their answers group into a small number of themes, and each theme maps onto a known technical limit of general language models.

- **Answers are too long and not shaped for an exam.** Eleven students said the answers were lengthy or padded. A model tuned to be helpful writes a full essay. A board examiner gives three marks for three specific points. The student is left to do the shortening, which is the very skill they have not yet learned.
- **The wording is too difficult.** Seven students said the language was hard, or pitched above their level. One wrote that the model gives "high level answers that not suits college level students". A tutor has to match the reading level of the book in front of the student.
- **Answers are sometimes wrong or off the point.** Seven students reported wrong, vague or unrelated answers. Hallucination in open-domain generation is well documented. In a study setting the damage is one-sided. A student who cannot judge the answer will memorise the error.

Retrieval-Augmented Generation, introduced by Lewis and colleagues in 2020 (2020), answers all three complaints. The system fetches relevant passages from a trusted set of documents and places them in the model's context before it writes anything. This keeps the answer tied to material the system can point at, shortens it because the source is explicit, and greatly narrows the space in which the model can invent. The central idea of this project is that the trusted corpus for a Pakistani board student is not the internet. It is a shelf of eight or ten specific books.

1.2.3 Evidence from the requirement survey

Before any architecture was fixed, a structured survey was given to students of classes nine to twelve. Forty-eight complete responses were received. The full results appear in Chapter 3 and in Appendix B. Five findings shaped the whole design.

1. Students study mostly on their own. 45.8 per cent study mainly at home and 41.7 per cent at school, so the product must work without a teacher standing beside the student.
2. The main need is conceptual. 52.1 per cent need help with understanding concepts and a further 37.5 per cent need help with both concepts and question practice. This is why the system provides a teacher agent and a tester agent rather than only one of them.
3. Students already reach for artificial intelligence without being told to. 75 per cent said they use ChatGPT, and it was the second most common response to the question of what they do when a topic is unclear.
4. Board material and online material do not match. 50 per cent said there is a difference between board examination content and online study material, and a further 29.2 per cent said there is sometimes a difference. Only 20.8 per cent saw no difference. This is the strongest single piece of evidence for the premise of the project.
5. Official material is hard to find. 52.1 per cent said they do download past papers and textbooks but find them difficult to locate, which is the reason the platform includes a filtered resource library.

Two further results confirmed the shape of the product. 79.2 per cent said an application that gave questions and explanations based on their own board syllabus would help them, against 4.2 per cent who said it would not. The same share, 79.2 per cent, said a chatbot that tests them, shows their mistakes and explains the correct answer would help them learn better.

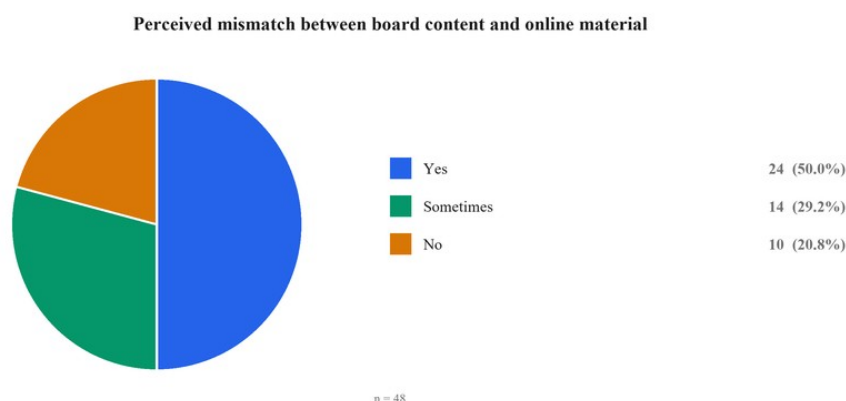


Figure 1.2: Reported mismatch between board content and online material (n = 48).

1.3 PROBLEM STATEMENT

Students of classes nine to twelve in Pakistan prepare for board-specific examinations using digital tools that are not board-specific. The material returned by search engines, video sites and general chatbots is not aligned with the textbook the student owns. It cannot tell one board's question patterns from another's. It is not shaped to the marking scheme the student will be judged against. It also carries an unmeasured risk of factual error. The guidance that would fix all of this is a tutor who explains from the prescribed book, sets questions in the board's own style and marks honestly. That guidance is available only through private tuition, which a large share of students cannot afford.

No existing system combines board-filtered curricular grounding, conversational explanation, automatic generation of board-styled practice questions and automatic marking of written answers in a single platform that a student can reach with a browser. The absence of such a system keeps an unfairness in place that is economic in origin but technical in remedy.

This project therefore addresses the following problem. How can a large language model be constrained, at acceptable cost and acceptable speed, to teach,

examine and mark strictly within the syllabus of a named Pakistani examination board, class and subject? And how can that ability be delivered through a secure, maintainable and usable web application?

1.4 AIM AND OBJECTIVES

1.4.1 Aim

To design, build, test and evaluate an intelligent, board-specific web platform that lets Pakistani students of classes nine to twelve learn, practise and be assessed in their school subjects, using artificial intelligence that is verifiably grounded in their own curriculum.

1.4.2 Objectives

The aim was broken into the measurable objectives listed in Table 1.1. Each one is revisited in Section 8.9, where the degree of achievement is judged against evidence.

Table 1.1: Project objectives and where each is addressed

ID	Objective	Addressed in
O1	Make board, class and subject the scope that drives every piece of content the student sees.	Chapters 3, 5, 6
O2	Build an offline ingestion pipeline that turns scanned board material into a metadata-tagged vector index.	Chapters 4, 6, 8
O3	Provide a retriever that applies board, class and subject as a hard filter during the search, not as a request in a prompt.	Chapters 4, 7, 8
O4	Provide a teacher agent that explains in exam language, supports English and Urdu, and is honest about what the corpus does not cover.	Chapters 4, 6, 8
O5	Provide a tester agent that writes board-styled items and labels each one with its source.	Chapters 4, 6, 8
O6	Provide an evaluator that marks written answers to the board scheme of one, three and eight marks.	Chapters 4, 6, 8
O7	Aggregate a practice session into a grade, strong and weak topics and a study recommendation.	Chapters 5, 6, 8
O8	Deliver a filtered resource library with download accounting.	Chapters 5, 6, 7
O9	Deliver a progress dashboard with subject-wise analysis.	Chapters 5, 6, 8
O10	Deliver an administrative console behind role-based access.	Chapters 5, 6, 7
O11	Contain and measure the operating cost of the artificial-intelligence tier.	Chapters 4, 6, 8

ID	Objective	Addressed in
O12	Verify the system by automated, integration, system and acceptance testing, and evaluate the retrieval and language-model layers directly.	Chapters 7, 8

1.5 SCOPE OF THE PROJECT

1.5.1 In scope

The delivered system models two examination boards, Federal and Azad Jammu and Kashmir, four class levels from nine to twelve, and seven subjects: Physics, Chemistry, Biology, Mathematics, English, Computer Science and Urdu. This gives 28 class-and-subject combinations and a curriculum map of 379 named chapters. Within that scope the platform provides account management with email verification, Google sign-in and optional two-factor authentication; an academic profile; a conversational tutor with saved, summarised session memory and voice input; chapter-scoped practice generation in three question formats; automatic marking and session-level evaluation; a resource library; a progress dashboard; and an administrative console.

Scope of the indexed corpus

The board, class and subject model supports both boards, and the interface offers both. The indexed corpus delivered with this project covers the Federal Board only, across 27 class-and-subject combinations and 8,057 chunks. Adding the AJK Board is a data-ingestion task rather than a code change, because the ingestion scripts, the metadata schema and the retrieval filter already carry the board field. Section 8.2 reports the measured corpus and Section 9.5 states this as a limitation.

1.5.2 Out of scope

The following were left out on purpose, either because they were beyond the time available or because they bring duties the project cannot responsibly meet. They are revisited as future work in Section 9.6.

- **Native mobile applications.** The interface is responsive and works in a mobile browser, but no Android or iOS package is produced.

- **Spoken replies.** Voice input is delivered, using the browser speech-recognition interface, so a student can dictate a question. The system does not read its answers aloud.
- **Offline operation.** Both retrieval and generation need network access to third-party services.
- **Boards beyond Federal and AJK.** The Punjab, Sindh, Khyber Pakhtunkhwa and Balochistan boards are supported by the design. Adding one is a data-ingestion exercise, not a change to the code.
- **Redistribution of copyrighted material.** Only documents that are already lawfully and publicly available are offered for download. The indexed corpus is used for retrieval only and is never served back as a whole document.
- **Teacher and parent portals.** The system models two roles, student and administrator. Classroom management is not addressed.

1.6 SIGNIFICANCE AND CONTRIBUTIONS

The contribution of this project is not the invention of Retrieval-Augmented Generation, and it is not the training of a new model. It is the demonstration that a carefully engineered retrieval layer over a small, deliberately chosen corpus can turn a cheap general-purpose model into a credible subject tutor for a curriculum that no commercial provider serves. The specific contributions are listed below.

1. A board, class and subject filtered retrieval design in which curricular scoping is a hard constraint on the vector query rather than a request made in a prompt. Section 8.3 shows this filter holding at a measured precision of 1.00 over sixty inspected passages, with no leakage across boards or classes.
2. An ingestion pipeline tuned to the particular noise found in Pakistani board material: scanned pages that need optical character recognition, running headers, watermarks and examination-paper furniture.
3. A three-agent split into teacher, tester and evaluator that shares one retrieval layer but differs in prompt, temperature and token budget, showing that one model can be specialised by configuration rather than by training.

4. A labelling scheme for generated questions that separates items taken from past papers, items modelled on past-paper style, and items newly written from the textbook.
5. A conversation-memory strategy that bounds prompt growth, so the per-turn cost of a tutoring session stays roughly constant instead of rising with its length. Section 8.6 measures the saving.
6. Per-call token and cost instrumentation that turns the economics of the system into a measured, engineered property. This is a precondition for any claim that the platform is affordable at scale.
7. A reproducible evaluation harness for the retrieval and language-model layers, reported in Chapters 7 and 8, which tests grounding, marking accuracy, question validity and prompt growth against recorded baselines rather than by demonstration alone.

1.7 ALIGNMENT WITH THE SUSTAINABLE DEVELOPMENT GOALS

The project was mapped against the United Nations Sustainable Development Goals during the proposal phase. The mapping is repeated here with the delivered features that realise each goal, so that the claim can be checked rather than merely asserted.

Table 1.2: Sustainable Development Goals and the features that meet them

SDG	Goal	How the delivered system contributes
SDG 4	Quality Education	Free, board-aligned explanation, practice and marking for classes nine to twelve, available to any student with a browser.
SDG 10	Reduced Inequalities	Supplies the guidance layer that is otherwise bought through private tuition, at a running cost of well under one cent per exchange.
SDG 9	Industry, Innovation and Infrastructure	Demonstrates a low-cost, locally relevant application of retrieval-augmented artificial intelligence built on standard cloud services.

1.8 COMPLEX ENGINEERING PROBLEM ATTRIBUTES

The project was assessed against the complex engineering problem attributes required of a final year project in an accredited engineering programme. Table 1.3 records each attribute with the specific part of this system that shows it.

Table 1.3: Complex engineering problem attributes shown by the project

Attribute	How it appears in this project
Depth of knowledge	Requires information retrieval, vector embeddings, prompt design, relational modelling, web security and cloud cost control.
Range of conflicting requirements	Answer quality pulls towards longer prompts and larger models, while cost and latency pull the other way. The memory manager and the retrieval budget are the negotiated settlement.
Depth of analysis	No standard answer exists for how much curricular context to retrieve, or how to mark free text against a board scheme. Both were settled by measurement, reported in Chapter 8.
Familiarity of issues	Grounding a language model in a scanned, low-resource, non-English-first curriculum is not a routine problem with a published recipe.
Extent of applicable codes	Copyright over board material, protection of minors' data and examination integrity all constrain the design.
Stakeholder involvement	Students, teachers, the supervisor and the examination boards have different and partly opposed interests.
Interdependence	The system spans four tiers and two external providers. A change in the chunking strategy alters retrieval quality, prompt size, cost and answer quality together.

1.9 REPORT ORGANISATION

The rest of this report is organised as follows.

- **Chapter 2, Literature Review.** Surveys artificial intelligence in education, intelligent tutoring systems, conversational agents, large language models and their limits, retrieval-augmented generation, vector search, automatic marking and the Pakistani educational technology market, and states the research gap this project fills.
- **Chapter 3, Requirement Analysis and Methodology.** Describes the Agile process followed, the survey and its results, the stakeholder and user analysis, the functional and non-functional requirements, the behavioural models taken from the Software Requirements Specification, the feasibility study and the traceability matrix.

- **Chapter 4, System Architecture.** Presents the architectural approach, the subsystem split, the data flow diagrams, the design of the artificial-intelligence subsystem, the ingestion pipeline, the request-time retrieval path, the security model and the deployment view.
- **Chapter 5, Detailed System Design.** Gives the database design and data dictionary, the class model, a ten-attribute specification of each software component, the interface design and the prompt design.
- **Chapter 6, Implementation.** Documents the development environment, the repository and its history, and the module-by-module implementation with annotated code listings.
- **Chapter 7, System Testing.** States the testing strategy, reports the automated suite, presents the functional and non-functional test cases, and describes the dedicated retrieval and language-model evaluation harness.
- **Chapter 8, Results and Evaluation.** Presents the working system, the measured corpus, the retrieval results, the grounding and marking experiments, the performance and cost measurements, the acceptance testing and the assessment against the objectives.
- **Chapter 9, Conclusion and Future Work.** Summarises the contribution, states the strengths and limitations openly, gives the cost position and sets out future work.

References follow in Chicago author-date style. Five appendices give the glossary, the survey instrument and full results, additional code listings, a user manual and the similarity report.

LITERATURE REVIEW

This chapter reviews the work this project builds on. It moves from the broad field of artificial intelligence in education to the specific techniques the system uses: intelligent tutoring, conversational agents, large language models and their limits, retrieval-augmented generation, vector search, automatic marking, and how such systems are evaluated. It then surveys what is actually available to a Pakistani school student today, and states the gap this project fills.

2.1 INTRODUCTION

This chapter surveys the body of work on which the project rests. It is organised as a narrowing argument. Sections 2.2 to 2.4 establish what is already known about using computers to tutor, beginning with Bloom's demonstration of the power of one-to-one instruction and ending with the modern generation of conversational agents. Sections 2.5 to 2.7 examine large language models, the technology that makes conversational tutoring newly plausible. They also cover the failure mode that makes those models dangerous in a classroom, and the retrieval architecture that contains it. Section 2.8 reviews automatic marking of free-text answers, which is the second capability the system must provide. Section 2.9 covers the document-processing problem specific to a low-resource corpus of scanned Pakistani board material. Section 2.10 surveys the educational-technology products actually available to a Pakistani secondary student today, and Section 2.11 compares them systematically. Section 2.12 states the resulting research gap, which this project occupies.

2.2 ARTIFICIAL INTELLIGENCE IN EDUCATION

The empirical case for individual tutoring predates computing in education. Bloom's study of mastery learning reported that students taught one-to-one performed approximately two standard deviations better than students taught in a conventional classroom (Bloom 1984). The finding — universally known as the two-sigma problem — framed an agenda that has occupied educational technology ever since: the effect of individual tutoring is enormous, and individual tutoring is unaffordable at population scale. Every subsequent generation of educational

technology can be read as an attempt to recover some fraction of that two-sigma effect at a cost per student that a state or a family can bear.

Holmes, Bialik and Fadel survey the modern field and separate three roles that artificial intelligence plays in education (Holmes, Bialik, and Fadel 2019; Luckin et al. 2016). In learning with AI the system delivers the instruction. In learning about AI the technology is itself the subject. In learning through AI, analytics inform a human teacher. This project sits firmly in the first category. Zawacki-Richter and colleagues reviewed 146 studies of artificial intelligence in higher education (2019). They found that applications fall into four groups: profiling and prediction, assessment and evaluation, adaptive and personalised systems, and intelligent tutoring systems. They also noted two persistent weaknesses. The field reflects too little on what its tools mean for teaching and ethics, and almost all of the work comes from well-resourced institutions. Nye's review of intelligent tutoring in the developing world identifies exactly the constraints this project operates under: intermittent connectivity, low-specification devices, limited local content and the absence of trained facilitators (Nye 2015).

UNESCO's guidance for policy-makers is explicit that equity must be the organising principle, not an afterthought (2021). It warns that systems trained and tested on the material of wealthy education systems tend to serve everyone else badly. That warning is directly applicable here: a general-purpose model has seen an enormous quantity of American and European curricular material and comparatively little Pakistani board material. The remedy adopted in this project is not to retrain the model but to supply the missing material at inference time.

Pardos and Heffernan situate the contemporary opportunity in the availability of large-scale learner interaction data and the maturation of machine-learning methods that can exploit it (Pardos and Heffernan 2020). Rahman, Khan and Iqbal examine the specific case of Pakistan and identify infrastructure, teacher preparedness and the absence of localised content as the principal obstacles to adoption (Rahman, Khan, and Iqbal 2023). The present work addresses the third of these directly.

2.3 INTELLIGENT TUTORING SYSTEMS

An intelligent tutoring system is usually described as four components. A domain model holds what is to be learned. A student model holds what this learner currently knows. A tutoring model holds the teaching strategy. The fourth is the user interface. Anderson and colleagues' cognitive tutors, built on the ACT-R theory of cognition, remain the canonical implementation and were deployed at scale in American algebra classrooms (Anderson et al. 1995). Their student model rested on knowledge tracing, introduced by Corbett and Anderson (1994). It holds a probability that the learner has mastered each knowledge component, and updates that probability after every answer. Piech and colleagues later showed that a recurrent neural network could learn this mapping directly from interaction sequences, an approach known as deep knowledge tracing (Piech et al. 2015).

VanLehn's meta-analysis is the most useful single result for calibrating expectations. He compared human tutoring, intelligent tutoring systems and computer-aided instruction. Step-based tutoring systems came close to human tutors, and were considerably better than answer-based systems that only mark a final answer right or wrong (VanLehn 2011). The operative variable is granularity of feedback: systems that respond to the steps of a solution outperform systems that respond only to its conclusion.

This finding shaped two decisions in the present design. First, the evaluator never returns a bare score. It is told to state in one sentence what was correct and in one sentence what was missing. For mathematics it must name the exact step where the reasoning failed. Second, the session-level evaluator aggregates an attempt into named strong and weak topics rather than a single percentage, so that feedback points at content rather than at performance. What this project does not attempt is a formal student model in the knowledge-tracing sense: the mastery estimate is the empirical distribution of past scores by subject, not a Bayesian latent variable. This is a deliberate scope limitation and is revisited in Section 9.6.

2.4 CONVERSATIONAL AGENTS IN EDUCATION

Conversational interfaces have a long history in this domain, beginning with Weizenbaum's ELIZA, whose pattern-matching Rogerian therapist

demonstrated in 1966 how readily people attribute understanding to a system that produces plausible conversational turns (Weizenbaum 1966). Adamopoulou and Moussiades trace the subsequent development from rule-based and retrieval-based designs to generative ones, and note that the field's persistent difficulty has been maintaining coherence and factual reliability across a long exchange (Adamopoulou and Moussiades 2020).

Winkler and Söllner's state-of-the-art analysis of chatbots in education is particularly relevant because it is critical rather than promotional. Reviewing eighty studies, they report that most educational chatbots are narrow, flow-based systems. They fail as soon as a learner leaves the expected script. The studies also tend to measure satisfaction rather than learning (Winkler and Söllner 2018). Kasneci and colleagues wrote after the release of large instruction-tuned models (2023). They argue that these systems remove the brittleness problem but bring new ones: unreliable facts, reasoning that cannot be inspected, bias inherited from training data, and the risk that learners hand over their thinking instead of developing it. Baidoo-Anu and Ansah reach a similar conclusion and recommend that generative systems in education be constrained to curated content and paired with assessment that rewards process (Baidoo-Anu and Ansah 2023).

The design adopted here follows that recommendation. The tutor is generative, so it does not break when a student asks an unexpected question. Retrieval holds it to curated content. An assessment loop then requires the student to produce an answer before any feedback is given.

2.5 LARGE LANGUAGE MODELS AND THEIR LIMITATIONS

The transformer architecture introduced by Vaswani and colleagues replaced recurrence with self-attention and made it practical to train language models at a scale previously unreachable (2017). Devlin and colleagues showed with BERT that bidirectional pre-training, followed by fine-tuning on a specific task, gave the best results then available across many language-understanding tasks (Devlin et al. 2019). Minaee and colleagues record how completely these methods replaced the earlier hand-engineered approaches to text classification (2021).

Brown and colleagues then demonstrated with GPT-3 that a sufficiently large autoregressive model performs many tasks from a natural-language description and a handful of examples, without any gradient update (Brown et al. 2020). This capability is called in-context learning, and it is what puts the present project within reach of a final-year team. The system gets a subject tutor, a question generator and an examiner out of one model by giving it three different prompts. No training infrastructure is needed at all. Ouyang and colleagues later showed that reinforcement learning from human feedback greatly improves how closely a model follows instructions (2022), a finding echoed in related prompting research (Wei et al. 2022). That property is exactly what this system relies on when it requires the evaluator to return a strictly formatted score line.

Against these capabilities stands a well-characterised failure. Ji and colleagues survey hallucination in natural language generation (2023). They define it as content that is unfaithful to the supplied source, or that no world knowledge supports. They trace its causes to both the training data and the decoding step. The GPT-4 technical report acknowledges that hallucination persists in current frontier models, though at reduced frequency (OpenAI 2023). In an educational setting the consequence is unusually severe: the reader is a novice, the output is fluent, and the whole purpose of the interaction is that the reader should commit the content to memory.

Two mitigations are available. Decoding can be made more conservative by lowering the sampling temperature, and the model can be given the correct material in its context window. This project uses both, with temperature values chosen per agent — 0.3 for explanation, 0.4 for question generation and 0.2 for marking — and with retrieval supplying the material.

2.6 RETRIEVAL-AUGMENTED GENERATION

Lewis and colleagues introduced Retrieval-Augmented Generation as a hybrid of a parametric model and a non-parametric memory (2020). A retriever selects passages from an external index, and a generator writes its answer conditioned on them. Knowledge is then updated by changing the index rather than by retraining the model. Karpukhin and colleagues showed with Dense Passage

Retrieval that a dual-encoder trained to embed questions and passages into a shared space outperforms the long-dominant sparse lexical baseline BM25 for open-domain question answering (Karpukhin et al. 2020; Robertson and Zaragoza 2009). Reimers and Gurevych's Sentence-BERT made such embeddings cheap enough to compute at scale (2019). Gao and colleagues' recent survey organises the resulting design space into naive, advanced and modular variants and identifies chunking strategy, retrieval granularity and context assembly as the principal engineering levers (Gao et al. 2024).

The architecture is a good fit for the present problem for four reasons that are worth stating explicitly, because they justify the central design decision of the project.

1. **Verifiability.** Every generated answer can be traced to retrieved excerpts, which means the claim that the system answers from the board textbook is auditable rather than rhetorical.
2. **Updatability.** When a board revises a textbook, the corpus is re-ingested. No model is retrained and no code changes.
3. **Access control by construction.** Because retrieval is filtered on metadata, curricular scoping is enforced by the query rather than requested in a prompt. A Class 9 student cannot be answered from a Class 12 textbook even if the model would prefer to.
4. **Economy.** A compact, inexpensive model supplied with the right two thousand words of context outperforms a far larger model supplied with none, at a fraction of the cost per request.

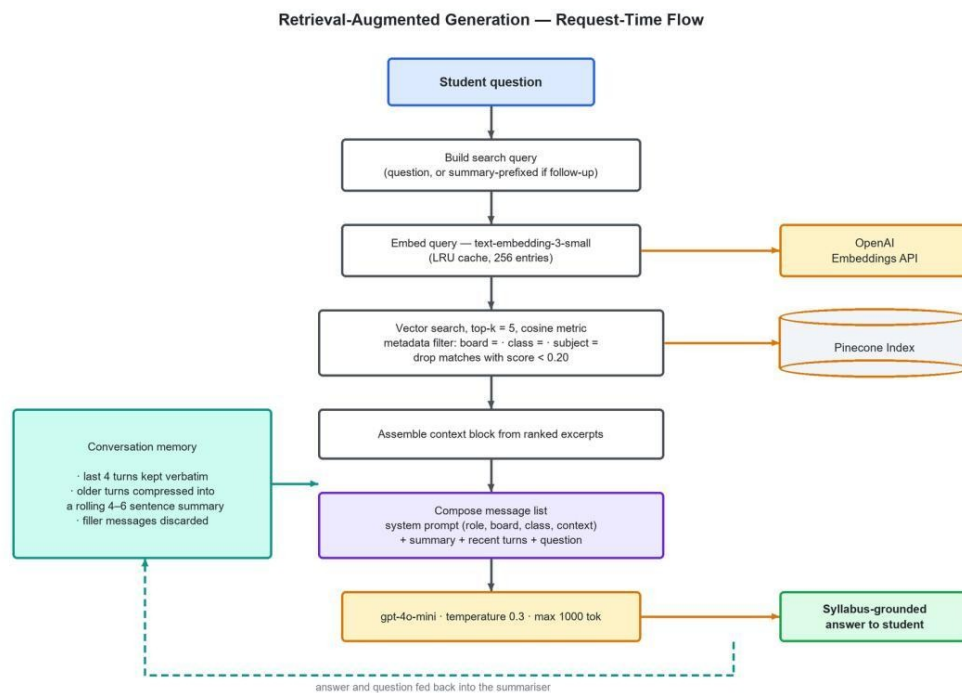


Figure 2.1: The Retrieval-Augmented Generation request path implemented in this project. The metadata filter applied to the vector query is what makes curricular scoping a structural guarantee rather than a prompt instruction.

2.7 VECTOR DATABASES AND SEMANTIC SEARCH

Retrieval at interactive latency requires an index structure that answers nearest-neighbour queries in high-dimensional space without exhaustive comparison. Johnson, Douze and Jégou's work on billion-scale similarity search established the practicality of GPU-accelerated approximate search and produced the FAISS library that underpins much of the field (Johnson, Douze, and Jégou 2021). Malkov and Yashunin's Hierarchical Navigable Small World graphs give logarithmic-time approximate search with high recall and are the algorithm behind most current managed vector services (Malkov and Yashunin 2020).

Managed services such as Pinecone package these structures behind an API and add the feature that matters most for this project: metadata filtering applied during the search rather than after it (Pinecone Systems 2024). A post-filter would first retrieve the k nearest neighbours globally and then discard those from the wrong class, frequently returning nothing. A pre-filter restricts the candidate set to vectors whose metadata matches before ranking, so the k results returned are the k

best results within the student's own syllabus. This distinction is the reason the project uses a managed vector service rather than an in-process index.

The embedding model chosen, text-embedding-3-small, produces 1,536-dimensional vectors at roughly two United States cents per million tokens. Embedding an entire board corpus is therefore a one-off cost of a few dollars. Cosine similarity is used as the distance metric, which is standard for normalised text embeddings.

2.8 AUTOMATED ASSESSMENT OF FREE-TEXT ANSWERS

Automatic grading is older than the personal computer. Page's Project Essay Grade, reported in 1966, predicted human essay scores from surface features such as word length and sentence count (1966). Burrows, Gurevych and Stein survey automatic short answer grading and divide the following half-century into five eras: concept mapping, information extraction, corpus-based methods, machine learning and evaluation (Burrows, Gurevych, and Stein 2015). They observe that until recently every approach needed a large set of human-marked answers for the specific question being graded. That requirement is fatal for the present application: the questions are generated on demand and have never been marked by anybody.

Instruction-tuned large language models remove the requirement. Mizumoto and Eguchi evaluated automated essay scoring with a general-purpose model and reported correlations with human raters that, while below the best purpose-built systems, were achieved with no training data at all (Mizumoto and Eguchi 2023). Board examinations ask for short, structured answers, and their marking scheme is a list of expected points rather than a judgement on the quality of the prose. That makes the task considerably easier than essay scoring.

The design adopted here reflects three lessons from this literature. The marking rubric is stated in the prompt in the board's own terms: one mark for a multiple-choice item, three for a short question and eight for a long question. Each score band carries its own description. The evaluator is given the retrieved textbook passage and instructed to mark against it rather than against its own knowledge. The numeric score is never trusted blindly. It is parsed from a structured response

line, then clamped in application code to the maximum allowed for that question type. A model error cannot award nine marks out of three.

2.9 EVALUATING RETRIEVAL-AUGMENTED SYSTEMS

A system that retrieves before it generates has two places where it can fail, and they need separate measurement. The retriever can fetch the wrong passages, and the generator can ignore the right ones. Reporting a single end-to-end accuracy figure hides which of the two is at fault, so the literature has settled on measuring them apart. Gao and colleagues, in their survey of retrieval-augmented generation (2024), organise the field exactly this way and treat retrieval quality and generation faithfulness as distinct axes.

On the retrieval side the measures are inherited from classical information retrieval (Robertson and Zaragoza 2009). Precision at k asks how many of the k returned passages are relevant, and recall at k asks how many of the relevant passages were returned. Dense retrievers such as Dense Passage Retrieval (Karpukhin et al. 2020) are normally reported this way. A judgement of relevance requires either human annotation or a proxy, and for a closed corpus with known subject matter a keyword proxy is a common and defensible substitute. Similarity score alone is a weak measure, because cosine similarity is not calibrated across queries, and is best reported alongside a relevance judgement rather than in place of one (Reimers and Gurevych 2019).

On the generation side the property of interest is faithfulness, sometimes called groundedness: whether every claim in the answer is supported by the retrieved passages. Ji and colleagues separate faithfulness from factuality (2023). An answer can be true in general while still being unfaithful to its source. That is precisely the failure mode that matters for a tutor bound to a syllabus. Measuring faithfulness at scale by hand is impractical, so recent work uses a stronger model as an automatic judge. Mizumoto and Eguchi (2023) show that a language model scoring written work agrees usefully with human raters. They caution that the judge inherits the biases of its training, and is not a substitute for an examiner.

Two methodological points from this literature shape the evaluation reported in Chapter 7. The first is the need for an ablation: the contribution of

retrieval can only be seen by running the same model, on the same questions, with retrieval switched off, and comparing. Without that control a groundedness score is a number without a scale. The second is that the automatic marking literature (Burrows, Gurevych, and Stein 2015) reports agreement with human marks as both exact agreement and mean absolute error, not as accuracy alone. A marking error of one point on an eight-mark question is not the same thing as a wrong answer. Both conventions are adopted in this project.

What the literature does not supply is a benchmark for this setting. The public retrieval benchmarks are open-domain and English-language, and no dataset exists of Pakistani board questions paired with ground-truth passages from the prescribed textbooks. The evaluation in this project therefore uses a purpose-built query set and a gold set of marked answers, with the limitations of that choice stated openly in Section 7.6.3.

2.10 DOCUMENT PROCESSING FOR LOW-RESOURCE EDUCATIONAL CORPORA

The corpus this project depends on does not exist in machine-readable form. Board textbooks, keybooks and past papers circulate as scanned PDF images. They are often of poor quality and often carry publisher watermarks. Urdu and Islamiyat material adds a right-to-left script with complex ligatures. Extracting usable text is therefore an optical character recognition problem rather than a text-extraction problem.

Commercial cloud OCR services materially outperform open-source engines on degraded scans and on non-Latin scripts, and Google Cloud Vision was selected for this reason after preliminary trials. Its document-text detection endpoint returns a full-page transcription with layout preserved, which is sufficient for the downstream chunking step.

OCR output alone is not usable for retrieval. Page numbers, running headers, publisher marks, examination roll-number boxes, section instructions and recognition artefacts all become text, and because such fragments repeat across hundreds of pages they act as high-frequency noise that degrades embedding quality. The literature offers no standard recipe for this material, so a rule-based

cleaning stage was developed specifically for the corpus, with a separate and more aggressive rule set for past papers than for textbooks. The chunking strategy follows the recommendation in Gao and colleagues' survey: a moderate chunk size with overlap (2024). Here that is about 2,800 characters with a 400-character overlap, breaking at a sentence or line boundary wherever possible.

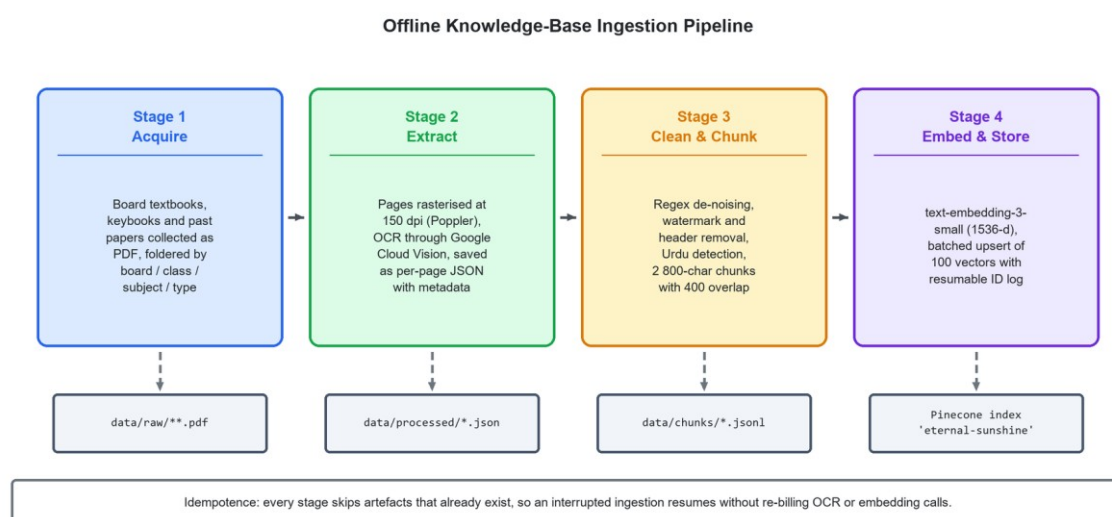


Figure 2.2: The four-stage offline ingestion pipeline that converts scanned board material into a filtered, metadata-tagged vector index.

2.11 THE EDUCATIONAL TECHNOLOGY LANDSCAPE IN PAKISTAN

A student of class nine to twelve in Pakistan has access to several categories of digital study support, none of which combines board-specific grounding with generative tutoring and automatic marking.

- **Video lecture platforms.** Sabaq Foundation and Taleemabad provide free, curriculum-mapped video lessons and are the most widely used domestic offerings. They are strong on explanation and weak on interaction: a student cannot ask a question, cannot generate a practice paper and cannot have an answer marked.
- **Past-paper repositories.** Sites such as ilmkidunya and a large number of smaller portals host board past papers and guess papers. They solve the availability problem and nothing else; the material is unstructured, of uncertain provenance and heavily advertisement-laden.

- **Test-preparation platforms.** Nearpeer and comparable services target entry-test preparation with recorded courses and question banks. Coverage of the board syllabus proper is incidental, and the model is paid.
- **Global open courseware.** Khan Academy provides high-quality instruction and an Urdu localisation, but its scope and sequence follow an American curriculum. It cannot tell a Federal Board Class 11 student what their board asks.
- **General-purpose assistants.** ChatGPT and similar systems are conversational, always available and free at the entry tier. They are also board-agnostic, unable to cite the prescribed textbook and subject to the hallucination risk discussed in Section 2.5.
- **Single-purpose solvers.** Photomath and comparable applications solve a photographed mathematics problem step by step. They are excellent within a narrow domain and do not address conceptual explanation, question generation or non-mathematical subjects.

2.12 COMPARATIVE ANALYSIS

Table 2.1 compares the platforms discussed above against the capabilities this project treats as essential. The comparison is drawn from the publicly documented feature sets of each platform at the time of writing.

Table 2.1: Comparison of existing platforms against the capability set targeted by this project

Capability	Sabaq / Taleemabad	Past-paper portals	Nearpeer	Khan Academy	ChatGPT	Eternal Sunshine
Aligned to a named Pakistani board	Partial	Yes	Partial	No	No	Yes
Class- and subject-scoped content filter	Yes	Partial	Yes	Yes	No	Yes
Conversational concept explanation	No	No	No	No	Yes	Yes
Answers grounded in the prescribed textbook	n/a	n/a	n/a	No	No	Yes
Auto-generated practice questions	No	No	Partial	Partial	Yes	Yes
Questions labelled by past-paper provenance	No	No	No	No	No	Yes
Automatic marking of free-text answers	No	No	No	No	Partial	Yes

Capability	Sabaq / Taleemabad	Past-paper portals	Nearpeer	Khan Academy	ChatGPT	Eternal Sunshine
Board marking scheme (1 / 3 / 8 marks)	No	No	No	No	No	Yes
Weak-topic identification per session	No	No	Partial	Yes	No	Yes
Downloadable past papers and textbooks	Partial	Yes	No	No	No	Yes
Urdu-language support	Yes	Yes	Partial	Yes	Yes	Yes
Free to the student	Yes	Yes	No	Yes	Partial	Yes

Two observations follow from the table. First, no existing platform occupies the intersection of board-specific grounding and generative interaction: the products that know the board cannot converse, and the products that can converse do not know the board. Second, three capabilities appear in the final column alone — grounding answers in the prescribed textbook, labelling generated questions by past-paper provenance, and marking free-text answers against the board's own marking scheme. These three constitute the novel contribution of the work.

2.13 SUMMARY OF THE REVIEWED LITERATURE

Table 2.2 summarises the principal works reviewed in this chapter, grouped by the theme they inform, and records what each contributes to the design of this project. The table is not a complete bibliography; the full list of fifty-two sources appears in the references.

Table 2.2: Principal reviewed literature and its bearing on this project

Theme	Key sources	What it contributes here
Artificial intelligence in education	Holmes et al. (2019); Luckin et al. (2016); UNESCO (2021); Zawacki-Richter et al. (2019)	Establishes the field and its evidence base, and warns that educational tools are often built without educators, which is why this project began with a student survey.
Intelligent tutoring systems	Bloom (1984); VanLehn (2011); Anderson et al. (1995); Corbett and Anderson (1994); Piech et al. (2015)	Bloom sets the two-sigma benchmark that motivates one-to-one tutoring; VanLehn identifies step-level feedback as the active ingredient, which the evaluator agent implements.
Conversational agents	Weizenbaum (1966); Adamopoulou and Moussiades (2020); Winkler and Söllner (2018)	Establishes the chat interface as the expected interaction model and documents where earlier rule-based agents failed.
Large language models and their limits	Vaswani et al. (2017); Devlin et al. (2019); Brown et al. (2020); Ouyang	Explains the capability the project builds on and the hallucination failure mode that

Theme	Key sources	What it contributes here
	et al. (2022); Wei et al. (2022); Ji et al. (2023)	retrieval is introduced to contain.
Retrieval-augmented generation	Lewis et al. (2020); Gao et al. (2024); Karpukhin et al. (2020)	Supplies the core architectural pattern and the separation of retrieval quality from generation faithfulness.
Vector search	Reimers and Gurevych (2019); Johnson et al. (2021); Malkov and Yashunin (2020); Robertson and Zaragoza (2009); Pinecone (2024)	Justifies dense embeddings over keyword search and identifies metadata pre-filtering as the mechanism that makes board scoping structural.
Automatic marking of free text	Page (1966); Burrows et al. (2015); Mizumoto and Eguchi (2023)	Supplies the marking measures used in Chapter 8 and the evidence that a language model can mark written answers usefully.
Document processing	Google Cloud Vision (2024); Minaee et al. (2021)	Informs the optical character recognition and cleaning stages of the ingestion pipeline.
Educational technology in Pakistan	Ministry of Federal Education (2022); FBISE (Federal Board of Intermediate and Secondary Education 2024); Rahman et al. (2023); Nye (2015)	Documents the board-centred structure of the curriculum and the absence of board-specific digital provision.
Engineering practice	Fowler (2003); Sommerville (2016); Beck et al. (2001); Schwaber and Sutherland (2020); IEEE 830 (1998); OWASP (2021)	Supplies the architectural patterns, the Agile process and the requirements and security standards followed.

2.14 RESEARCH GAP AND POSITIONING

The literature and the market together define the gap this project addresses. The pedagogical case for individualised tutoring is settled and large (Bloom 1984; VanLehn 2011). The technology for a plausible conversational tutor now exists and is inexpensive (Brown et al. 2020; Ouyang et al. 2022). Its central defect in an educational context — unverifiable, occasionally false output — has a known architectural remedy in Retrieval-Augmented Generation (Lewis et al. 2020; Gao et al. 2024). Automatic marking of short structured answers, historically blocked by the need for question-specific training data, is now achievable from a rubric stated in a prompt (Burrows, Gurevych, and Stein 2015; Mizumoto and Eguchi 2023). Every ingredient is available.

What is missing is the assembly of those ingredients around a specific, under-served curriculum. No published system, and no product available to a Pakistani secondary student, grounds a generative tutor in the textbooks and past papers of a named Pakistani examination board, generates practice questions in that

board's style with an indication of past-paper provenance, and marks free-text answers against that board's marking scheme — while also containing and measuring the operating cost sufficiently well that the service could plausibly be offered free.

This project therefore positions itself not as a contribution to the machine learning literature but as a contribution in applied software engineering: the design, implementation, verification and cost characterisation of a retrieval-grounded, multi-agent educational system for a low-resource national curriculum. The technical claims made in later chapters are correspondingly engineering claims — about architecture, correctness, latency and cost — rather than claims about model capability.

2.15 SUMMARY

Individual tutoring produces very large learning gains but is unaffordable at scale; this is the problem educational technology has pursued for four decades. Intelligent tutoring systems recovered part of the effect, and the literature identifies step-level rather than answer-level feedback as the mechanism. Conversational agents were historically too brittle to sustain an educational dialogue; large language models remove the brittleness but introduce factual unreliability. Retrieval-Augmented Generation, supported by modern vector search with metadata pre-filtering, converts that unreliability into a manageable engineering property by supplying the model with authoritative material at inference time. Automatic marking of short answers is now possible from a stated rubric. The material required to ground such a system for Pakistani board students exists only as scanned documents and must be processed through OCR and domain-specific cleaning before it can be indexed. No available platform combines these capabilities. The following chapter converts this position into a validated requirement set.

REQUIREMENT ANALYSIS AND METHODOLOGY

This chapter explains how the requirements were established and how the work was organised. It describes the development process, the survey of forty-eight students that drove the design, the stakeholders and user classes, the functional and non-functional requirements, the behavioural models carried over from the Software Requirements Specification, the feasibility study and the traceability matrix.

3.1 INTRODUCTION

This chapter documents how the project was run and how its requirements were established. Section 3.2 describes the development methodology and the sprint structure actually followed. Sections 3.3 and 3.4 present the requirement elicitation instrument and its results, which are the empirical basis for every functional decision in the system. Section 3.5 analyses the stakeholders and user classes. Sections 3.6 and 3.7 give the complete functional and non-functional requirement sets. Section 3.8 presents the behavioural models — use cases, state machine and interaction sequence. Section 3.9 reports the feasibility study, Section 3.10 the requirement traceability matrix, and Section 3.11 the tools and technologies selected together with the justification for each.

3.2 DEVELOPMENT METHODOLOGY

3.2.1 Selection of an Agile process

The project adopted an Agile, Scrum-inspired process. The choice was made on the basis of two properties of the problem rather than on fashion. First, a large part of the system's behaviour could not be specified in advance. Nobody on the team could say, before building it, what chunk size would retrieve good context from a Pakistani physics textbook. Nor could anyone say what prompt would make a model mark a short answer consistently. These were empirical questions that had to be answered by building, measuring and revising. A plan-driven waterfall process, which assumes requirements can be fixed before construction begins, would have been actively harmful (Sommerville 2016). Second, the project had a

fixed academic deadline and a fixed team size, so scope was the only variable left to adjust. An iterative process with a prioritised backlog makes that adjustment visible and deliberate rather than accidental (Beck et al. 2001; Schwaber and Sutherland 2020).

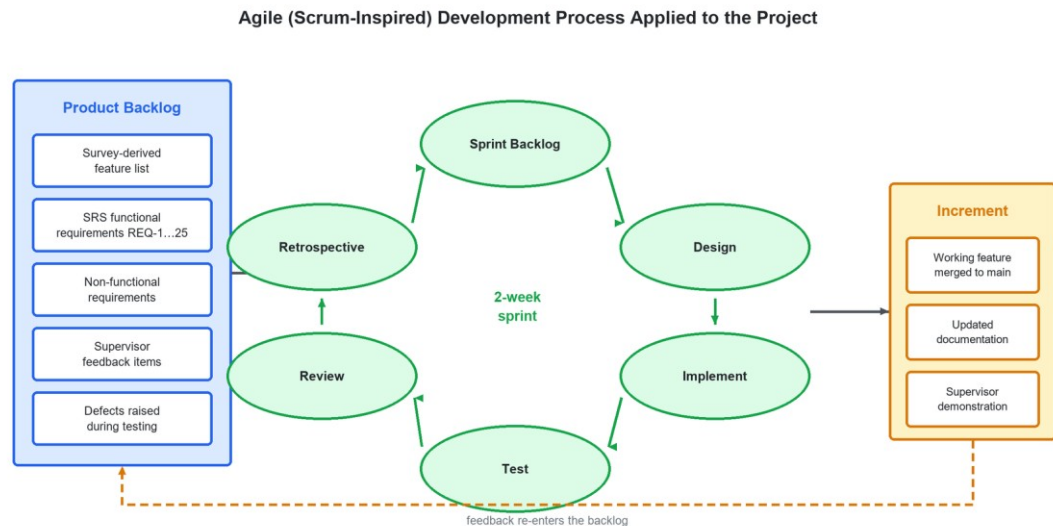


Figure 3.1: The Agile process applied to the project. Supervisor feedback and defects found during testing re-enter the product backlog, which is why the requirement set grew between the Software Requirements Specification and the delivered system.

3.2.2 Sprint structure and outcomes

Work was organised into two-week sprints. Each sprint began with planning against the backlog, ended with a demonstration to the supervisor, and produced an increment merged into the main branch. Table 3.1 records the sprints as they actually ran, with the principal outcome of each. The version history of the repository, which contains 114 commits, 89 of them excluding merges, across the development period, corroborates this schedule.

Table 3.1: Sprint plan and outcomes

Sprint	Focus	Principal outcome
S1	Requirement elicitation	Survey instrument designed and administered; 48 responses collected and analysed; module list revised against the evidence.
S2	Specification and modelling	Software Requirements Specification produced; use-case, state-machine and sequence models drawn; REQ-1 to REQ-25 baselined.
S3	Project skeleton and	Laravel 13 and Inertia React scaffold; Fortify registration,

Sprint	Focus	Principal outcome
	authentication	login, email verification, password reset and two-factor authentication.
S4	Database and academic profile	Users, chat and content schema migrated; Board, ClassLevel and Subject enumerations introduced; selection and profile pages.
S5	Corpus ingestion	Google Cloud Vision OCR extraction, cleaning and chunking scripts; first textbooks embedded into Pinecone.
S6	Teacher agent	FastAPI service, retriever with metadata filtering, teacher prompt, chat session and message persistence.
S7	Conversation memory	Memory manager with rolling summarisation; existing_summary persisted per session; prompt growth bounded.
S8	Tester agent and practice UI	Question generation for MCQ, short and long formats; provenance labelling; curriculum map of 379 chapters wired into the practice page.
S9	Evaluator and session results	Per-answer marking with score clamping; overall session evaluator; quiz_sessions and quiz_results persistence.
S10	Analytics, resources and admin	Progress dashboard, activity feed, resource library with download counting, administrative console for users and content.
S11	Optimisation and instrumentation	Embedding LRU cache; single-embed combined retrieval; per-call token and cost logging; evaluator response budget reduced.
S12	Hardening, testing and documentation	Google OAuth sign-in; automated test suite run and defects logged; system and acceptance testing; this report.

3.2.3 Version control and collaboration

The team used Git with a branch-per-concern convention. A long-lived branch carried backend work, a separate branch carried the artificial-intelligence service, and both were merged into the main branch at the end of each sprint after demonstration. Static analysis and formatting were enforced by Laravel Pint for PHP, ESLint and Prettier for TypeScript, so that review effort could be spent on logic rather than on style.

3.3 REQUIREMENT ELICITATION

Requirements were gathered in three ways that complement each other. A structured online questionnaire was given to students of classes nine to twelve and produced forty-eight complete responses. Short, informal interviews with some of those students were used to clarify free-text answers that were ambiguous. Finally,

a competitive analysis of the platforms reviewed in Section 2.10 established the baseline feature set that any credible new entrant has to meet.

Wherever possible the questionnaire asked about behaviour rather than preference. Where do you study? What do you already use? What do you find difficult? Stated preferences about products that do not yet exist are unreliable, so questions about behaviour carry more weight in the analysis that follows. Fourteen questions were asked in total, nine of them closed and five open. The full instrument and the complete results are reproduced in Appendix B. The sample was a convenience sample of students the team could reach, which is a limitation recorded in Section 8.11.

3.4 SURVEY RESULTS AND ANALYSIS

3.4.1 Respondent profile

Responses came from all four target classes. Class 12 gave 37.5 per cent of responses and Class 11 gave 35.4 per cent, followed by Class 10 at 14.6 per cent and Class 9 at 12.5 per cent. The lean towards the senior classes is expected, because examination pressure, and therefore interest in study tools, rises sharply in the final two years. It also confirmed that the system had to support the intermediate curriculum properly rather than treat it as an extension of the matriculation curriculum. Seventy-two per cent of respondents were therefore in the two classes whose results decide university entry.

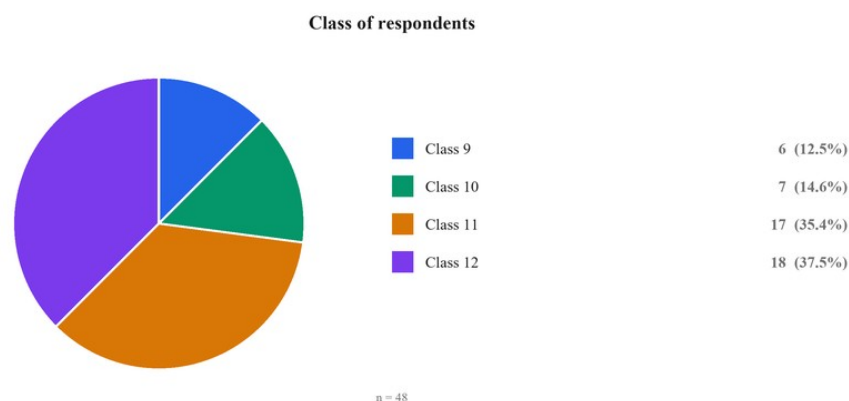


Figure 3.2: Class of survey respondents (n = 48).

3.4.2 Study environment and habits

Most students study on their own. 45.8 per cent reported studying mainly at home and 41.7 per cent mainly at school, against only 6.3 per cent at a tuition centre and 6.3 per cent who said all of these. This result influenced the design more than any other. A tool used at home, with no teacher present, has to be self-explanatory. It must answer questions rather than only present material, and it must tell the student whether their answer was right, because nobody else is there to do it.

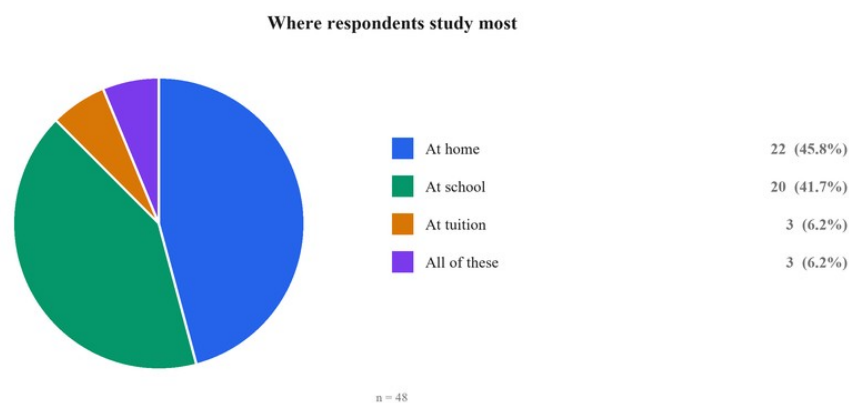


Figure 3.3: Where respondents study most (n = 48).

The students were then asked what they actually do when they do not understand a topic. More than one answer was allowed, and forty-seven students replied. Watching YouTube was the most common action, reported seventeen times, followed closely by ChatGPT with sixteen and a general web search with fifteen. Asking a teacher came fourth with eleven. Asking a friend and using notes or books tied for last with five each. The ordering is significant. For this population an artificial-intelligence assistant is already a more common first response to confusion than a human teacher.

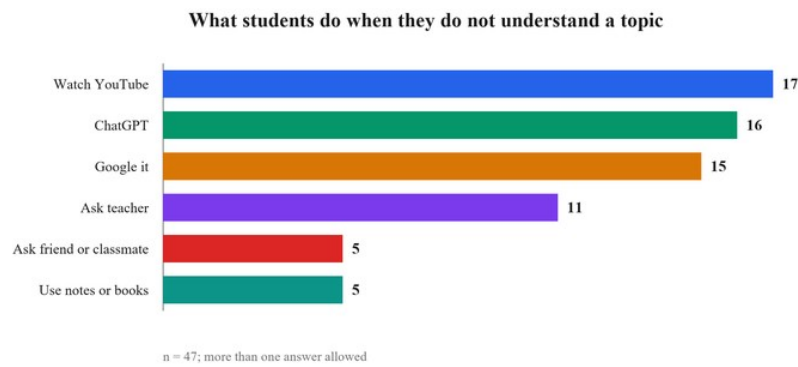


Figure 3.4: What students do when a topic is unclear (n = 47).

3.4.3 Existing tool use and its problems

Answers to the question on existing online study tools were spread widely, so the free text was grouped into three categories. The largest group, 54.2 per cent, uses no online study tool at all. A further 29.2 per cent named ChatGPT or another chatbot, and 16.7 per cent named some other tool such as YouTube, Google or a subject application. Two conclusions follow. First, a little over half of the target population is not served by any digital tool, which is the opportunity. Second, among those who are served, the incumbent is a general chatbot. That fixes the interaction model students already expect, a chat box, and it identifies the competitor the system has to beat on the one dimension that matters here, which is fidelity to the syllabus.

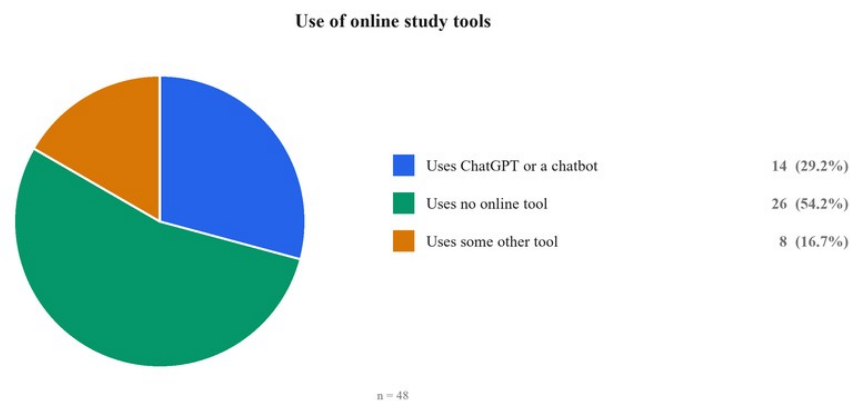


Figure 3.5: Use of online study tools (n = 48).

Asked directly, 75 per cent of respondents said they use ChatGPT. Those students were then asked what problems they face when they use it for learning. Forty-five answered, and twenty-six of them described a concrete problem; the

remaining nineteen said they had none. The free-text answers were coded into themes, and a single answer could carry more than one theme. Answers that are too long or padded was the largest theme with eleven mentions. Wording that is too difficult or pitched too high followed with seven, as did wrong, vague or unrelated answers. Three students said the model fails to understand their question, three pointed to limits of the model or its data, two reported weakness on numerical and mathematics questions, and one noted the absence of diagrams.

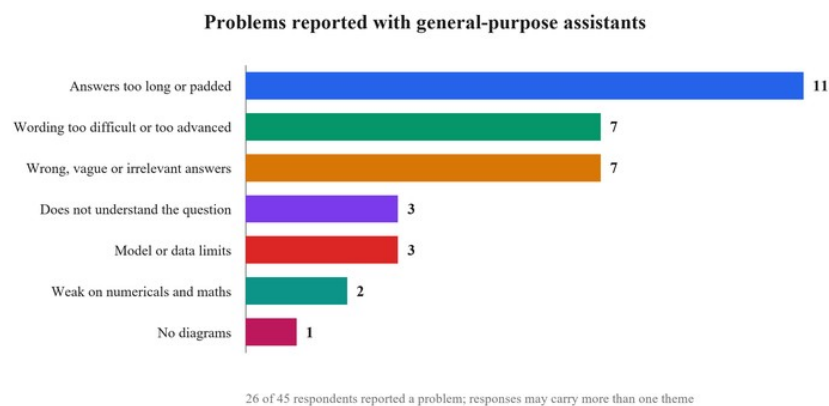


Figure 3.6: Problems reported with general-purpose assistants.

These complaints are not vague dissatisfaction. Each one maps onto a specific design decision, and together they account for four of the requirements in Section 3.6. Those are a prompt that limits answer length and reading level, retrieval that ties answers to the prescribed book, and a marking scheme that tells the student where they actually lost marks.

3.4.4 What students find difficult

Asked what they need more help with, 52.1 per cent of respondents said understanding concepts, 37.5 per cent said both understanding concepts and practising questions, 6.3 per cent said practising questions alone and 4.2 per cent were unsure. Taken together, 89.6 per cent need help with concepts and 43.8 per cent need help with practice. That distribution is the direct justification for building two separate agents rather than one. A system that only explains would fail nearly half the population, and a system that only tests would fail almost all of it.

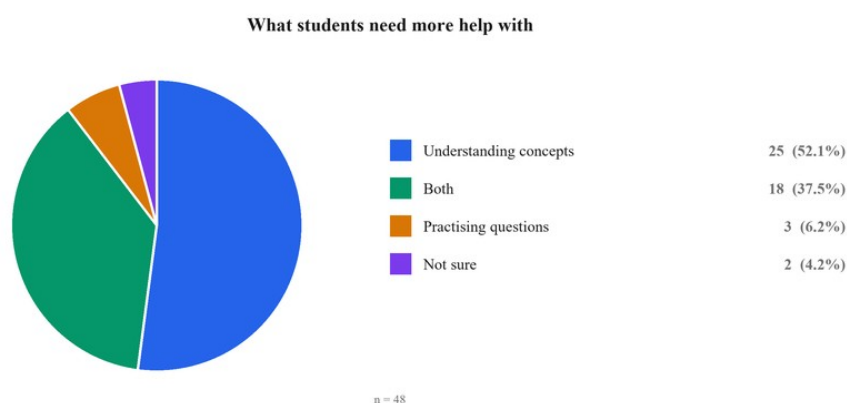


Figure 3.7: What students need more help with (n = 48).

A separate open question asked what the biggest difficulty is while studying. Forty-six students answered, and the responses were coded into themes. Understanding concepts led with fourteen mentions, followed by time management with eleven and a long syllabus with eight. Focus and distraction accounted for four, difficulty finding study material for four, and lack of guidance for three. The combination of a long syllabus and limited time is the reason the practice module allows a student to select a single chapter rather than always being tested on a whole book.

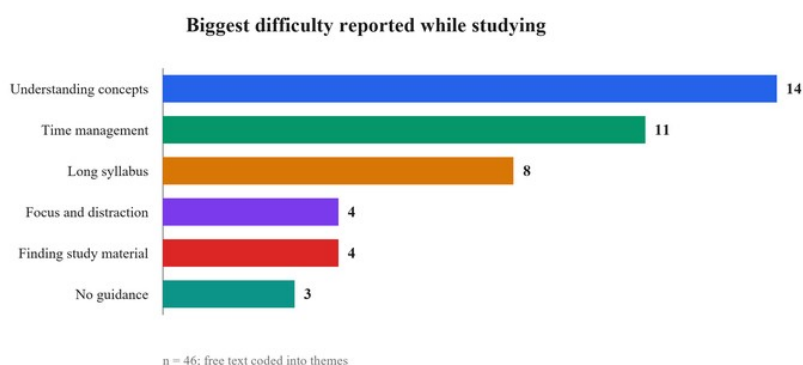


Figure 3.8: Biggest difficulty reported while studying (n = 46).

3.4.5 Demand for board-specific support

Three questions tested the premise of the project directly, and all three supported it. Asked whether board examination content differs from the study material available online, 50 per cent said yes and a further 29.2 per cent said sometimes. Only 20.8 per cent saw no difference. Nearly four students in five therefore report, unprompted, exactly the mismatch this project sets out to remove.

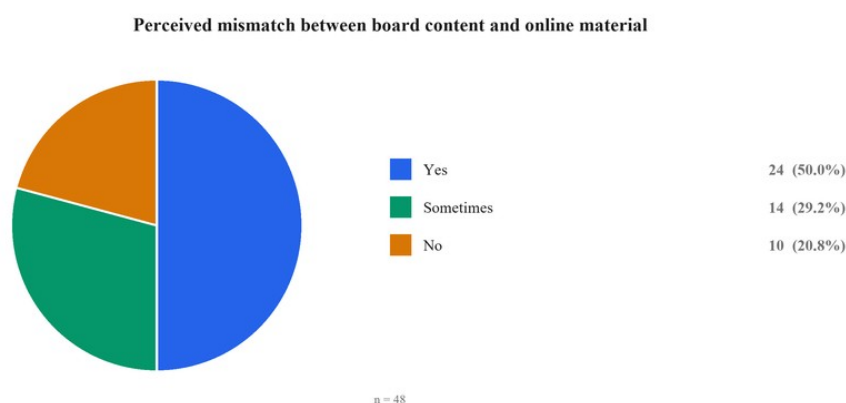


Figure 3.9: Reported mismatch between board and online material (n = 48).

Asked whether an application that gave questions and explanations based on their own board syllabus would help, 79.2 per cent said yes, 16.7 per cent said maybe and only 4.2 per cent said no. Asked whether a chatbot that tests them, shows their mistakes and explains the correct answer would help them learn better, the same 79.2 per cent said yes, 14.6 per cent said maybe and 6.3 per cent said no. A clear yes from four students in five, on two separate and specific product propositions, is a strong demand signal from a sample of this size.

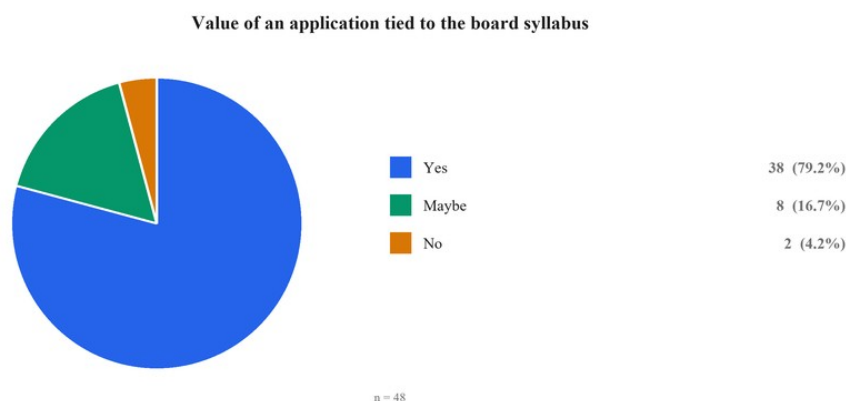


Figure 3.10: Value of an application tied to the board syllabus (n = 48).

A fourth question established the need for the resource library. Asked whether they download past papers or textbooks from the internet, 52.1 per cent said they do but find them difficult to find, 33.3 per cent said they do not download them at all, and only 14.6 per cent said the material was easy to find. The problem is therefore not that the documents are unavailable. It is that they are scattered, and students cannot filter them by their own board, class and subject.

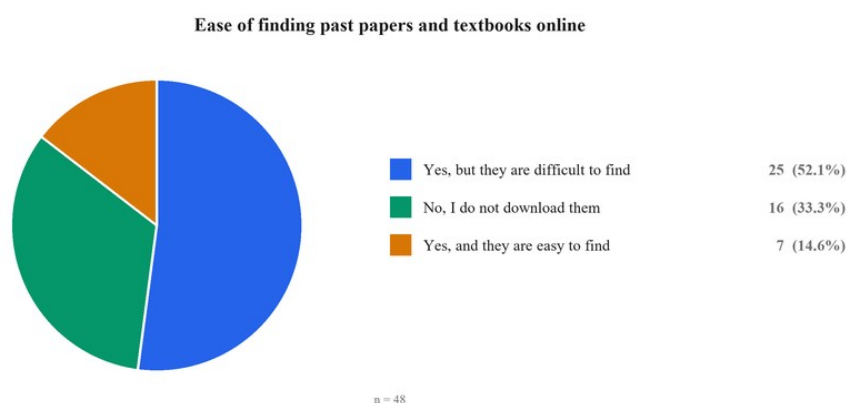


Figure 3.11: Ease of finding past papers and textbooks online (n = 48).

3.4.6 Features students asked for

The final open question asked what features a good learning application should have. All forty-eight students answered, and the free text was coded into themes. Access to past papers led with eleven mentions. Simple wording and to-the-point answers followed with ten, then keybooks and textbooks with eight and chapter videos with six. Quizzes and multiple-choice questions and voice answers tied at four each, ahead of an artificial-intelligence chatbot and correct answers at three each. Syllabus-aligned content and freedom from subscription charges drew two mentions each, and an Urdu option one.

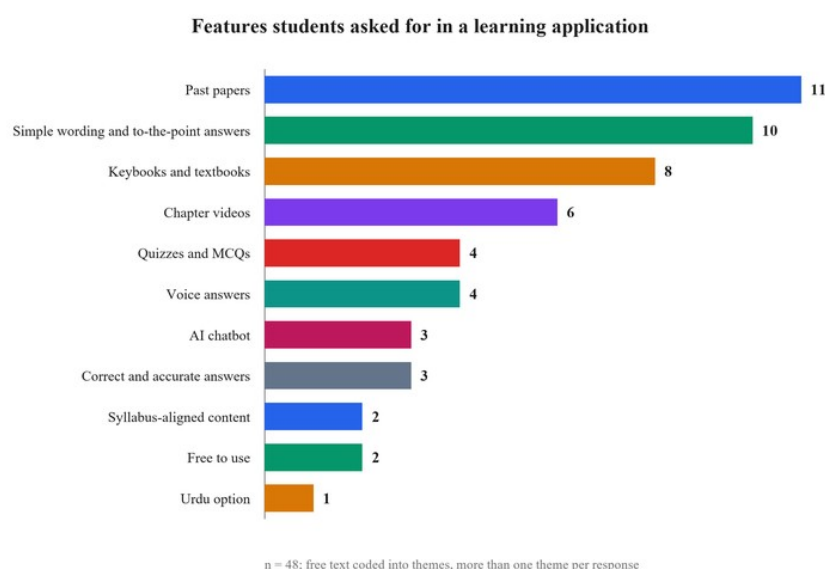


Figure 3.12: Features students asked for in a learning application (n = 48).

Two of these deserve comment. The request for voice answers, made by four students, was not anticipated when the proposal was written and was added to

the backlog during Sprint 6; voice input is delivered in the final system. The request for freedom from subscription charges confirms that cost per interaction is a product requirement and not only an engineering concern, which is why the system measures it. Six of the eleven themes map onto features that were already planned, and the platform delivers past papers, keybooks, textbooks, quizzes, an artificial-intelligence chatbot, syllabus-aligned content, free access, an Urdu option and voice input. Chapter videos are the one frequently requested feature the platform does not provide, and it is recorded as future work in Section 9.6.

3.4.7 Requirements derived from the survey

Table 3.2 traces each significant survey finding through to the design decision it forced and the requirements that record it. Every row is evidence-backed; no requirement in the table was introduced on the team’s own assumption alone.

Table 3.2: From survey findings to system requirements

Survey finding	Design consequence	Requirements
45.8 % study at home, unsupervised	The system must explain, test and mark on its own, with no teacher in the loop.	REQ-12, REQ-18, REQ-19
89.6 % need help with concepts	A conversational teacher agent grounded in the prescribed textbook.	REQ-12 to REQ-15
43.8 % also need question practice	A tester agent that generates MCQ, short and long questions.	REQ-16, REQ-17
75 % use ChatGPT; 11 complain answers are too long	The teacher prompt constrains answers to a short, exam-focused form.	REQ-15
7 complain the wording is too difficult	The prompt fixes the reading level to the class and prefers textbook wording.	REQ-14, REQ-15
7 report wrong or unrelated answers	Retrieval grounds every answer in the board corpus, and marking explains what was missed.	REQ-8 to REQ-11, REQ-19
79.2 % report a board-versus-online mismatch	Board, class and subject become a hard retrieval filter rather than a preference.	REQ-8 to REQ-11
Long syllabus and poor time management dominate difficulties	Chapter-scoped practice, so a student can drill one chapter instead of a whole book.	REQ-17, REQ-31
52.1 % find past papers hard to locate	A filtered, downloadable resource library scoped to the student’s own board and class.	REQ-21 to REQ-25
Requests for simpler and Urdu explanations	A bilingual response policy in the teacher prompt.	REQ-14
Requests for voice answers	Browser speech recognition on the tutor composer.	REQ-13
Requests for a subscription-free product	Per-call cost accounting, so affordability can be demonstrated.	REQ-32

3.5 STAKEHOLDER AND USER CLASS ANALYSIS

Table 3.3: Stakeholder analysis

Stakeholder	Interest in the system	Influence on requirements
Students of classes 9–12	Primary beneficiaries; want faster understanding and better examination results.	High — the survey population; requirements were derived directly from their reported behaviour.
Project supervisor	Academic quality, engineering rigour and defensibility of the work.	High — review feedback entered the backlog each sprint.
Department and evaluation panel	Conformance to programme learning outcomes and report standards.	Medium — shaped documentation and evaluation requirements.
Examination boards and publishers	Correct representation and lawful use of curricular material.	Medium — constrains what may be redistributed; see Section 3.7.5.
Parents	Value for money and safety of the material their child studies.	Low direct, high indirect — motivates the grounding requirement.
Platform administrator	Ability to curate content and manage accounts without developer involvement.	Medium — produced the administrative console requirements REQ-33 to REQ-38.

3.5.1 User classes

- **Student (primary).** A learner aged approximately fourteen to nineteen, enrolled in class nine to twelve under the Federal or AJK board. Basic computer literacy is assumed; technical knowledge is not. Uses the system unsupervised, frequently on a mobile browser, in sessions of ten to forty minutes. Represents the overwhelming majority of traffic and every AI-driven interaction.
- **Administrator (secondary).** A technical or academic staff member responsible for uploading and curating downloadable resources, monitoring registered accounts, and blocking or reactivating accounts. Small in number, high in privilege, and therefore the focus of the role-based access control described in Section 4.11.
- **System operator (implicit).** The person responsible for running the ingestion pipeline when a new board, class or subject is added. Interacts through command-line scripts rather than the web interface.

3.5.2 Representative personas

Table 3.4: User personas used to guide interface decisions

Persona	Situation	What the system must do for them
Ayesha, 17, Class 12 Pre-Medical, AJK Board	Studies at home after school; cannot afford a tuition academy; two months from the board examination.	Generate long questions in her board's style from her own Biology textbook, mark her written answers honestly and tell her which chapters to revise.
Bilal, 15, Class 9, Federal Board	Comfortable in Urdu, less so in English; finds the Chemistry textbook's phrasing difficult.	Explain a concept in simple language, in Urdu when he asks in Urdu, using the definitions his own book uses.
Hina, 16, Class 11, Federal Board	Strong student, short of time, wants to know what is likely to be asked.	Produce practice items labelled by past-paper provenance so she can prioritise what recurs.
Mr. Tariq, platform administrator	Non-developer; maintains the resource library each term.	Upload, categorise, edit and remove resources and manage accounts through a web console with no database access.

3.6 FUNCTIONAL REQUIREMENTS

Requirements REQ-1 to REQ-25 are those baselined in the Software Requirements Specification defended at the end of Phase I. Requirements REQ-26 to REQ-42 were added during Phase II as the implementation matured, principally to cover security hardening, administration, analytics and the artificial-intelligence service contract that the original specification treated as a single opaque feature. Both sets are listed here so that the delivered system can be assessed against a complete specification. The requirement statements follow the recommended practice of IEEE Std 830 (1998): each is uniquely identified, singular, and expressed as a verifiable “shall” statement. Priority is recorded as High, Medium or Low.

3.6.1 Authentication and profile management

Table 3.5: Functional requirements — authentication and profile

ID	Requirement	Priority
REQ-1	The system shall allow users to register using an email address and a password.	High
REQ-2	The system shall authenticate users securely during login.	High

ID	Requirement	Priority
REQ-3	The system shall allow users to register and log in using a Google account via OAuth 2.0.	High
REQ-4	The system shall allow users to update their profile information.	Medium
REQ-5	The system shall provide password-recovery functionality.	High
REQ-6	The system shall prevent unauthorised access to user data.	High
REQ-7	The system shall link a Google-authenticated account to an existing profile where the email address matches.	Medium
REQ-26	The system shall require email-address verification before granting access to learning features.	High
REQ-27	The system shall offer optional time-based one-time-password two-factor authentication with recovery codes.	Medium
REQ-28	The system shall rate-limit authentication attempts to resist credential-stuffing attacks.	High
REQ-29	The system shall require password confirmation before security-sensitive operations.	Medium

3.6.2 Academic selection and curriculum scoping

Table 3.6: Functional requirements — academic selection

ID	Requirement	Priority
REQ-8	The system shall allow a student to select their board (Federal or AJK).	High
REQ-9	The system shall allow class selection from 9 to 12.	High
REQ-10	The system shall display subject options appropriate to the selected class.	High
REQ-11	The system shall automatically scope all board-specific content — textbooks, keybooks and past papers — to the selected board, class and subject.	High
REQ-30	The system shall persist the academic selection to the user profile so that it is applied on subsequent visits.	Medium
REQ-31	The system shall expose the chapter list of the selected class and subject and allow practice to be scoped to selected chapters or to the whole book.	High

3.6.3 Conversational tutoring

Table 3.7: Functional requirements — AI tutoring

ID	Requirement	Priority
REQ-12	The system shall provide an AI chatbot for learning support.	High
REQ-13	The chatbot shall generate syllabus-aligned responses.	High

ID	Requirement	Priority
REQ-14	The chatbot shall support both English and Urdu.	Medium
REQ-15	The chatbot shall provide short, exam-focused explanations.	High
REQ-32	The system shall retrieve context from the vector index under a hard metadata filter on board, class and subject before every generation.	High
REQ-33	The system shall maintain multiple named chat sessions per user and allow a session to be resumed or deleted.	Medium
REQ-34	The system shall maintain a rolling summary of each conversation so that context is preserved without unbounded prompt growth.	Medium
REQ-35	The chatbot shall state explicitly when an answer is drawn from general knowledge rather than the prescribed textbook.	High

3.6.4 Practice, assessment and evaluation

Table 3.8: Functional requirements — practice and assessment

ID	Requirement	Priority
REQ-16	The system shall allow a student to select a practice type before starting: MCQ, short question or long question.	High
REQ-17	The system shall generate a mock test of the selected type, aligned to the student's board, class and subject.	High
REQ-18	The system shall evaluate student answers and provide detailed feedback.	High
REQ-19	The system shall identify mistakes and highlight weak areas after each practice session.	High
REQ-20	The system shall store quiz and mock-test results for progress tracking.	High
REQ-36	The system shall label each generated question according to whether it is taken from a past paper, modelled on past-paper style, or newly written from the textbook.	Medium
REQ-37	The system shall apply the board marking scheme of 1 mark per MCQ, 3 per short question and 8 per long question, and shall never award more than the permitted maximum.	High
REQ-38	The system shall produce a session-level result comprising total score, percentage, grade band, strong areas, weak areas and a study recommendation.	High

3.6.5 Resources, analytics and administration

Table 3.9: Functional requirements — resources, analytics and administration

ID	Requirement	Priority
REQ-21	The system shall allow students to filter downloadable resources by board, class and subject.	High

ID	Requirement	Priority
REQ-22	The system shall provide past papers for download for the selected board, class and subject.	High
REQ-23	The system shall provide textbooks and keybooks for download where lawfully permitted.	Medium
REQ-24	The system shall organise all downloadable resources by board, class and subject.	Medium
REQ-25	The system shall display a clear message when no resource exists for a selected combination.	Medium
REQ-39	The system shall record a download count per resource.	Low
REQ-40	The system shall present a progress dashboard showing quizzes taken, questions attempted, average score and subject-wise performance with a focus recommendation.	High
REQ-41	The system shall maintain a bounded recent-activity feed per user covering chat, quiz and download events.	Low
REQ-42	The system shall provide an administrative console, restricted by role, for uploading, editing and deleting resources and for listing, editing, blocking and deleting user accounts.	High

3.7 NON-FUNCTIONAL REQUIREMENTS

3.7.1 Performance

Table 3.10: Non-functional requirements — performance

ID	Requirement	Acceptance criterion
NFR-1	Ordinary page and API requests that do not invoke a language model shall respond quickly.	Between 1 and 3 seconds under normal load.
NFR-2	AI-generated responses shall be returned within an acceptable interactive window.	Between 2 and 5 seconds depending on query complexity.
NFR-3	Resource downloads shall begin promptly after the request.	Between 1 and 4 seconds.
NFR-4	The system shall support concurrent use without degradation.	Minimum 50 and target 500 concurrent users.
NFR-5	Prompt size for a tutoring session shall not grow without bound as the session lengthens.	Verbatim history capped at four turns; older turns compressed to a summary of at most 200 tokens.

3.7.2 Security

Table 3.11: Non-functional requirements — security

ID	Requirement	Acceptance criterion
NFR-6	Passwords shall never be stored in recoverable form.	Bcrypt hashing through the framework's hashed cast.
NFR-7	All state-changing requests shall be protected against cross-site request forgery.	CSRF token verified on every non-idempotent request.
NFR-8	Administrative capability shall be restricted by role.	Administrative routes reachable only by accounts whose role attribute is administrator.
NFR-9	Communication with third-party services shall be encrypted.	TLS enforced for all outbound API calls.
NFR-10	Secrets shall not appear in the source repository.	All credentials supplied through environment variables.
NFR-11	Two-factor secrets and recovery codes shall be stored encrypted.	Framework-level encryption at rest.

3.7.3 Usability, reliability and maintainability

Table 3.12: Non-functional requirements — quality attributes

ID	Attribute	Requirement
NFR-12	Usability	The interface shall be operable by a student with basic computer literacy and no training, and shall be responsive from a 360-pixel mobile viewport upward.
NFR-13	Reliability	The system shall target 99.5 per cent availability and shall degrade gracefully — with an explanatory message rather than an error page — when a third-party AI service is unavailable.
NFR-14	Scalability	Additional boards, classes and subjects shall be addable by ingesting data, without modification to application code.
NFR-15	Maintainability	The codebase shall be modular, statically analysed and consistently formatted, with the AI workload isolated behind an HTTP boundary so it can be replaced independently.
NFR-16	Interoperability	The system shall integrate with external identity, embedding, vector-search and OCR services through documented APIs.
NFR-17	Privacy	Personal data shall be limited to what the service requires, and student answers shall not be shared with any party other than the model provider processing the request.
NFR-18	Portability	The system shall run on Windows, Linux and macOS servers and in any modern browser.
NFR-19	Observability	Every language-model call shall be logged with its endpoint, token counts and computed monetary cost.

3.7.4 Safety

- The system shall not expose students to inappropriate or misleading content; grounding in curated curricular material is the primary control.
- The system shall not lose a student's work through accidental navigation during an active practice session.
- The system shall warn on incomplete submission rather than silently discarding an answer.

3.7.5 Legal and ethical constraints

- Only material that is lawfully and publicly available shall be offered for download. Ingested copyrighted material is used for retrieval grounding, not for redistribution.
- The system processes data belonging to minors and shall therefore collect the minimum necessary: name, email address, academic selection and learning history.
- Generated content shall be presented as study assistance rather than as an authoritative substitute for the prescribed textbook, and the tutor shall declare when it is answering from general knowledge.

3.8 BEHAVIOURAL MODELS

3.8.1 Use case model

Figure 3.13 shows the use-case model as baselined in the Software Requirements Specification. Two human actors interact with the system. The Student registers, selects board, class and subject, asks the tutor questions, practises questions, downloads resources and views performance analysis. The Administrator logs in, uploads study resources, manages users and logs out. Two external systems participate: the AI chatbot system, which generates responses and evaluations, and the database system, which stores and retrieves all persistent state. Both human actors pass through authentication before any other capability becomes reachable.

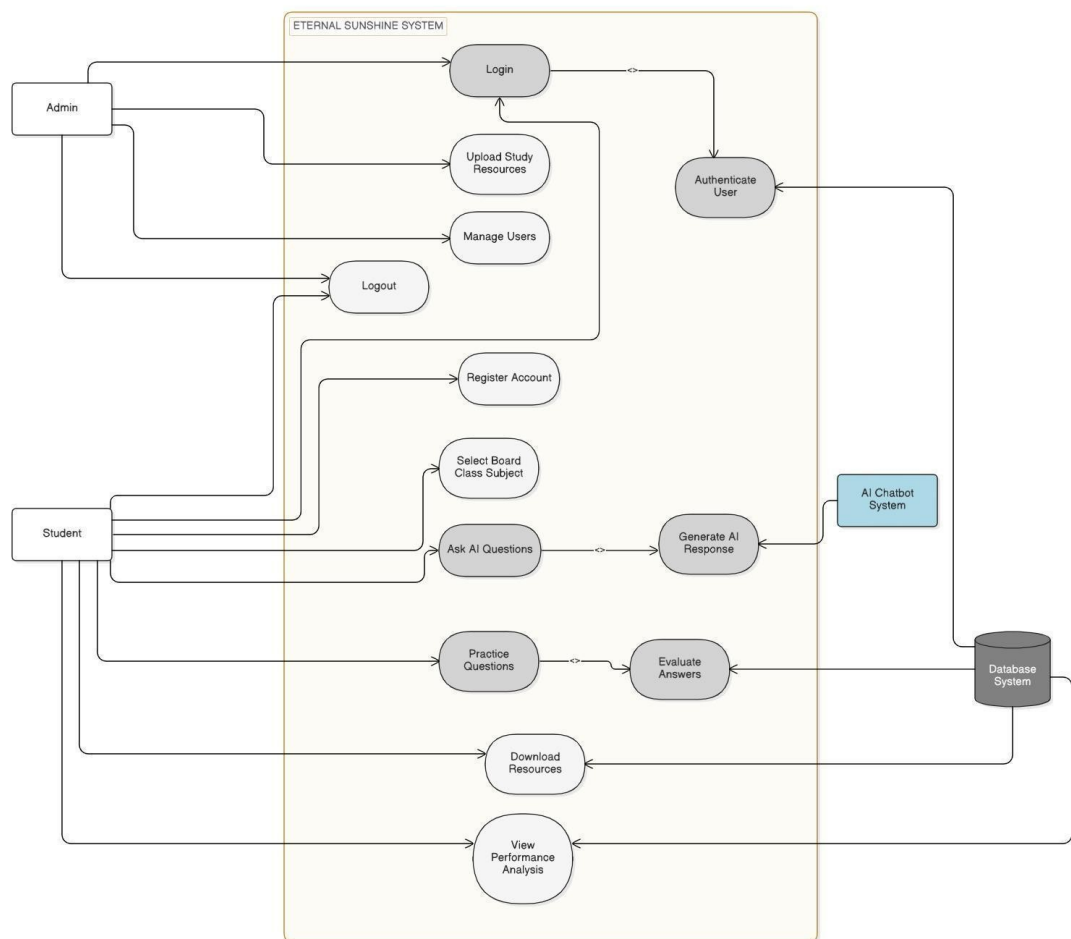


Figure 3.13: Use case diagram for the Eternal Sunshine platform, as baselined in the Software Requirements Specification.

3.8.2 Use case specifications

Four representative use cases are specified in full. The remainder follow the same template and are omitted for brevity.

Table 3.13: Use case UC-01 — Ask the AI tutor a question

Field	Description
Identifier	UC-01
Actors	Student (primary); AI chatbot system, database system (supporting)
Preconditions	The student is authenticated, their email is verified, and a board, class and subject have been selected.
Trigger	The student submits a question in the chat interface.
Main flow	1. The student types a question and submits it. 2. The system resolves the board, class and subject from the session, falling back to the profile. 3. The system loads the most recent ten messages of the session and the stored conversation summary. 4. The system persists the student's message. 5. The system calls the AI service with the question, the academic scope, the recent history and the summary. 6. The

Field	Description
	AI service embeds the query, retrieves filtered context and generates an answer. 7. The system persists the answer, updates the stored summary and records an activity entry. 8. The answer is displayed.
Alternative flows	3a. No session exists: a new session is created and titled from the first 40 characters of the question. 6a. No sufficiently similar context is retrieved: the tutor answers from general knowledge and states that it is doing so.
Exception flows	5a. The AI service is unreachable: an explanatory message is shown and the student's message remains in the transcript.
Postconditions	The exchange is persisted; the session summary reflects it; the activity feed contains an entry.
Requirements	REQ-12 to REQ-15, REQ-32 to REQ-35

Table 3.14: Use case UC-02 — Take a practice test

Field	Description
Identifier	UC-02
Actors	Student (primary); AI chatbot system, database system (supporting)
Preconditions	The student is authenticated and has a board, class and subject either in profile or selected inline.
Trigger	The student opens the practice page and starts a test.
Main flow	1. The student chooses whole-book or selected-chapter scope. 2. The student chooses MCQ, short or long format. 3. The system requests generation from the tester agent. 4. Questions are rendered one at a time. 5. For each answer submitted, the system requests evaluation and displays the score and feedback. 6. After the final question the system requests session-level evaluation. 7. The session and every answer are persisted and the summary is displayed.
Alternative flows	1a. The student selects no chapter: the whole book is used. 5a. The student leaves an answer blank: it is submitted as empty and scored zero with explanatory feedback.
Exception flows	3a. Generation fails: an error is displayed and the student may retry without losing the selection.
Postconditions	A quiz_sessions row and one quiz_results row per question exist; the dashboard reflects the new attempt.
Requirements	REQ-16 to REQ-20, REQ-31, REQ-36 to REQ-38

Table 3.15: Use case UC-03 — Download a resource

Field	Description
Identifier	UC-03
Actors	Student (primary); database system (supporting)
Preconditions	The student is authenticated.
Trigger	The student selects a resource and requests a download.
Main flow	1. The student filters the library by board, class, subject and resource type. 2. The matching resources are listed with title, type and size. 3. The student requests a

Field	Description
	download. 4. The system increments the download counter, records an activity entry and streams the file.
Alternative flows	2a. No resource matches: an explanatory empty state is shown (REQ-25).
Exception flows	4a. The stored file is missing: an error is reported and the counter is not incremented.
Postconditions	The download counter and activity feed are updated.
Requirements	REQ-21 to REQ-25, REQ-39, REQ-41

Table 3.16: Use case UC-04 — Administer content

Field	Description
Identifier	UC-04
Actors	Administrator (primary); database system, file storage (supporting)
Preconditions	The actor is authenticated and holds the administrator role.
Trigger	The administrator opens the content console.
Main flow	1. The console lists all resources with filters. 2. The administrator uploads a file with title, type, board, class and subject. 3. The system validates the request and the file size limit, stores the file on the public disk and creates a content record. 4. The administrator may subsequently edit metadata or delete a resource, in which case the stored file is also removed.
Alternative flows	2a. Validation fails: field-level errors are returned and no record is created.
Exception flows	3a. Storage is unavailable: the upload is rejected and no orphan record is created.
Postconditions	The resource library reflects the change immediately for all students.
Requirements	REQ-42

3.8.3 State machine model

Figure 3.14 shows the state transition model for a user session. The user moves between registration, login, the authenticated dashboard, learning, practice, evaluation and logout in response to their own actions and the system's responses. The model was used to verify that every state has a defined exit and that no learning state is reachable without passing through authentication.

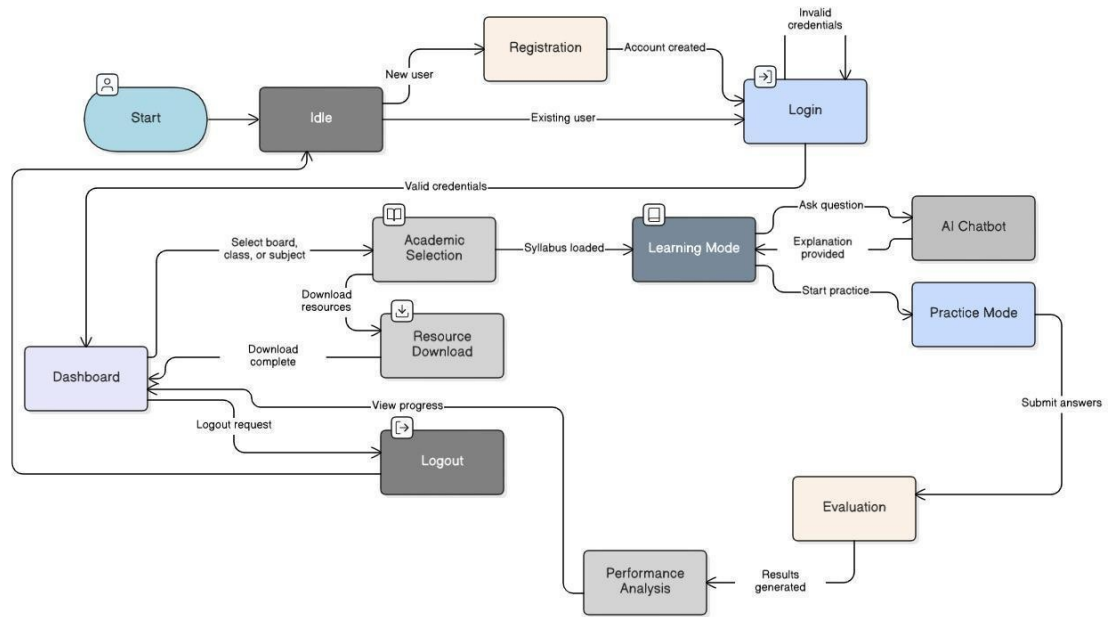


Figure 3.14: State transition diagram for a user session, as baselined in the Software Requirements Specification.

3.8.4 Interaction sequence model

Figure 3.15 shows the principal interaction sequences: login and authentication, content loading after academic selection, the AI question loop, the practice and evaluation flow, resource download and logout. The loop fragment around the question exchange captures an important fact. A tutoring session is any number of turns against one persistent session record. That observation later drove the conversation-memory design of Section 4.9.

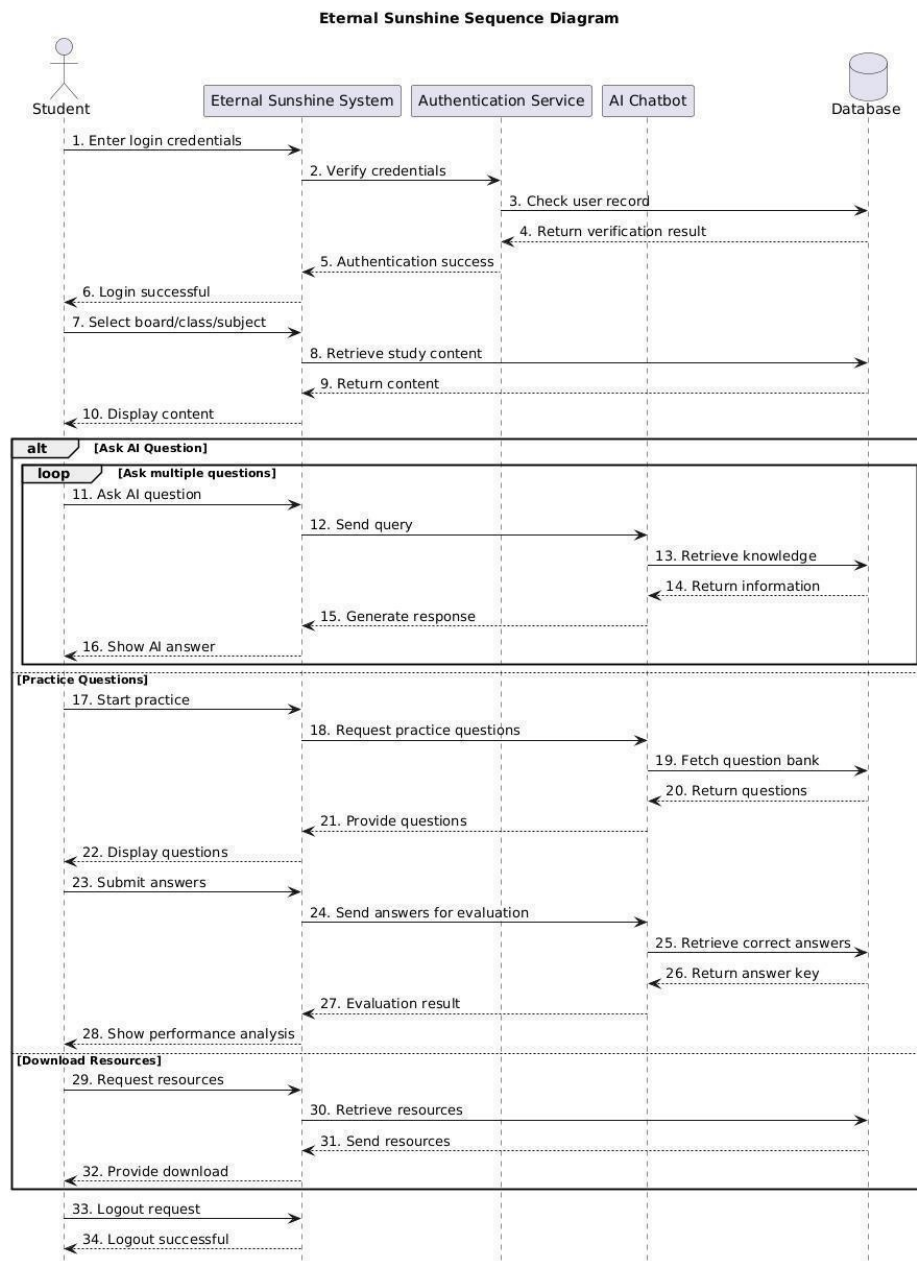


Figure 3.15: Sequence diagram covering login, content loading, AI questioning, practice evaluation, resource download and logout.

3.9 FEASIBILITY STUDY

3.9.1 Technical feasibility

Every technical component required by the design either existed as a mature commodity or was demonstrably within the team's competence. Laravel, React and MySQL are established and well documented. Managed vector search removes the need to implement or operate an approximate nearest-neighbour index. Language-model capability is consumed through an API, so no training infrastructure, no

graphics hardware and no machine-learning expertise beyond prompt design is required. The only genuinely uncertain element at the outset was retrieval quality over recognised text from scanned board material. That risk was retired in Sprint 5 by ingesting a single textbook end to end and reading the retrieved excerpts by hand before committing to the architecture.

3.9.2 Economic feasibility

The project's direct cost is dominated by third-party service consumption. Table 3.17 gives the development-phase position. The pricing used is the published rate for gpt-4o-mini: 0.150 United States dollars per million input tokens and 0.600 per million output tokens. The embedding model is 0.020 per million tokens. These are the same constants the system uses internally to compute the cost of every call. The consequence is that the marginal cost of serving a student is small enough that free access is a realistic operating model, which is the economic precondition for the equity claim in Section 1.7.

Table 3.17: Development-phase cost position

Item	Basis	Cost
Language-model usage (gpt-4o-mini)	USD 0.150 per 1M input tokens; USD 0.600 per 1M output tokens; metered per call by the built-in token logger	Usage-dependent; see Section 8.6
Embedding generation (text-embedding-3-small)	USD 0.020 per 1M tokens; one-off for corpus ingestion plus a small per-query cost	A few US dollars for the whole corpus
Vector index (Pinecone serverless)	Free starter tier sufficient for the ingested corpus	Nil during development
Google Cloud Vision OCR	First 1,000 units per month free; billed thereafter	Within free tier for staged ingestion
Google OAuth 2.0	No charge	Nil
Development hardware and tooling	Team-owned laptops; open-source toolchain; free tiers of Figma and GitHub	Nil
Production hosting (projected)	Shared PHP hosting or a small cloud instance plus a domain	Approximately PKR 1,500–3,000 per month plus PKR 1,500 per year

3.9.3 Operational feasibility

The interaction model — a chat box, a multiple-choice card, a download list — is already familiar to the target users, which minimises the adoption barrier. The administrative console requires no database or command-line knowledge. The only operational task that requires technical skill is corpus ingestion, which is performed by the operator through three scripts and is required only when a new board, class or subject is added.

3.9.4 Schedule feasibility

The work was scoped to twelve two-week sprints across the academic year. The highest-risk item, corpus ingestion and retrieval quality, was deliberately scheduled in the middle rather than at the end. A negative result would then still leave time to adjust the scope. The schedule was met, with the two-factor authentication and Google sign-in features arriving later than originally planned and voice support being dropped, as recorded in Section 1.5.2.

3.9.5 Risk assessment

Table 3.18: Principal project risks and mitigations

Risk	Likelihood	Impact	Mitigation adopted
Retrieval returns irrelevant context from OCR-degraded text	Medium	High	Aggressive rule-based cleaning; similarity floor of 0.20; manual inspection of retrieved excerpts before architectural commitment.
Language-model cost grows uncontrollably with session length	Medium	High	Rolling conversation summarisation; verbatim history capped at four turns; per-call cost logging to detect regressions.
Model awards impossible scores	Medium	Medium	Score parsed from a structured line and clamped in application code to the maximum for the question type.
Third-party API outage during demonstration	Low	High	Graceful degradation with an explanatory message; persisted transcripts so no student input is lost.
Copyright exposure from redistributing textbooks	Medium	High	Ingested material used for grounding only; downloads restricted to lawfully available documents.
Scope growth beyond the available time	High	Medium	Prioritised backlog; voice interaction and native mobile applications explicitly deferred.

3.10 REQUIREMENT TRACEABILITY MATRIX

Table 3.19 traces each requirement group forward to the design element that realises it, the implementation artefact that contains it and the test case that verifies it. The full test-case catalogue is given in Chapter 7.

Table 3.19: Requirement traceability matrix

Requirements	Design element	Implementation artefact	Verified by
REQ-1 to REQ-7, REQ-26 to REQ-29	Authentication subsystem (§4.4, §4.11)	`Fortify` actions, `GoogleController`, `SecurityController`	TC-01 to TC-12
REQ-8 to REQ-11, REQ-30, REQ-31	Academic scoping model (§5.2, §5.5)	`Board`, `ClassLevel`, `Subject` enums; `SelectionController`; `CURRICULUM_DATA`	TC-13 to TC-17
REQ-12 to REQ-15, REQ-32 to REQ-35	Teacher agent and retriever (§4.6–4.9)	`teacher_bot.py`, `retriever.py`, `memory_manager.py`, `ChatController`	TC-18 to TC-26
REQ-16 to REQ-20, REQ-36 to REQ-38	Assessment subsystem (§4.6, §5.7)	`tester_bot.py`, `overall_evaluator.py`, `QuizController`	TC-27 to TC-36
REQ-21 to REQ-25, REQ-39	Resource subsystem (§5.2, §5.5)	`ResourceController`, `Admin\ContentController`, `Content` model	TC-37 to TC-42
REQ-40, REQ-41	Analytics subsystem (§5.2)	`DashboardController`, `ProgressController`, `Activity` model	TC-43 to TC-46
REQ-42	Administration subsystem (§4.11)	`Admin\UserController`, `Admin\ContentController`, role attribute	TC-47 to TC-52
NFR-1 to NFR-5	Performance design (§4.7, §4.9)	LRU embedding cache, combined retrieval, summarisation	TC-53 to TC-57
NFR-6 to NFR-11	Security architecture (§4.11)	Middleware pipeline, hashed casts, environment configuration	TC-58 to TC-63
NFR-12 to NFR-19	Quality attributes (§4.12, §6.1)	Component structure, Pint and ESLint configuration, token logger	TC-64 to TC-70

3.11 TOOLS AND TECHNOLOGIES

Table 3.20 records the technology selections and the reason each was chosen. Where the choice departs from the proposal, the reason for the change is stated. The official documentation of the principal frameworks and services was the primary reference throughout implementation (Google Cloud 2024)-(Inertia.js 2025).

Table 3.20: Tools and technologies with selection rationale

Layer	Technology	Rationale
Language (backend)	PHP 8.4	Typed enumerations, constructor property promotion and readonly properties give the domain model real type safety; widely available on inexpensive Pakistani hosting.
Backend framework	Laravel 13 (2025)	Batteries-included authentication, migrations, validation, queuing and an expressive ORM; the largest team-familiar ecosystem.
Authentication	Laravel Fortify, Sanctum, Socialite	Headless, audited implementations of registration, verification, password reset, TOTP two-factor and OAuth 2.0 — none of which should be written by hand.
Frontend library	React 19 (Meta Open Source 2025)	Component model matched the interface; large ecosystem; team familiarity.
SPA bridge	Inertia.js v3	Gives single-page navigation without building or versioning a separate REST API for page data, halving the integration surface.
Styling	Tailwind CSS v4 with shadcn/ui and Radix primitives	Utility-first styling with accessible, unstyled component primitives; consistent design without a bespoke design system.
Type-safe routing	Laravel Wayfinder	Generates TypeScript functions from Laravel routes so a renamed route becomes a compile-time error rather than a broken link.
Charting	Recharts	Declarative React charts for the progress dashboard.
Build tooling	Vite 8	Fast development server and optimised production bundles.
AI service	FastAPI on Uvicorn (Ramírez 2025)	Python is where the AI client libraries live; FastAPI's Pydantic models give request validation and a typed contract at the service boundary.
Language model	OpenAI gpt-4o-mini	Sufficient quality for grounded explanation and rubric-based marking at roughly one-sixteenth the input cost of the frontier model; cost is a first-class requirement here.
Embeddings	text-embedding-3-small (1,536-d)	Inexpensive, strong retrieval quality, compact vectors.
Vector store	Pinecone (serverless, AWS us-east-1)	Metadata pre-filtering during search — the property that makes board scoping structural — plus a free tier adequate for the corpus.
OCR	Google Cloud Vision	Materially better than open-source engines on degraded scans and on Urdu script.
Relational store	MySQL 8	Ubiquitous on target hosting; adequate JSON support for the areas columns.
Testing	Pest 4 on PHPUnit 12	Expressive syntax over a mature runner; fast feedback in a small team.
Code quality	Laravel Pint, ESLint 9, Prettier 3, TypeScript 5.7	Automated style and static analysis so review effort is spent on logic.
Version control	Git and GitHub	Branch-per-concern workflow with merge points at sprint boundaries.
Design	Figma	Wireframing and interface iteration before implementation.

Deviation from the proposal

The proposal listed open-weight HuggingFace models (Hugging Face 2024) as an alternative AI provider and left the final model choice as a to-be-determined item in the Software Requirements Specification. The decision to use a hosted OpenAI model with an external vector index, rather than self-hosted open-weight models, was taken in Sprint 5 on grounds of cost and of the hardware available to the team: self-hosting an open-weight model of comparable quality would have required GPU infrastructure the project could not fund, whereas the metered API cost proved to be a fraction of a cent per interaction.

3.12 SUMMARY

The project followed a twelve-sprint Agile process with demonstration-driven increments. Requirements were elicited from forty-eight students and were validated against their reported behaviour rather than against team assumptions; the survey results map onto specific requirements in Table 3.1. Two user classes were identified, with the student class dominating every design decision. Forty-two functional requirements and nineteen non-functional requirements were specified, the first twenty-five of the former carried forward unchanged from the defended Software Requirements Specification. Behavioural models were produced for use cases, state transitions and interaction sequences. The project was found feasible on technical, economic, operational and schedule grounds, with retrieval quality over OCR output identified as the principal risk and retired early by experiment. The next chapter turns this requirement set into an architecture.

SYSTEM ARCHITECTURE

This chapter presents the architecture of the system. It sets out the principles behind the split into tiers, the subsystems that resulted, the flow of data between them, the design of the artificial-intelligence tier that forms the technical core of the project, the contract between tiers, the security model and the deployment view.

4.1 INTRODUCTION

This chapter presents the architecture of the Eternal Sunshine platform. It sets out the principles that guided the split into parts, the tiers and subsystems that resulted, and the flow of data between them. It then covers the design of the artificial-intelligence subsystem, which is the technical core of the work, followed by the interface contract between tiers, the security architecture and the design patterns applied. Detailed component-level design — the database schema, the class model and the algorithms — follows in Chapter 5.

4.2 ARCHITECTURAL DESIGN APPROACH

The architecture combines two styles. Vertically, the system is a layered architecture in the classical sense: presentation, application, artificial intelligence, and data. Each layer depends only on the layer beneath it, and no layer reaches upward. Horizontally, the boundary between the application layer and the artificial-intelligence layer is service-oriented: the two are separate processes in separate language runtimes communicating over HTTP with a JSON contract.

The decision to split the artificial-intelligence workload into its own service, rather than calling the model provider directly from PHP, was the single most consequential architectural decision in the project. Four considerations drove it.

1. **Ecosystem.** The client libraries for embedding models, vector databases and language models are first-class in Python and second-class or absent in PHP. Writing the AI layer in Python meant using supported, documented, actively maintained libraries rather than reimplementing them.

2. Independent evolution. Prompt text, model selection, retrieval parameters and chunking strategy changed far more often during development than the application's data model or its routes. Isolating them behind an HTTP boundary meant the AI service could be restarted and redeployed on its own cadence.
3. Substitutability. Because the contract is four JSON endpoints, the entire AI service could be replaced — by a self-hosted open-weight model, by a different provider, or by a stub during testing — without touching a line of application code. This is the property that makes the requirement in Section 3.7.3 for maintainability real rather than aspirational.
4. Failure isolation. A slow or failing model call cannot exhaust the web server's worker pool in the way an in-process synchronous call would; it fails at a clearly defined boundary where it can be caught and reported.

The cost of this decision is one additional network hop per AI request and one more process to deploy and supervise. Both were judged acceptable, and Section 8.5 quantifies the latency contribution of the hop as negligible relative to model inference time.

4.2.1 Architectural principles

- **Curricular scoping is structural.** Board, class and subject constrain retrieval as a metadata filter on the vector query. They are not requested in a prompt, because a prompt is advice and a filter is a guarantee.
- **The application layer owns truth.** Scores, grades, totals and percentages are computed in application code from values the model produced, never accepted verbatim from the model. The model contributes judgement; the application contributes arithmetic.
- **Persistence is the application layer's responsibility alone.** The AI service is stateless. It holds no database connection and no session state, which is what allows it to be scaled or replaced independently.

- **Cost is a measured property.** Every model call is metered at the point of use and written to a structured log, so that a change which doubles token consumption is visible immediately rather than at the end of a billing period.
- **Degrade with an explanation.** When an external dependency fails, the user sees a sentence describing what happened, and no user input is discarded.

4.3 HIGH-LEVEL ARCHITECTURE

Figure 4.1 shows the complete architecture. Reading downward, the browser runs a React application whose page components are resolved by Inertia. The Laravel application server terminates every browser request, applies the middleware pipeline, and either renders an Inertia page or returns a JSON response. The FastAPI service performs retrieval and generation. The data layer holds relational state in MySQL, vector state in Pinecone and uploaded documents on disk, alongside the third-party identity and model services.

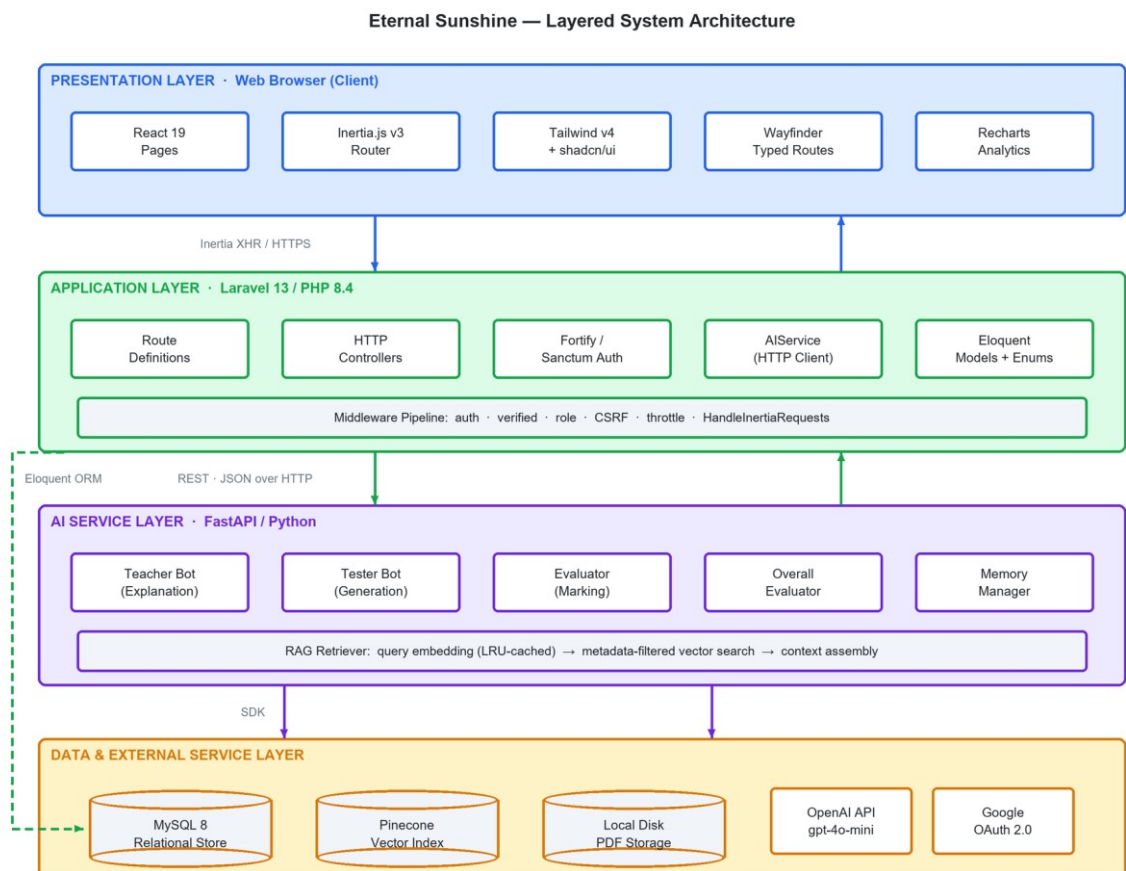


Figure 4.1: Complete layered architecture with the components of each tier and the protocols used between them.

Two properties of this diagram deserve emphasis. First, the browser never communicates with the AI service, with the vector index or with the model provider. Every such call is proxied through the Laravel application. That is what lets authentication, authorisation, rate limiting, persistence and activity logging be applied in one place, and it keeps the provider API key out of client code. Second, the Eloquent path from the application layer to MySQL bypasses the AI layer entirely: the AI service has no database access, by design.

4.4 SUBSYSTEM ARCHITECTURE

4.4.1 Presentation subsystem

The presentation subsystem is a React 19 application of 100 components and pages totalling approximately 10,640 lines of TypeScript. Page components live under `resources/js/pages` and are resolved by name when the server returns an Inertia response. Three layout components — an authentication layout, a student layout with the sidebar navigation, and an administrative layout — wrap the pages and provide consistent chrome. Shared interface primitives such as buttons, dialogs, selects, tables and form inputs are Radix-based components under `resources/js/components/ui`. Two domain components, the question card and the resource card, hold the presentation logic that would otherwise be repeated across pages.

Communication with the server takes two forms. Page navigation and form submission use Inertia's router, which issues an XHR request and receives the props for the next page component, giving single-page behaviour without a separately versioned REST API for page data. Some operations do not change the page: sending a chat message, generating a quiz, submitting an answer for marking, uploading a file. These use a thin fetch wrapper in `resources/js/lib/api.ts`. The wrapper makes sure the CSRF cookie has been obtained, attaches the token header and sets the correct content type. It omits the content type for multipart uploads, so the browser can supply the boundary itself.

4.4.2 Application subsystem

The Laravel application comprises 42 PHP classes and approximately 1,845 lines of code across controllers, models, enumerations, form requests, middleware and one service class. Its responsibilities are authentication and session management, authorisation, request validation, persistence through Eloquent, orchestration of calls to the AI service, activity logging, and the assembly of page props.

Routing is split by purpose. Session-authenticated page routes are declared in `routes/web.php` inside an `auth` and `verified` middleware group, with the administrative routes nested in a further prefixed group. Token- and cookie-authenticated JSON routes are declared in `routes/api.php` behind Sanctum. Account settings routes are separated into `routes/settings.php`. Several page controllers are single-action invokable classes, which keeps a controller that renders exactly one page from acquiring unrelated methods over time.

4.4.3 Artificial-intelligence subsystem

The AI service is a FastAPI application of 18 Python modules and approximately 2,868 lines. It exposes four functional endpoints and two health endpoints. Every request and response shape is declared as a Pydantic model with explicit length and range bounds. The endpoints delegate to four agent modules, which share one retrieval module and one set of provider clients. The service is stateless: conversation history and the rolling summary are supplied by the caller on every request and returned in the response for the caller to persist.

4.4.4 Data subsystem

Persistent state is divided across three stores according to its shape. MySQL holds the relational state — users, chat sessions and messages, quiz sessions and results, activities and content records — across nine application tables plus the framework's session, cache and queue tables. Pinecone holds the embedded corpus as 1,536-dimensional vectors with board, class, subject, type, language and chunk-index metadata. The local public disk holds uploaded resource files, referenced

from the contents table by relative path so that the storage backend can later be changed to object storage without a schema change.

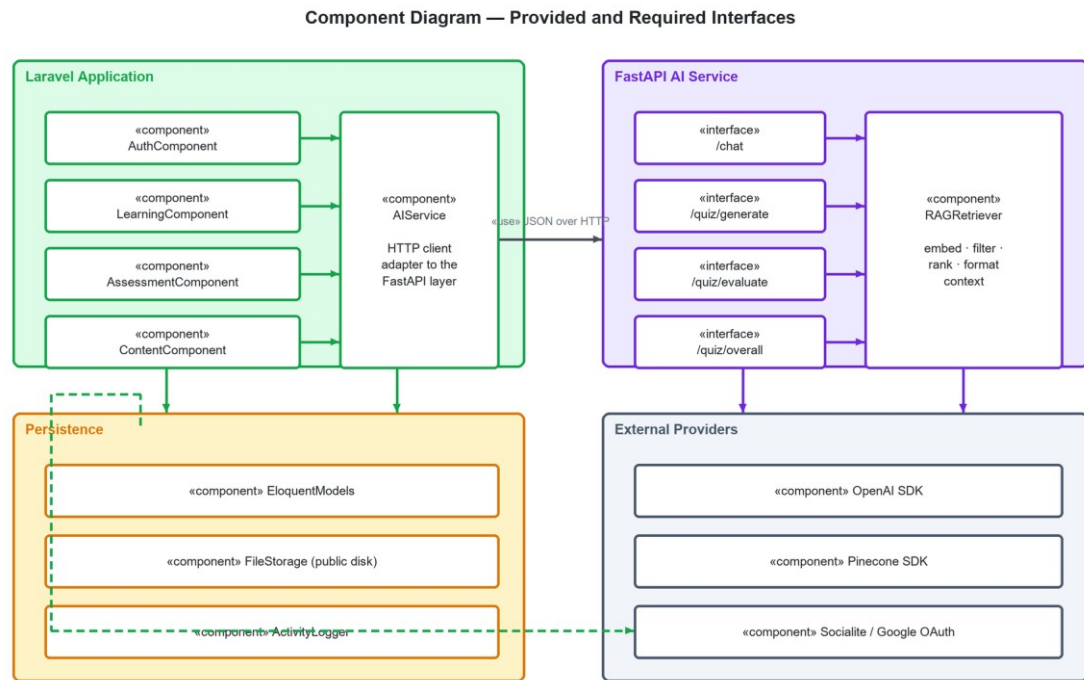


Figure 4.2: Component diagram showing the provided and required interfaces across the application, artificial-intelligence, persistence and external-provider subsystems.

4.5 DATA FLOW

4.5.1 Context diagram

Figure 4.3 places the system in its environment. Two human actors and three external service providers exchange data with the platform. The student supplies questions, answers and an academic selection, and receives explanations, generated questions, marks, feedback and downloadable files. The administrator supplies uploads and account actions and receives content records and usage statistics. The OpenAI platform receives prompts and text to embed, and returns completions and vectors. Pinecone receives filtered vector queries and returns ranked chunks. The Google platform supplies identity assertions at run time, and optical character recognition during offline ingestion.

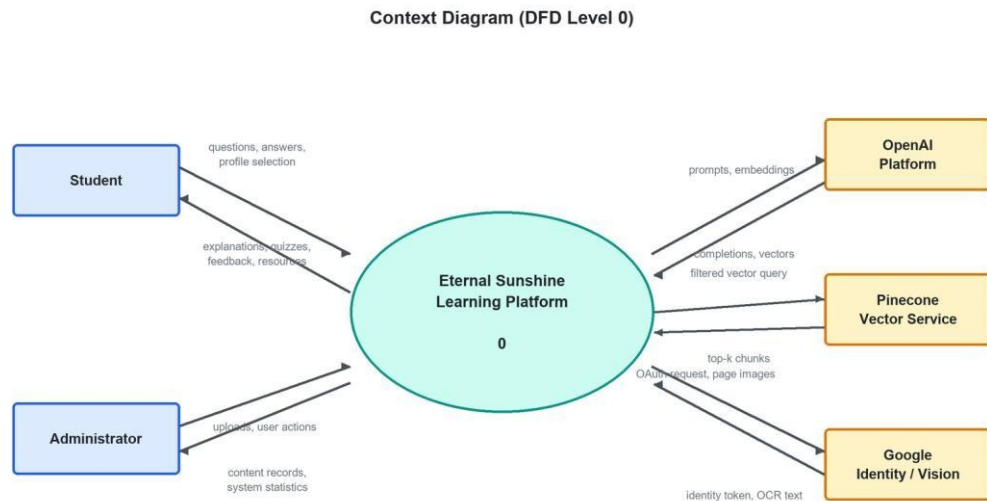


Figure 4.3: Context diagram (Data Flow Diagram level 0) showing the system boundary and the external entities that exchange data with it.

4.5.2 Level 1 decomposition

Figure 4.4 decomposes the system into five processes and six data stores. Process 1.0 authenticates users and maintains the academic profile against store D1. Process 2.0 conducts tutoring sessions, reading and writing the session and message stores D2 and calling the teacher agent. Process 3.0 generates and administers practice, and process 4.0 evaluates answers; both write to D3 and both call the tester and evaluator agents. Process 5.0 provides administration over the content store D4. Every process that represents a meaningful student action also appends to the activity store D5. Store D6, the vector index, is read exclusively by the AI agents and never by the application layer, which is the diagrammatic expression of the boundary described in Section 4.2.

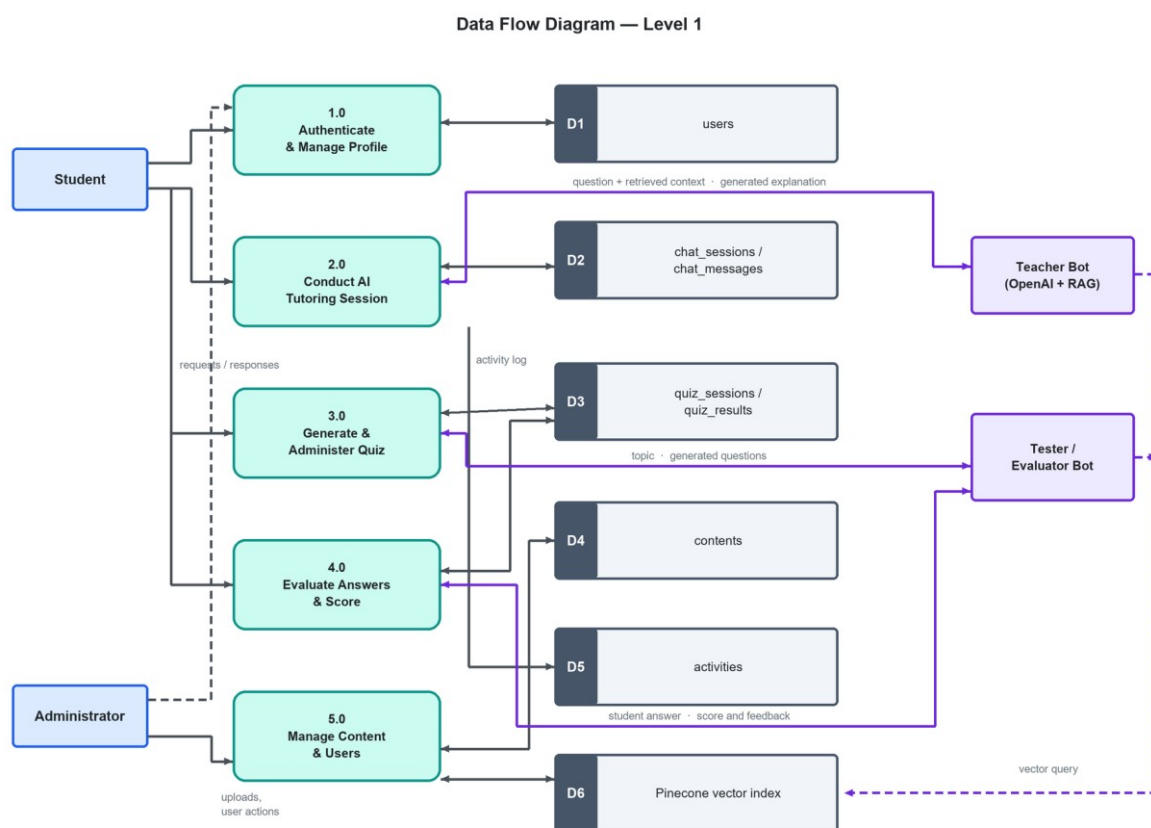


Figure 4.4: Level 1 Data Flow Diagram showing the five principal processes, the six data stores and the interaction with the artificial-intelligence agents.

4.6 ARTIFICIAL-INTELLIGENCE SUBSYSTEM DESIGN

The AI subsystem is organised as four agents over one shared retrieval layer. The agents differ only in prompt, decoding parameters and token budget; they use the same model and the same retriever. This is a deliberate demonstration. Specialising by prompt, combined with retrieval grounding, is enough to get clearly different and reliable behaviour out of one general-purpose model. No fine-tuning was performed, and none was needed.

Table 4.1: The four agents of the artificial-intelligence subsystem

Agent	Endpoint	Purpose	Tem p.	Max tokens	Retrieval
Teacher	'POST /chat'	Explain a concept in exam-oriented language, in the student's language, from the prescribed textbook.	0.3	1,000	top-k = 5, unfiltered by type
Tester	'POST /quiz/generate'	Generate MCQ, short or long questions in the board's style with provenance labels.	0.4	2,000	top-k = 4 textbook + 3 past papers, combined and

Agent	Endpoint	Purpose	Temp.	Max tokens	Retrieval
					deduplicated
Evaluator	'POST /quiz/evaluate'	Mark one free-text answer against the retrieved passage and the board marking scheme.	0.2	200	top-k = 3
Session evaluator	'POST /quiz/overall'	Aggregate a completed attempt into strong areas, weak areas, encouragement and a study recommendation.	0.2	600	none — reasons over the session record
Memory summariser	(internal)	Compress older conversation turns into a rolling four-to-six-sentence summary.	0.0	200	none

Three observations about this table are worth drawing out. The temperature falls as the task becomes more nearly deterministic: question generation benefits from some variety, explanation from a little, marking from as little as possible, and summarisation from none at all. The token budget is matched closely to the expected output. The evaluator's ceiling was cut from 500 tokens to 200 once it became clear that two sentences is what a student actually reads. That change cut the output cost of the most frequently called endpoint by roughly sixty per cent. And the session evaluator performs no retrieval at all, because its input is the student's own session record rather than the textbook.

4.7 KNOWLEDGE INGESTION PIPELINE

The corpus that grounds every generated answer is built offline by a four-stage pipeline, shown in Figure 4.5. The pipeline is run by the operator when a board, class or subject is added, and never at request time.

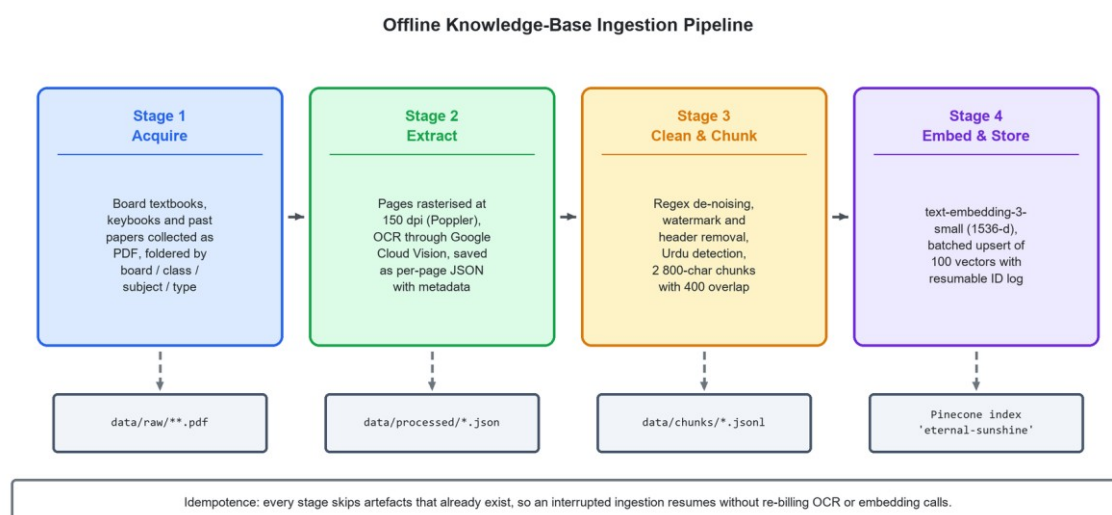


Figure 4.5: The four-stage offline ingestion pipeline, its intermediate artefacts and its idempotence property.

4.7.1 Stage 1 — acquisition and metadata by convention

Source documents are placed in a directory tree whose shape carries the metadata: `data/raw/{board}/{class}/{subject}/{type}/file.pdf`. The extraction script derives board, class, subject and document type from the path components rather than from a manifest file. Choosing convention over configuration removed a whole class of metadata-entry error. A Class 9 chemistry past paper cannot be filed in the wrong place and still be tagged correctly, because the place is the tag.

4.7.2 Stage 2 — optical character recognition

Pages are rasterised at 150 dots per inch through Poppler and submitted to Google Cloud Vision's text-detection endpoint one page at a time. Pages are processed in batches of ten and each page image is released immediately after recognition, because holding an entire six-hundred-page textbook as decoded images exhausts memory on a laptop. The output is one JSON document per source file. It holds the text of each page, character counts and the metadata derived in stage 1. Alongside it sits a quality report giving total characters, the number of near-empty pages and the mean characters per page. That report flags any document whose scan is too poor to be useful.

4.7.3 Stage 3 — cleaning and chunking

Recognition output is not usable for retrieval without cleaning. Two rule sets are applied. The general set removes standalone page numbers, collapses runs of blank lines and spaces, strips publisher names and web addresses, removes bullet glyphs and repeated recognition artefacts, and deletes keybook watermarks. A second, more aggressive set of rules applies only to documents typed as past papers. It strips roll-number boxes, page-of-page references, board and examination headers, time and marks allocations, instruction lines, section headers and paper codes. It also removes the characteristic garbage produced by recognising empty multiple-choice bubbles. Language is then detected by measuring the proportion of characters in the Arabic-script Unicode range, and short broken lines are rejoined into sentences.

Cleaned text is segmented into chunks of approximately 2,800 characters with a 400-character overlap, preferring to break at a line boundary and falling back to a sentence boundary. The overlap exists because a definition that straddles a chunk boundary would otherwise be retrievable in neither half. Each chunk is written as one line of a JSONL file carrying a deterministic identifier composed of board, class, subject, type, source filename and chunk index.

4.7.4 Stage 4 — embedding and indexing

Each chunk is embedded with text-embedding-3-small and upserted into a Pinecone serverless index of 1,536 dimensions using the cosine metric, in batches of one hundred vectors. The metadata written alongside each vector is board, class, subject, type, language, chunk index, and the first thousand characters of the chunk text. The text is stored so that retrieval returns readable context without a second lookup. Every successfully embedded identifier is appended to a log file, and the script consults that log on start-up, so an interrupted ingestion resumes without paying twice for the same embedding.

Design rationale

Idempotence is a property of every stage: extraction skips source files whose output JSON already exists, chunking skips documents already chunked, and embedding skips identifiers already recorded. This was not an elegance exercise. Optical character recognition and embedding are the only two metered operations in the pipeline, and a pipeline that must be

restarted from the beginning after a failure at page 480 is a pipeline that costs money every time the network drops.

4.8 RETRIEVAL AND GENERATION AT REQUEST TIME

Figure 4.6 traces a single tutoring request from the student's question to the displayed answer.

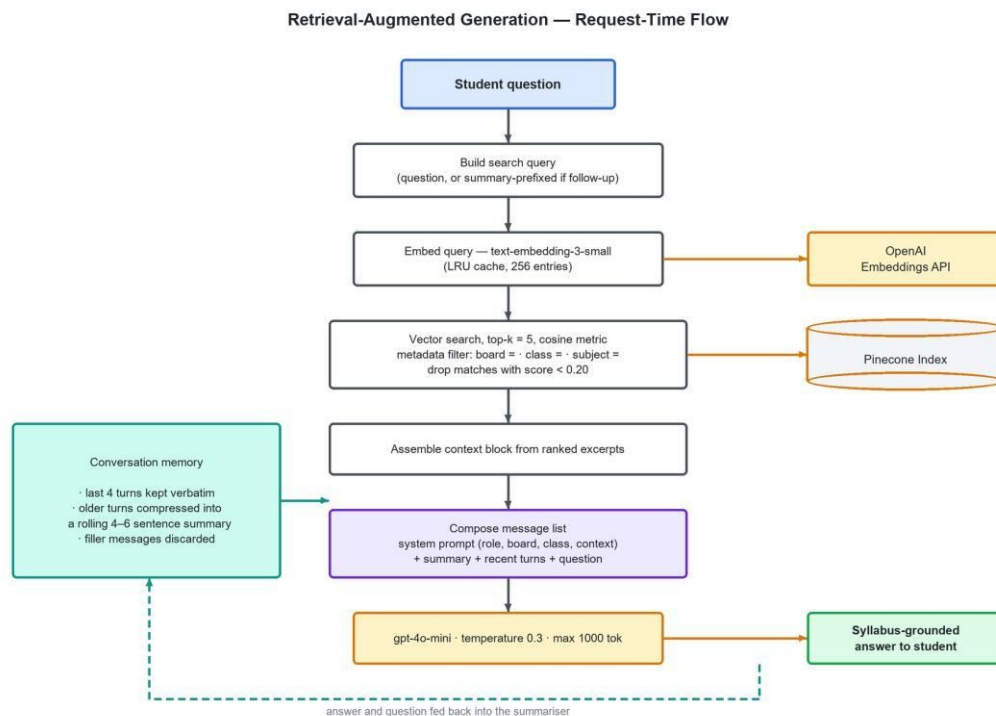


Figure 4.6: Request-time Retrieval-Augmented Generation flow, including the embedding cache, the metadata-filtered vector search and the injection of conversation memory.

4.8.1 Query construction

The search query is not always the student's raw question. A short follow-up such as “why?” or “give an example” carries almost no retrievable signal on its own. When the question is shorter than six words and a conversation summary exists, the first sentence of the summary is prepended to the question to restore topical context before embedding. The composed query is truncated to 300 characters, which bounds the payload and prevents a long question from diluting the embedding with incidental words.

4.8.2 Embedding with a cache

The query is normalised — trimmed and lower-cased — and embedded. A least-recently-used cache of 256 entries sits in front of the embedding call. The normalisation step exists to raise the hit rate: “What is Ohm's law” and “what is ohm's law ” are the same cache key. The cache costs about 1.5 megabytes of process memory. In return it removes a network round trip and a metered API call on every repeated query. In a classroom, where many students ask the same question, that is a meaningful share of the traffic.

4.8.3 Filtered vector search

The query vector is submitted to Pinecone with a metadata filter requiring exact equality on board, class and subject, and optionally on document type. This is the architectural heart of the system. The filter is applied during the search, not after it. The k results are therefore the k best matches within the student's own syllabus. They are not the k best matches overall, some of which happen to come from the right book. A similarity floor of 0.20 is then applied in application code, and matches below it are discarded. It is better to return no context at all, and let the teacher agent say it is answering from general knowledge. Returning an irrelevant passage would invite the model to invent a connection to it.

4.8.4 Combined retrieval for question generation

The tester agent needs two kinds of material: past papers, from which real examination questions can be extracted, and textbook or keybook content, from which new questions can be written. A naive implementation embeds the topic twice and issues two independent searches. The implementation here embeds the query once, because the second call is served from the cache. It then issues one type-filtered search for four textbook chunks and another for three past-paper chunks. The union is de-duplicated on the first hundred characters of each chunk and formatted as a single context block. This removed one metered embedding call from every question-generation request.

4.8.5 Context assembly and generation

Retrieved chunks are rendered as numbered excerpts, each annotated with its similarity score, and interpolated into the agent's prompt template together with the board, class and subject. The message list submitted to the model is then assembled as the system prompt, followed by the conversation summary if one exists, followed by the recent verbatim turns, followed by the student's question. The model's reply is returned to the application layer, which persists it and displays it.

4.9 CONVERSATION MEMORY ARCHITECTURE

A tutoring conversation is inherently multi-turn: a student asks about photosynthesis, then asks a follow-up, then asks for an example. Naively, the entire transcript is resent on every turn, so the prompt — and therefore the input cost and the latency — grows linearly with the length of the session. A session of forty turns would cost roughly twenty times as much per turn as a session of two.

The memory manager bounds this growth. Figure 4.7 shows the strategy.

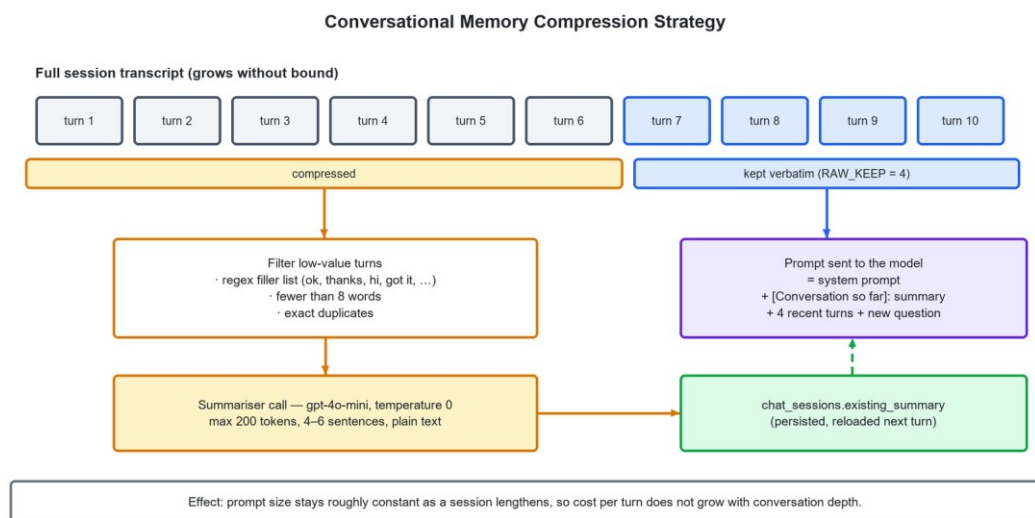


Figure 4.7: Conversation memory compression. Only the four most recent turns are resent verbatim; everything older is represented by a rolling summary that is itself capped at 200 tokens.

- **Recency window.** The four most recent turns are always resent verbatim, because the immediately preceding exchange is what a follow-up question refers to and paraphrasing it loses the referent.

- **Filtering.** Before older turns are summarised they are filtered. A regular expression removes conversational filler — acknowledgements, greetings, single-word confirmations; messages of fewer than eight words are dropped; and exact duplicates are removed. Summarising “ok thanks” consumes tokens and contributes nothing.
- **Rolling summarisation.** Surviving older turns are sent to the model with a dedicated system prompt instructing it to maintain four to six sentences capturing topics covered, the student's apparent level of understanding, misconceptions observed, explanations that worked and questions left open. Temperature is zero and the output is capped at 200 tokens. The result replaces the previous summary.
- **Persistence.** The summary is stored on the chat session row, so it survives the student closing the browser and is reloaded when the session is resumed.

The effect is that prompt size approaches a constant as a session lengthens: a system prompt with retrieved context, plus a bounded summary, plus four turns. Cost per turn therefore becomes approximately independent of session depth. The trade-off is one extra model call per turn to maintain the summary. That call runs at temperature zero against a 200-token output ceiling, so it costs far less than the input tokens it prevents. Section 8.6 quantifies the saving.

4.10 INTERFACE CONTRACT BETWEEN TIERS

The application layer reaches the AI service through a single class, AIService, which owns the base URL and exposes one method per endpoint. No other application class knows that the AI service exists as an HTTP resource. Table 4.2 records the contract.

Table 4.2: Contract between the Laravel application and the FastAPI AI service

Endpoint	Request fields	Response fields
POST /chat	'question' (≤ 1000), 'board', 'class_level', 'subject', 'language', 'chat_history' (≤ 20), 'existing_summary' (≤ 2000)	'answer', 'updated_summary', 'success'
POST /quiz/generate	'topic' (≤ 500), 'board', 'class_level', 'subject', 'question_type', 'num_questions' (1–20)	'questions' (formatted text), 'success'
POST	'question' (≤ 1000), 'student_answer'	'feedback', 'score', 'success'

Endpoint	Request fields	Response fields
/quiz/evaluate`	(≤3000), `board`, `class_level`, `subject`, `question_type`	
`POST /quiz/overall`	`board`, `class_level`, `subject`, `question_type`, `results` (≤50 objects of question, student_answer, score 0–10, feedback)	`total_score`, `percentage`, `grade`, `strong_areas`, `weak_areas`, `overall_feedback`, `study_tip`, `success`
`GET /`, `GET /health`	none	service status and version

Every bound in the request column is enforced by a Pydantic field constraint rather than a hand-written check. A malformed or oversized request is therefore rejected with a structured validation error before any metered operation begins. The upper bounds are cost controls as much as correctness controls: they place a hard ceiling on the number of tokens any single request can cause to be billed.

4.10.1 Application API surface

Table 4.3: HTTP API exposed by the Laravel application

Method and path	Purpose	Guard
`POST /api/register`, `POST /api/login`	Token-based authentication for API clients	public
`POST /api/logout`, `GET /api/user`	Session teardown and identity	sanctum
`POST /api/chat/send`	Send a tutoring message; creates a session on first use	sanctum
`GET /api/chat/sessions`	List the caller's chat sessions, most recent first	sanctum
`GET /api/chat/messages/{id}`	Load the transcript of one session	sanctum
`GET /api/chat/history/{id}`	Transcript formatted as role/content pairs for the model	sanctum
`DELETE /api/chat/{id}`	Delete a session and its messages	sanctum
`POST /api/quiz/generate`	Generate practice questions	sanctum
`POST /api/quiz/evaluate`	Mark one submitted answer	sanctum
`POST /api/quiz/overall`	Aggregate and persist a completed attempt	sanctum
`GET POST /api/content`	List and upload resources	sanctum
`GET /api/content/download/{id}`	Stream a resource and increment its counter	sanctum
`PUT DELETE /api/content/{id}`	Edit or remove a resource	sanctum
`GET /api/users`, `PUT DELETE /api/users/{id}`, `PATCH /api/users/toggle/{id}`	Administrative account management	sanctum

Known limitation

The administrative endpoints under /api are guarded by authentication but their role check is applied at the page layer rather than at the route layer. This is recorded as a known weakness in Section 7.7 and as a recommendation in Section 9.6: the correct fix is a dedicated role middleware applied to the route group, so that authorisation does not depend on the caller having arrived through the administrative interface.

4.11 SECURITY ARCHITECTURE

Security is applied in depth across five bands, informed by the OWASP catalogue of web application risks (2021).

Table 4.4: Security controls by architectural band

Band	Control	Implementation
Transport	Encryption in transit	HTTPS to the browser; TLS on every outbound call to OpenAI, Pinecone and Google.
Identity	Credential handling	Bcrypt hashing via the framework's hashed cast; passwords never logged; password-reset tokens single-use and expiring.
Identity	Federated sign-in	Google OAuth 2.0 through Socialite; the account is linked by email address if one already exists, otherwise created with the email marked verified.
Identity	Second factor	Optional TOTP with encrypted secret and encrypted recovery codes; challenge screen enforced before session establishment.
Session	Request integrity	CSRF token required on every state-changing request; the client wrapper obtains and attaches it automatically.
Session	Abuse resistance	Rate limiting on login and on password update; email verification required before learning features are reachable.
Authorisation	Role separation	A role attribute on the user record distinguishes student from administrator and gates the administrative interface and navigation.
Data	Ownership scoping	Chat, quiz and activity queries are constrained to the authenticated user's identifier; cascading deletes remove dependent rows with the parent.
Data	Input bounds	Length and range constraints on every AI request field; upload size limit of 100 megabytes; validation of every content field.
Secrets	Credential storage	All API keys and database credentials supplied through environment variables and excluded from version control.

Authentication and Authorisation Flow

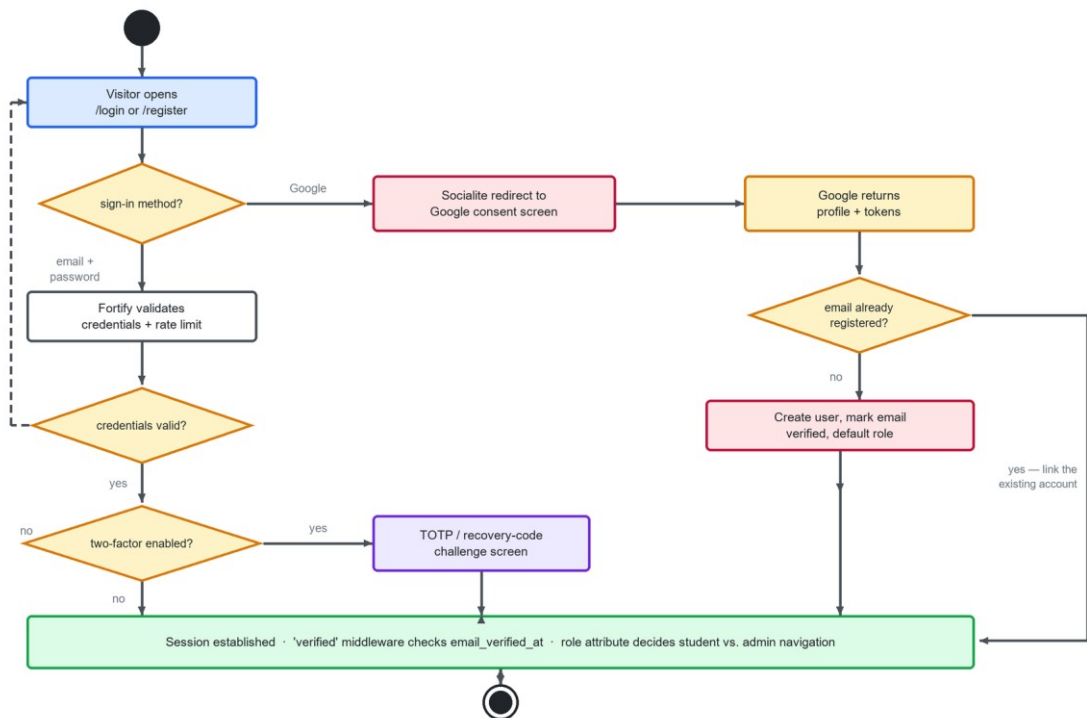


Figure 4.8: Authentication and authorisation flow covering password sign-in, Google OAuth sign-in, account linking, two-factor challenge and role resolution.

4.12 DESIGN PATTERNS APPLIED

Table 4.5: Design patterns used and where they appear

Pattern	Where applied	Benefit obtained
Layered architecture	The four tiers of Figure 4.1	Dependencies point in one direction only; each tier is separately comprehensible and separately testable.
Model-View-Controller	Laravel controllers, Eloquent models, React pages as views	Conventional separation that a maintainer already understands.
Service layer	`AIService`	The HTTP nature of the AI tier is known to exactly one class; four controllers depend on a method signature rather than on a protocol.
Active Record	Eloquent models	Domain objects carry their own persistence, which suits a schema of this size (Fowler 2003).
Repository-by-convention	Metadata-derived corpus paths	Board, class, subject and type are structural rather than declared, removing a class of tagging error.
Dependency injection	Constructor promotion of `AIService` into controllers	The service can be substituted with a fake in tests without touching the controller.
Strategy	One prompt template file per agent	Behaviour is changed by editing a text file, with no code change and no redeployment of application logic.
Adapter	`resources/js/lib/api.ts`	CSRF acquisition, header assembly and content-type selection are handled once rather than at every call site.

Pattern	Where applied	Benefit obtained
Cache-aside	LRU cache in front of the embedding call	Repeated queries avoid a metered network call.
Memento	`chat_sessions.existing_summary`	Conversation state is captured, persisted and restored without retaining the full transcript in the prompt.
Template method	The four-stage ingestion pipeline	Each stage has the same skip-if-done, process, log structure, so a new document type slots into the existing shape.

4.13 DEPLOYMENT ARCHITECTURE

Figure 4.9 shows the deployment view. In development, and in the modest production configuration the project targets, the Laravel application and the FastAPI service run on the same host, the AI service bound to port 8001 and reachable only from localhost. MySQL may be co-located or on a separate host. The browser holds only the compiled front-end bundle. All four third-party services are reached over TLS. The AI service is stateless. If concurrent load ever requires it, the natural first scaling step is to run several instances behind a local load balancer, with no change to the application layer.

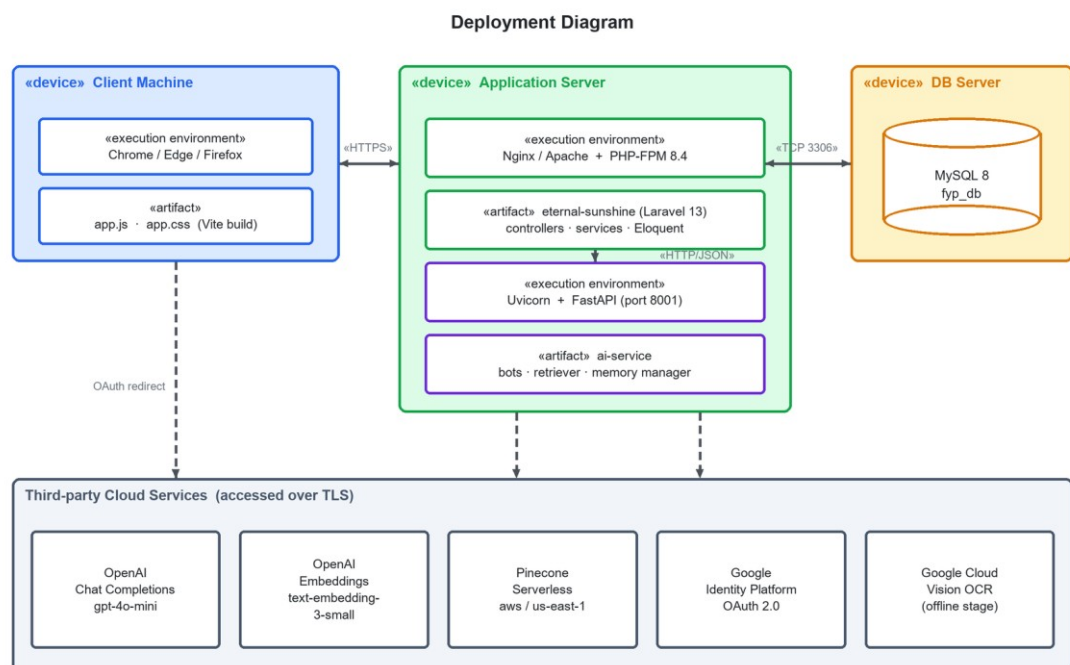


Figure 4.9: Deployment diagram showing devices, execution environments, deployed artefacts and the communication paths between them.

4.14 SUMMARY

The system is a four-tier layered architecture with a service-oriented boundary between the application and artificial-intelligence tiers, adopted for ecosystem access, independent evolution, substitutability and failure isolation. Curricular scoping is enforced as a metadata pre-filter on the vector query rather than as a prompt instruction, which is what converts board alignment from a claim into a structural guarantee. A four-stage idempotent pipeline converts scanned board material into a filtered vector index. Four agents share one retriever and differ only in prompt and decoding parameters. Conversation cost is bounded by rolling summarisation, and embedding cost by a normalised cache. Security is applied across transport, identity, session, authorisation and secret-handling bands, with one known authorisation weakness recorded honestly. The next chapter descends from this architecture to the detailed design of each component.

DETAILED SYSTEM DESIGN

This chapter takes the architecture of Chapter 4 down to the level of individual components. It gives the database design and data dictionary, the class model, a specification of each software component against a fixed set of attributes, the screen and navigation design, and the design of the four prompts that determine how the agents behave.

5.1 INTRODUCTION

Chapter 4 established the tiers, the subsystems and the flows between them. This chapter descends to the level at which the system was actually built. Section 5.2 gives the database design with a complete data dictionary. Section 5.3 gives the class model. Section 5.4 specifies each software component against the ten standard component attributes. Section 5.5 documents the interface design and navigation model. Section 5.6 presents the prompt designs, which in a system of this kind are as much a part of the design as any class. Section 5.7 gives the principal algorithms in pseudocode.

5.2 DATABASE DESIGN

5.2.1 Design objectives and normalisation

The relational schema was designed to meet four objectives. Every learning artefact must be traceable to a user. Deleting a user must leave no orphaned rows. The academic scope of a record must be recoverable from the record itself. And the analytics the dashboard needs must be answerable without joining more than two tables.

The schema is in third normal form with two deliberate departures. First, board, class level and subject are duplicated onto chat_sessions and quiz_sessions rather than being read from the owning user row. This is intentional denormalisation. A student may change their academic profile between sessions. A chat session held under Class 11 Physics must still be scoped to Class 11 Physics when it is resumed, even if the student has since switched to Class 12 Chemistry. The scope belongs to the session, not to the user. Second, the session-level

aggregates on quiz_sessions — total score, percentage, grade — are stored rather than recomputed from quiz_results. They are stored because the grade band is a function of the marking scheme in force at the time of the attempt, and because the dashboard reads them on every page load.

5.2.2 Entity-relationship model

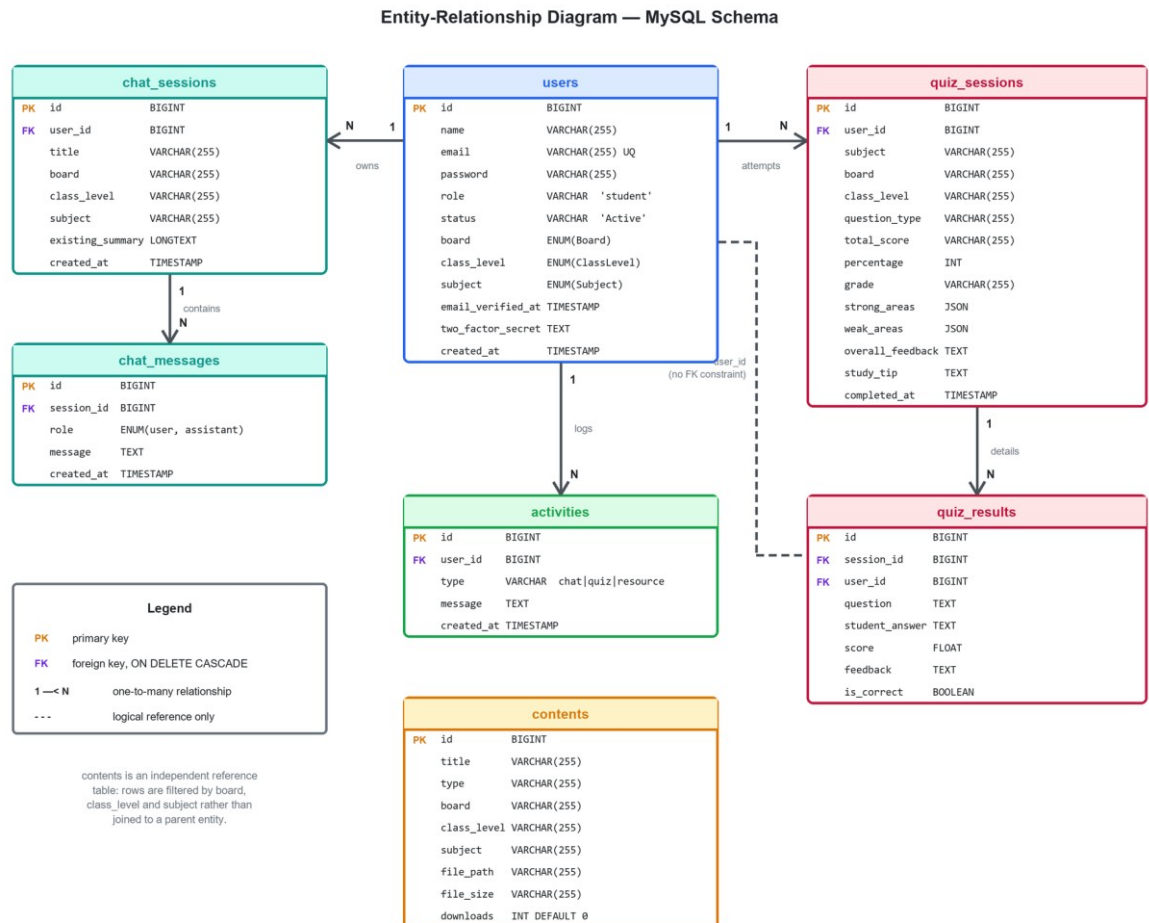


Figure 5.1: Entity-relationship diagram of the MySQL schema, showing primary keys, foreign keys, cascade behaviour and cardinalities.

The model has one aggregate root, users, from which three one-to-many relationships descend: to chat_sessions, to quiz_sessions and to activities. Two second-level relationships descend further, from chat_sessions to chat_messages and from quiz_sessions to quiz_results. Every foreign key on these paths is declared with ON DELETE CASCADE, so deleting a user removes their entire learning history in one operation and no orphan can survive. The contents table

stands outside this hierarchy: a resource belongs to a board, class and subject rather than to a user, and is located by filtering rather than by traversal.

Known schema inconsistency

quiz_results carries a user_id column that duplicates the owner already reachable through session_id, and that column is declared without a foreign-key constraint. It exists so that the dashboard can count a student's attempted questions with a single-table query rather than a join. The absence of the constraint is an inconsistency rather than a design decision, and correcting it is listed in Section 9.6.

5.2.3 Data dictionary

Table 5.1: Data dictionary — users

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, auto-increment	Surrogate identifier.
name	VARCHAR(255)	NOT NULL	Display name; taken from the Google profile for federated accounts.
email	VARCHAR(255)	NOT NULL, UNIQUE	Login identifier and the key used to link a Google account to an existing profile.
role	VARCHAR(255)	NOT NULL, default 'student'	Authorisation role; 'admin' unlocks the administrative console.
status	VARCHAR(255)	NOT NULL, default 'Active'	Account state; an administrator may set it to Blocked.
password	VARCHAR(255)	NOT NULL	Bcrypt hash applied by the model's hashed cast.
board	VARCHAR(255)	NULLABLE	Academic profile; cast to the Board enumeration.
class_level	VARCHAR(255)	NULLABLE	Academic profile; cast to the ClassLevel enumeration.
subject	VARCHAR(255)	NULLABLE	Default subject; cast to the Subject enumeration.
email_verified_at	TIMESTAMP	NULLABLE	Set on verification; null blocks access to learning routes.
two_factor_secret	TEXT	NULLABLE, encrypted	TOTP shared secret.
two_factor_recovery_codes	TEXT	NULLABLE, encrypted	Single-use recovery codes.
two_factor_confirmed_at	TIMESTAMP	NULLABLE	Set when the second factor is confirmed.
remember_token	VARCHAR(100)	NULLABLE	Persistent-login selector.
created_at / updated_at	TIMESTAMP	NULLABLE	Framework timestamps.

Table 5.2: Data dictionary — chat_sessions

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK	Surrogate identifier.
user_id	BIGINT UNSIGNED	FK → users.id, ON DELETE CASCADE	Owning student.
title	VARCHAR(255)	NULLABLE	Derived from the first 40 characters of the opening question.
board	VARCHAR(255)	NULLABLE	Academic scope frozen at session creation.
class_level	VARCHAR(255)	NULLABLE	Academic scope frozen at session creation.
subject	VARCHAR(255)	NULLABLE	Academic scope frozen at session creation.
existing_summary	LONGTEXT	NULLABLE	Rolling conversation summary maintained by the memory manager.
created_at / updated_at	TIMESTAMP	NULLABLE	Framework timestamps; ordering key for the session list.

Table 5.3: Data dictionary — chat_messages

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK	Surrogate identifier.
session_id	BIGINT UNSIGNED	FK → chat_sessions.id, ON DELETE CASCADE	Owning session.
role	ENUM('user','assistant')	NOT NULL	Speaker; maps directly onto the model's message roles.
message	TEXT	NOT NULL	Utterance text.
created_at / updated_at	TIMESTAMP	NULLABLE	Ordering key for transcript replay.

Table 5.4: Data dictionary — quiz_sessions

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK	Surrogate identifier.
user_id	BIGINT UNSIGNED	FK → users.id, ON DELETE CASCADE	Owning student.
subject / board / class_level	VARCHAR(255)	NOT NULL	Academic scope of the attempt.
question_type	VARCHAR(255)	NOT NULL	mcq, short or long; determines the marking scale.

Column	Type	Constraints	Description
total_score	VARCHAR(255)	NULLABLE	Human-readable 'earned / possible' string computed by the application.
percentage	INT	NULLABLE	Rounded percentage; the key input to all analytics.
grade	VARCHAR(255)	NULLABLE	Band: Excellent, Good, Needs Work or Poor.
strong_areas	JSON	NULLABLE	Topics the student answered well, identified by the session evaluator.
weak_areas	JSON	NULLABLE	Topics the student struggled with.
overall_feedback	TEXT	NULLABLE	Two to three sentences of encouragement and assessment.
study_tip	TEXT	NULLABLE	Specific revision recommendation.
completed_at	TIMESTAMP	NULLABLE	Set when the session-level evaluation succeeds.
created_at / updated_at	TIMESTAMP	NULLABLE	Framework timestamps.

Table 5.5: Data dictionary — quiz_results

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK	Surrogate identifier.
session_id	BIGINT UNSIGNED	FK → quiz_sessions.id, ON DELETE CASCADE	Owning attempt.
user_id	BIGINT UNSIGNED	NOT NULL, no FK constraint	Denormalised owner for single-table analytics; see the note in Section 5.2.2.
question	TEXT	NOT NULL	Question text including its provenance label.
student_answer	TEXT	NULLABLE	The student's submitted answer, stored verbatim.
score	FLOAT	NOT NULL, default 0	Marks awarded, clamped to the maximum for the question type.
feedback	TEXT	NULLABLE	Two to three sentences from the evaluator.
is_correct	BOOLEAN	NOT NULL, default false	Convenience flag derived from the score.
created_at / updated_at	TIMESTAMP	NULLABLE	Framework timestamps.

Table 5.6: Data dictionary — contents and activities

Table.Column	Type	Constraints	Description
contents.id	BIGINT UNSIGNED	PK	Surrogate identifier.
contents.title	VARCHAR(255)	NOT NULL	Display name of the resource.

Table.Column	Type	Constraints	Description
contents.type	VARCHAR(255)	NOT NULL	Book, keybook or past paper.
contents.board / class_level / subject	VARCHAR(255)	NOT NULL	Filter keys used by the resource library.
contents.file_path	VARCHAR(255)	NOT NULL	Path relative to the public storage disk.
contents.file_size	VARCHAR(255)	NULLABLE	Size in bytes, captured at upload.
contents.downloads	INT	NOT NULL, default 0	Incremented on each successful download.
activities.id	BIGINT UNSIGNED	PK	Surrogate identifier.
activities.user_id	BIGINT UNSIGNED	FK → users.id, ON DELETE CASCADE	Owning student.
activities.type	VARCHAR(255)	NOT NULL	chat, quiz or resource.
activities.message	TEXT	NOT NULL	Human-readable description shown in the dashboard feed.
activities.created_at	TIMESTAMP	NULLABLE	Ordering key; also drives the pruning rule.

5.2.4 Enumerations and value normalisation

Board, class level and subject are modelled as backed PHP enumerations rather than as free strings. The stored values are lower-case, underscore-delimited tokens — federal, ajk; class_9 to class_12; physics through urdu — and each enumeration carries a label method that returns the human-readable form for display. This gives three benefits. The compiler rejects an invalid value at every call site. The display form is defined in exactly one place. And the stored token is identical to the metadata value written into the vector index, so a retrieval filter can be built straight from a model attribute with no translation step.

Early development stored display strings such as 'Federal Board' and '9'. A dedicated data migration normalised the existing rows across the users, chat_sessions and contents tables to the token form, and is reversible. Retaining that migration in the repository is deliberate: it documents that the identity between the database token and the vector metadata value is a property the system depends on, not a coincidence.

5.3 CLASS DESIGN

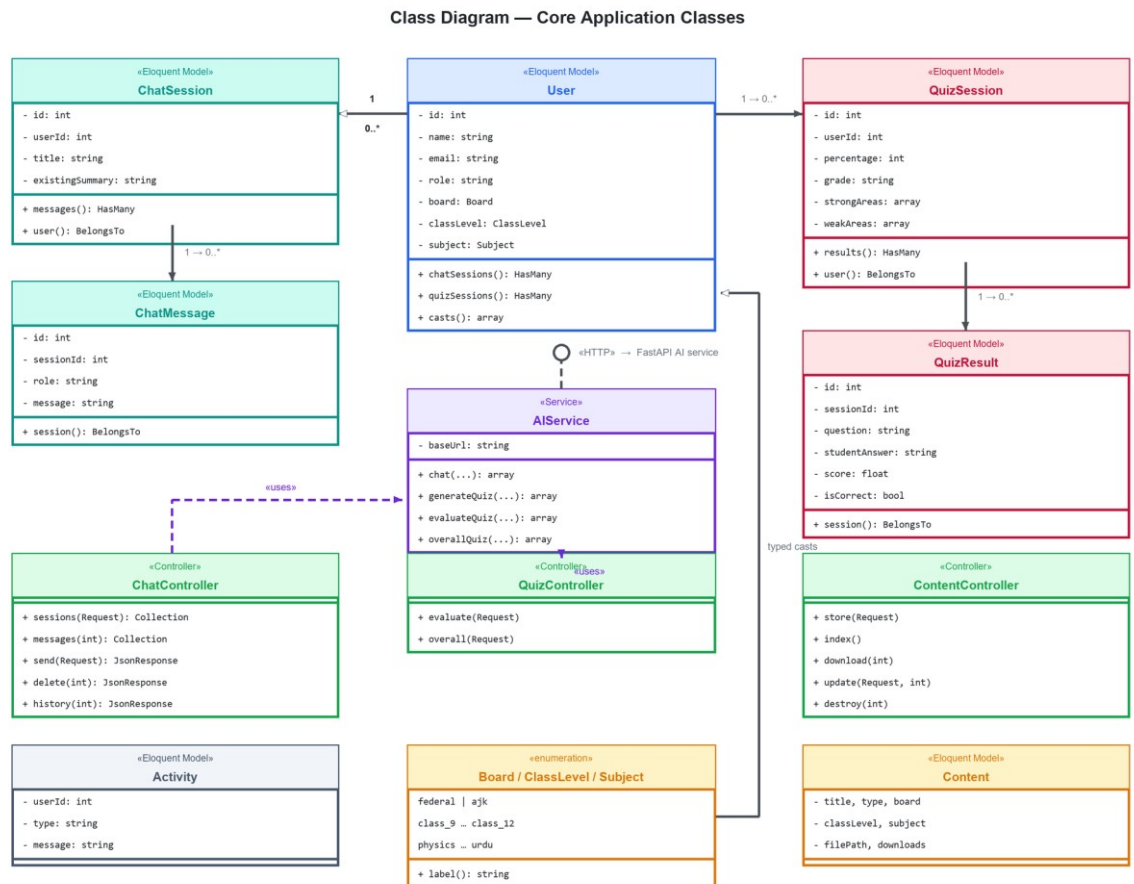


Figure 5.2: Class diagram of the core application classes, showing Eloquent models, the service class, controllers and the typed enumerations.

The class model is deliberately thin. Eloquent models carry their attributes, casts and relationships and no business logic beyond that. Controllers hold the orchestration for one HTTP concern each. The single service class, AIService, is the only place in the PHP codebase that knows the AI tier is reached over HTTP. Enumerations carry the vocabulary.

Table 5.7: Principal classes and their responsibilities

Class	Stereotype	Responsibility
`User`	Eloquent model	Identity, credentials, role, status and academic profile; casts board, class level and subject to enumerations and the password to a hash; declares hasMany relationships to chat and quiz sessions.
`ChatSession`	Eloquent model	One tutoring conversation, its frozen academic scope and its rolling summary.
`ChatMessage`	Eloquent model	One utterance within a session, typed by role.

Class	Stereotype	Responsibility
`QuizSession`	Eloquent model	One completed practice attempt and its aggregate result.
`QuizResult`	Eloquent model	One question, the answer given, the mark awarded and the feedback returned.
`Activity`	Eloquent model	A single entry in the bounded recent-activity feed.
`Content`	Eloquent model	A downloadable resource and its filter keys.
`AIService`	Service	The sole adapter to the FastAPI tier; exposes chat, generateQuiz, evaluateQuiz and overallQuiz.
`ChatController`	Controller	Session listing, transcript retrieval, message send with memory handling, and deletion.
`QuizController`	Controller	Answer evaluation pass-through and session-level aggregation with persistence.
`AIController`	Controller	Thin JSON pass-through for chat and quiz generation.
`DashboardController`	Controller	Computes questions practised, average score, weakest subject and the activity feed.
`ProgressController`	Controller	Computes per-subject averages, status bands and the focus recommendation.
`ResourceController`	Controller	Supplies the resource library to the student page.
`Admin\ContentController`	Controller	Upload, list, download, edit and delete resources.
`Admin\UserController`	Controller	List, edit, block and delete accounts.
`Auth\GoogleController`	Controller	OAuth redirect and callback, including account linking.
`Board`, `ClassLevel`, `Subject`	Enumeration	Curricular vocabulary with display labels.

5.4 COMPONENT SPECIFICATIONS

Each major software component is specified below against the ten standard component attributes: identification, type, purpose, function, subordinates, dependencies, interfaces, resources, processing and data.

5.4.1 Authentication and Profile Component

Table 5.8: Component specification — Authentication and Profile

Attribute	Specification
Identification	AUTH-01, Authentication and Profile Component.
Type	Composite component spanning framework packages, controllers, form requests and page components.
Purpose	Establish and maintain the identity of every actor and hold the academic profile that scopes all subsequent content. Realises REQ-1 to REQ-7 and REQ-26 to REQ-30.
Function	Registration; password authentication with rate limiting; Google OAuth 2.0 sign-in with

Attribute	Specification
	account linking; email verification; password reset; optional TOTP two-factor with recovery codes; password confirmation for sensitive operations; profile and academic-selection update; account deletion.
Subordinates	Fortify registration, login, verification and reset actions; <code>'GoogleController'</code> ; <code>'Settings\ProfileController'</code> ; <code>'Settings\SecurityController'</code> ; <code>'ProfileController'</code> ; <code>'SelectionController'</code> ; the auth page components under <code>resources/js/pages/auth</code> .
Dependencies	Laravel Fortify; Laravel Sanctum; Laravel Socialite; the mail transport for verification and reset messages; the users table; the Board, ClassLevel and Subject enumerations.
Interfaces	Provided: the <code>'/login'</code> , <code>'/register'</code> , <code>'/forgot-password'</code> , <code>'/reset-password'</code> , <code>'/two-factor-challenge'</code> , <code>'/auth/google/redirect'</code> , <code>'/auth/google/callback'</code> and <code>'/settings/*'</code> routes, plus <code>'POST /api/register'</code> and <code>'POST /api/login'</code> . Required: the Google OAuth endpoints and an SMTP transport.
Resources	One database connection; one outbound HTTPS connection during OAuth; the session store; the mail queue.
Processing	Credentials are validated and rate-limited; a successful password authentication either establishes a session or diverts to the two-factor challenge. An OAuth callback resolves the returned email against the users table and either logs in the matching account or creates one with the email pre-verified. The verified middleware rejects any learning request whose user has a null <code>email_verified_at</code> .
Data	users row including role, status, academic profile, hashed password and encrypted two-factor material; the session record; password-reset tokens.

5.4.2 Retrieval Component

Table 5.9: Component specification — Retrieval

Attribute	Specification
Identification	RAG-01, Retrieval Component (<code>ai-service/rag/retriever.py</code>).
Type	Python module providing pure functions over singleton provider clients.
Purpose	Return the passages of the student's own board, class and subject that are most relevant to a query, and format them as a context block. Realises REQ-11 and REQ-32.
Function	<code>'embed_query'</code> normalises and embeds a query with a least-recently-used cache; <code>'retrieve'</code> performs a metadata-filtered vector search with an optional document-type constraint and a similarity floor; <code>'retrieve_combined'</code> obtains textbook and past-paper context in one embedding call and deduplicates the union; <code>'format_context'</code> renders ranked excerpts for prompt interpolation.
Subordinates	The cached embedding function; the metadata-filter builder; the deduplication and formatting helpers.
Dependencies	The OpenAI embeddings API; the Pinecone index; environment configuration supplying both credentials and the index name.
Interfaces	Provided: <code>'embed_query'</code> , <code>'retrieve'</code> , <code>'retrieve_combined'</code> , <code>'format_context'</code> to the four agent modules. Required: <code>'embeddings.create'</code> and <code>'index.query'</code> .
Resources	One OpenAI client and one Pinecone client created once at import; approximately 1.5 megabytes of process memory for a 256-entry embedding cache.
Processing	The query is trimmed and lower-cased to raise the cache hit rate, embedded to 1,536 dimensions, and submitted with an equality filter on board, class and subject. Matches scoring below 0.20 are discarded. In combined mode, four textbook chunks and three past-paper chunks are retrieved, deduplicated on the first hundred characters and concatenated.

Attribute	Specification
Data	Query string; 1,536-dimension float vector; ranked matches carrying score, text, subject, type and chunk index.

5.4.3 Teacher Agent Component

Table 5.10: Component specification — Teacher Agent

Attribute	Specification
Identification	AI-01, Teacher Agent (ai-service/teacher_bot.py).
Type	Python module exposing a single entry point.
Purpose	Produce a syllabus-grounded, exam-oriented explanation of a concept in the student's own language. Realises REQ-12 to REQ-15 and REQ-35.
Function	'get_teacher_response' composes a search query, retrieves context, builds the prompt, assembles the message list with conversation memory, calls the model and returns the answer together with an updated conversation summary.
Subordinates	The retrieval component; the memory manager; the token logger.
Dependencies	prompts/teacher.txt, read once at import; the OpenAI chat-completions API; the shared retriever client.
Interfaces	Provided: 'get_teacher_response(question, board, class_level, subject, language, chat_history, existing_summary) → {answer, updated_summary}', surfaced by 'POST /chat'. Required: retrieval and chat-completion.
Resources	One outbound HTTPS call for generation, one for retrieval when the embedding is not cached, and one for summary maintenance.
Processing	For a follow-up question of fewer than six words, the first sentence of the existing summary is prepended before embedding, and the composed query is truncated to 300 characters. Generation runs at temperature 0.3 with a 1,000-token ceiling. The summary is updated only when the answer exceeds ten words, and a failure to update the summary is logged and swallowed rather than being allowed to fail the request.
Data	Question; academic scope; up to twenty prior turns; existing summary; retrieved context; generated answer; updated summary; token counts.

5.4.4 Tester and Evaluator Component

Table 5.11: Component specification — Tester and Evaluator

Attribute	Specification
Identification	AI-02, Tester and Evaluator (ai-service/tester_bot.py).
Type	Python module exposing two entry points.
Purpose	Generate board-styled practice questions and mark free-text answers against the board marking scheme. Realises REQ-16 to REQ-19, REQ-36 and REQ-37.
Function	'generate_questions' produces the requested number of items of the requested type from combined textbook and past-paper context, each labelled by provenance. 'evaluate_answer' marks one answer against retrieved context and returns a parsed score and short feedback.
Subordinates	The retrieval component in combined and simple modes; the structured-response parser; the score clamp; the token logger.

Attribute	Specification
Dependencies	prompts/tester.txt and prompts/evaluator.txt; the chat-completions API.
Interfaces	Provided: <code>'generate_questions(...) → str'</code> and <code>'evaluate_answer(...) → {score, feedback}'</code> , surfaced by <code>'POST /quiz/generate'</code> and <code>'POST /quiz/evaluate'</code> .
Resources	One generation call per request; one retrieval per request; up to 2,000 output tokens for generation and 200 for evaluation.
Processing	Generation runs at temperature 0.4 with past-paper content given explicit priority in the prompt. Evaluation runs at temperature 0.2 and requires a response whose first line matches SCORE: n/m and whose second begins FEEDBACK:. The score is parsed from that line and clamped to 1, 3 or 8 according to question type; a malformed response yields a score of zero and the raw text as feedback rather than an exception.
Data	Topic or question; academic scope; question type and count; student answer; retrieved context; generated items or score and feedback.

5.4.5 Session Evaluation Component

Table 5.12: Component specification — Session Evaluation

Attribute	Specification
Identification	AI-03, Session Evaluator (ai-service/overall_evaluator.py).
Type	Python module exposing a single entry point.
Purpose	Convert a completed attempt into a grade, strong and weak topic lists, encouragement and a revision recommendation. Realises REQ-19 and REQ-38.
Function	<code>'get_overall_feedback'</code> computes the arithmetic totals in Python, asks the model only for qualitative judgement, parses the structured reply and merges the two.
Subordinates	The structured-response parser; the grade-band function; the token logger.
Dependencies	prompts/overall_feedback.txt; the chat-completions API.
Interfaces	Provided: <code>'get_overall_feedback(board, class_level, subject, question_type, results) → dict'</code> , surfaced by <code>'POST /quiz/overall'</code> .
Resources	One generation call of at most 600 output tokens per completed attempt.
Processing	Total possible marks are computed as the number of questions multiplied by the per-type maximum, the percentage is rounded to two decimals, and the grade band is assigned at 80, 60 and 40 per cent. These values are computed in code and then written over whatever the model returns, so a model arithmetic error cannot reach the student. The model is asked only for STRONG_AREAS, WEAK_AREAS, OVERALL_FEEDBACK and STUDY_TIP.
Data	Per-question results; computed totals; parsed qualitative fields; the merged result dictionary.

5.4.6 Conversation Memory Component

Table 5.13: Component specification — Conversation Memory

Attribute	Specification
Identification	MEM-01, Memory Manager (ai-service/memory_manager.py).
Type	Python module of pure functions plus one model call.

Attribute	Specification
Purpose	Keep the prompt of a tutoring session bounded while preserving conversational continuity. Realises REQ-34 and NFR-5.
Function	`compress_history` splits a transcript into a verbatim tail and a compressible head; `_filter_messages` discards filler, short and duplicate turns; `_update_summary` maintains the rolling summary; `build_messages_with_memory` assembles the final message list.
Subordinates	The filler regular expression; the deduplication pass; the summariser prompt.
Dependencies	The shared OpenAI client from the retrieval module.
Interfaces	Provided: `build_messages_with_memory(system_prompt, chat_history, question, existing_summary) → (messages, summary)` and `_update_summary`, both used by the teacher agent.
Resources	At most one additional model call per turn, capped at 200 output tokens at temperature zero.
Processing	Transcripts of four turns or fewer are passed through untouched. Longer transcripts retain the last four turns verbatim; when a summary already exists only the four newest of the older turns are considered, which makes summarisation incremental rather than quadratic. Turns matching the filler pattern, shorter than eight words, or duplicating an earlier turn are removed. If nothing survives filtering, the existing summary is returned unchanged and no model call is made.
Data	Message list of role and content pairs; existing summary; updated summary; assembled message list.

5.4.7 Assessment Orchestration Component

Table 5.14: Component specification — Assessment Orchestration

Attribute	Specification
Identification	APP-02, Assessment Orchestration (`QuizController`, `AIController`).
Type	Laravel controller pair with an injected service.
Purpose	Mediate between the practice interface and the AI tier and persist every attempt. Realises REQ-16 to REQ-20 and REQ-38.
Function	Forward generation and evaluation requests; on session completion, obtain the aggregate result, create the quiz_sessions row, create one quiz_results row per question, append an activity entry and prune the feed.
Subordinates	`AIService`; the `QuizSession`, `QuizResult` and `Activity` models.
Dependencies	Sanctum authentication; the AI service; MySQL.
Interfaces	Provided: `POST /api/quiz/generate`, `POST /api/quiz/evaluate`, `POST /api/quiz/overall`. Required: the four AIService methods.
Resources	One database transaction context per completed attempt; one outbound HTTP call per question and one per session.
Processing	Evaluation is a pass-through, so a single slow question cannot block the others. On completion the aggregate is requested first, the session row is written with the returned aggregate fields, and each result row is written with is_correct derived from the score. The activity feed is then pruned to the eight most recent entries older than three minutes.
Data	Question type; per-question results; aggregate result; persisted session and result rows.

5.4.8 Content Management Component

Table 5.15: Component specification — Content Management

Attribute	Specification
Identification	APP-03, Content Management ('Admin\ContentController', 'ResourceController').
Type	Laravel controller pair with file-storage access.
Purpose	Curate and deliver downloadable study resources. Realises REQ-21 to REQ-25, REQ-39 and REQ-42.
Function	Validated upload with storage on the public disk; listing with board, class, subject and type filters; authenticated download with counter increment and activity logging; metadata edit; delete with file removal.
Subordinates	The 'Content' model; the storage disk; the 'Activity' model; the resource-card page component.
Dependencies	Sanctum authentication; the filesystem; MySQL.
Interfaces	Provided: 'POST GET /api/content', 'GET /api/content/download/{id}', 'PUT DELETE /api/content/{id}', and the '/resources' and '/admin/content' pages.
Resources	Disk capacity for uploaded documents; an upload ceiling of 100 megabytes per file.
Processing	Uploads are validated on title, type, board, class, subject, file presence and size before storage, so a rejected upload creates no record. Deletion removes the stored file before the record, so a failure cannot leave a record pointing at nothing.
Data	contents rows; stored files under storage/app/public/contents; activity rows of type resource.

5.4.9 Analytics Component

Table 5.16: Component specification — Analytics

Attribute	Specification
Identification	APP-04, Analytics ('DashboardController', 'ProgressController').
Type	Single-action Laravel controllers producing Inertia page props.
Purpose	Turn stored attempt history into an actionable picture of progress. Realises REQ-40 and REQ-41.
Function	Count questions practised; average the session percentages; group by subject to find the weakest; band each subject; compose a focus recommendation; return the recent activity feed.
Subordinates	The 'QuizSession', 'QuizResult' and 'Activity' models; the Recharts components on the progress page.
Dependencies	Session authentication; MySQL aggregate functions.
Interfaces	Provided: the '/dashboard' and '/progress' pages with their props.
Resources	Four aggregate queries per dashboard load; five per progress load.
Processing	Subject averages are computed by a grouped AVG over quiz_sessions and banded at 85, 70 and 60 per cent into Excellent, Strong, Good and Needs Work. The weakest subject is the minimum of that grouping and becomes the focus recommendation. An empty history yields defined empty states rather than division by zero.
Data	Aggregated percentages by subject; counts; the eight most recent activity rows.

5.4.10 Ingestion Pipeline Component

Table 5.17: Component specification — Ingestion Pipeline

Attribute	Specification
Identification	ING-01, Ingestion Pipeline (ai-service/scripts/01–03).
Type	Three sequential offline command-line scripts.
Purpose	Convert scanned board documents into a filtered, metadata-tagged vector index. Realises objective O2.
Function	Stage 1 rasterises and recognises text; stage 2 cleans, detects language and chunks; stage 3 embeds and upserts with resume support.
Subordinates	The path-metadata parser; the quality reporter; the general and past-paper cleaners; the chunker; the batch uploader; the embedded-identifier log.
Dependencies	Poppler; Google Cloud Vision with a service-account credential; the OpenAI embeddings API; the Pinecone control and data planes.
Interfaces	Provided: a command-line interface per stage. Required: the three external services above.
Resources	Ten decoded page images at a time; one hundred vectors per upsert batch; local disk for intermediate JSON and JSONL artefacts.
Processing	Every stage is idempotent: extraction skips source files whose output exists, chunking skips documents already chunked, and embedding consults the identifier log. Chunking uses a 2,800-character window with 400-character overlap, breaking at line or sentence boundaries and discarding fragments under 100 characters.
Data	Source PDFs; per-page JSON; chunk JSONL; 1,536-dimension vectors with board, class, subject, type, language, chunk index and truncated text metadata.

5.5 USER INTERFACE DESIGN

5.5.1 Design principles

- **One decision per screen.** The practice flow asks for chapter scope, then question type, then presents questions. Presenting all three at once was tried and abandoned during review because it produced a wall of controls before any value was delivered.
- **The academic profile is ambient.** Once board, class and subject are set they are applied everywhere and shown in the header rather than re-requested. A student who has not set them is offered an inline selector on the page that needs them, rather than being redirected away from their task.
- **Empty states carry instruction.** Every list — chat sessions, resources, progress, activity — has a designed empty state that says what will appear

there and how to make it appear, because a new account sees empty states everywhere.

- **Feedback is immediate and specific.** A marked answer shows the score, the feedback and the correct answer together, rather than deferring everything to the end of the session.
- **Responsive from 360 pixels.** The sidebar collapses to a sheet, tables become stacked cards and the chat occupies the full viewport on a mobile device.

5.5.2 Navigation model

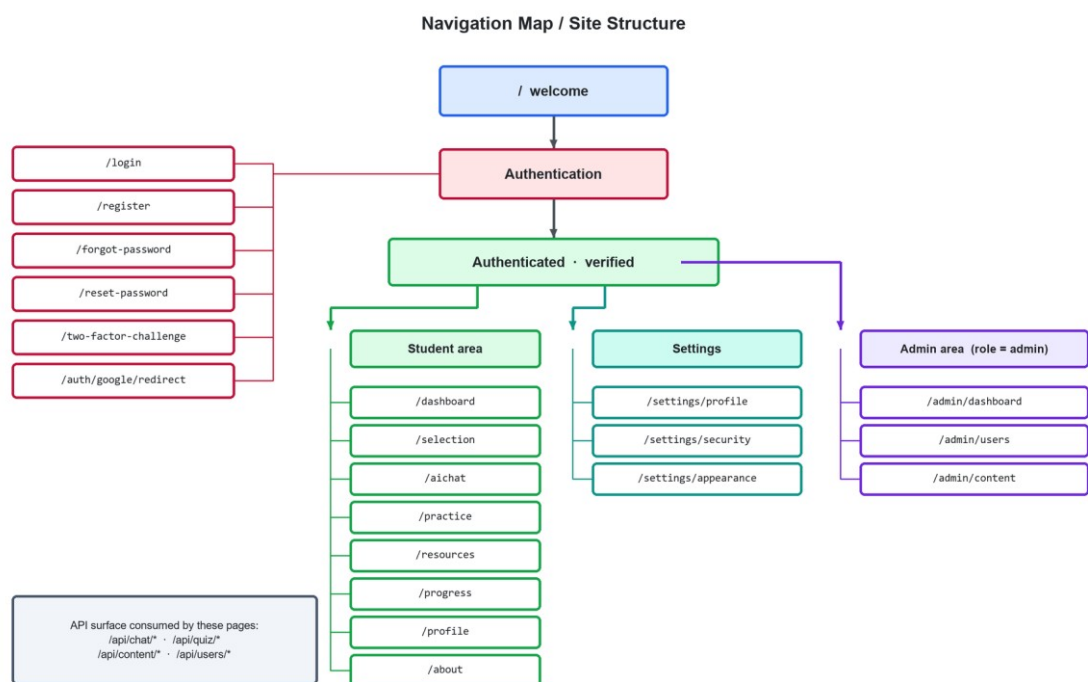


Figure 5.3: Navigation map showing the public entry point, the authentication routes, the authenticated student area, the account settings group and the role-gated administrative area.

5.5.3 Screen inventory

Table 5.18: Screen inventory

Route	Screen	Principal elements
<code>`/`</code>	Welcome	Product proposition, feature summary, sign-in and registration entry points.
<code>`/login`</code>	Sign in	Email and password fields, remember-me, Google button, links to registration and password reset.
<code>`/register`</code>	Register	Name, email, password and confirmation with inline validation.

Route	Screen	Principal elements
`/forgot-password`, `/reset-password`	Password recovery	Email request form and token-bound reset form.
`/two-factor-challenge`	Two-factor challenge	One-time-password input with a recovery-code alternative.
`/dashboard`	Student dashboard	Progress summary tiles, quick actions and the recent-activity feed.
`/selection`	Academic selection	Board, class and subject pickers with dependent options.
`/aichat`	AI tutor	Session list, transcript, composer, new-session and delete controls.
`/practice`	Practice	Scope selection, chapter multi-select, question-type selection, question card, per-answer feedback and the session result.
`/resources`	Resource library	Filter bar and resource cards with type, size and download action.
`/progress`	Progress	Overall statistics, subject-wise performance chart and focus recommendation.
`/profile`	Profile	Personal information and academic preference update.
`/settings/profile`, `/settings/security`, `/settings/appearance`	Account settings	Name and email, password change and two-factor management, light and dark theme.
`/about`	About	Project purpose, team and technology summary.
`/admin/dashboard`	Admin overview	Aggregate counts, recent registrations and quick links.
`/admin/users`	User management	Searchable, filterable account table with edit, block and delete.
`/admin/content`	Content management	Upload form and resource table with edit and delete.

5.6 PROMPT DESIGN

In a retrieval-grounded system the prompts are design artefacts of the same standing as the class model. They determine behaviour, they are versioned, and they live in separate template files that are loaded once at process start. Four templates exist, one per agent, and each is interpolated with the academic scope and the retrieved context before use.

5.6.1 Teacher prompt

The teacher prompt first fixes the role and the scope. It then sets a source-priority policy. Prefer the retrieved textbook or keybook content. If the exact topic is absent, answer from general knowledge but say so in a specified sentence. Never invent a textbook reference or a page number. Never reply that no response is available. It then fixes the shape of the answer by question type. A definition gets a

definition, a simple explanation, key points and an example. A comparison gets an X, Y and key-difference structure. A list question gets a numbered list. Mathematics gets a Given, To Find, Method and numbered-step layout that ends with units. Finally it fixes the language policy: reply in the language the student used, and keep Urdu natural and simple.

Design decision traced to an observed failure

The instruction never to say “no response” and the explicit permission to fall back to general knowledge were added in response to an observed failure. Early transcripts in the development database contain the reply “This topic is not in your Class class_11 biology syllabus.” to the question “cell theory” — a topic that is plainly in the syllabus but was not retrieved because that portion of the corpus had not yet been ingested. The system was refusing on the basis of a retrieval gap rather than a curriculum gap, and the prompt now distinguishes the two. The incident is discussed in Section 6.5.

5.6.2 Tester prompt

The tester prompt ranks its sources explicitly: past-paper questions first, because they recur, and textbook or keybook content second, for new items. It fixes the marks weighting at one, three and eight, and requires exactly the requested number of items of exactly the requested type. It also requires a provenance label on every question: a star for an item taken directly from a past paper, a tilde for an item written in past-paper style, and a filled star for an item newly written from the textbook. It then specifies the exact output layout for each question type, including the Correct and Explanation lines that the front end parses to build a multiple-choice card.

5.6.3 Evaluator prompt

The evaluator prompt states the marking scheme, then describes each band within each question type. For a three-mark short question: three for complete and correct, two for mostly correct with minor omissions, one for partially correct and zero for incorrect. It tells the model to mark only against the supplied textbook content and never from outside knowledge. It must give one sentence on what was correct and one on what was missing. For mathematics it must name the exact wrong step. It must never exceed the maximum for the question type. It closes by

fixing the output format to a SCORE line and a FEEDBACK line, which is what makes the response machine-parsable.

5.6.4 Session-evaluation prompt

The session-evaluation prompt receives the full record of the attempt and the totals already computed by the application, and is told in terms to use those totals and not to recalculate them. It asks for four labelled fields in a fixed format: strong areas, weak areas, overall feedback and a study tip. It tells the model to be encouraging but honest, and to name topics rather than give generic advice.

Table 5.19: Summary of the four prompt templates

Template	Fixes	Guards against
`prompts/teacher.txt`	Source priority, response shape by question type, bilingual policy, disclosure of general-knowledge answers	Refusing to answer because of a retrieval gap; fabricated textbook references; over-long answers
`prompts/tester.txt`	Past-paper priority, provenance labels, marks weighting, exact output layout	Out-of-syllabus questions; unparsable output; wrong item count or type
`prompts/evaluator.txt`	Marking bands per question type, mark-from-context rule, two-sentence feedback, SCORE and FEEDBACK lines	Marks above the maximum; discursive feedback; marking from outside knowledge
`prompts/overall_feedback.txt`	Use of application-computed totals; four labelled output fields	Model arithmetic errors reaching the student; vague, non-actionable advice

5.7 ALGORITHM DESIGN

5.7.1 Overlapping chunk segmentation

Listing 5.1: Overlapping chunk segmentation with boundary preference

```

procedure CHUNK_TEXT(text, chunk_tokens = 700, overlap_tokens = 100)
  char_size ← chunk_tokens × 4           // ≈ 2800 characters
  char_overlap ← overlap_tokens × 4       // ≈ 400 characters
  step ← char_size - char_overlap         // ≈ 2400 characters
  if step ≤ 0 then step ← char_size       // guard against non-advancing loop

  chunks ← [] ; start ← 0
  while start < length(text) do
    end ← min(start + char_size, length(text))

    if end < length(text) then             // prefer a clean
boundary                                boundary
      b ← last index of '\n' in text[start .. end]
      if b = -1 or (end - b) > 200 then
        b ← last index of '.' in text[start .. end]
      if b ≠ -1 and b > start then end ← b + 1

```

```

piece ← trim(text[start .. end])
if length(piece) > 100 then append piece to chunks // drop fragments

start ← start + step // always advance, never on `end`
return chunks

```

Source: Implemented in `ai-service/scripts/02_clean_and_chunk.py`

Two properties of this algorithm are worth noting. The cursor advances by a fixed step rather than by the position of the chosen boundary. Advancing on the boundary would let an awkward input, one with no boundary anywhere in a 2,800-character window, make no progress at all. And the boundary search is bounded: if the nearest line break is more than 200 characters back, the algorithm prefers a sentence break rather than discarding a fifth of the window.

5.7.2 Filtered retrieval

Listing 5.2: Metadata-filtered semantic retrieval

```

procedure RETRIEVE(query, board, class_level, subject, top_k = 5, doc_type = null)
  normalised ← lowercase(trim(query))
  if normalised ∈ embedding_cache then
    vector ← embedding_cache[normalised] // no metered call
  else
    vector ← EMBED(normalised, "text-embedding-3-small")
    embedding_cache[normalised] ← vector // LRU, capacity 256

  filter ← { board: eq board, class: eq class_level, subject: eq subject }
  if doc_type ≠ null then filter.type ← eq doc_type

  matches ← INDEX.query(vector, top_k, filter, include_metadata = true)
  return [ m ∈ matches where m.score ≥ 0.20 ] // relevance floor

```

Source: Implemented in `ai-service/rag/retriever.py`

5.7.3 Conversation memory compression

Listing 5.3: Bounded-prompt conversation memory

```

procedure BUILD_MESSAGES(system_prompt, history, question, summary)
  if length(history) ≤ 4 then
    recent ← history ; new_summary ← summary
  else
    older ← history[0 .. -5]
    recent ← history[-4 ..] // verbatim tail
    batch ← (summary ≠ "") ? last 4 of older : older // incremental
    useful ← [ m ∈ batch where not FILLER(m)
              and word_count(m) ≥ 8
              and m not already seen ]

    if useful = ∅ then
      new_summary ← summary // no model call at all
    else
      new_summary ← SUMMARISE(summary, useful) // ≤ 200 tokens, temp 0

  messages ← [ system: system_prompt ]
  if new_summary ≠ "" then
    append system: "[Conversation so far]: " + new_summary

```

```

append each of recent
append user: question
return (messages, new_summary)

```

Source: Implemented in ai-service/memory_manager.py

5.7.4 Score parsing, clamping and session aggregation

Listing 5.4: Defensive score handling and session aggregation

```

procedure PARSE_AND_CLAMP(model_text, question_type)
  score ← 0 ; feedback ← model_text
  for each line in model_text do
    if line starts with "SCORE:" then
      try
        score ← integer( part before '/' of value(line) )
      catch
        score ← 0 // malformed → zero, never an
  exception
    else if line starts with "FEEDBACK:" then
      feedback ← value(line)

  max_mark ← { mcq: 1, short: 3, long: 8 }[question_type] default 10
  score ← clamp(score, 0, max_mark) // the model cannot exceed the
  scheme
  return (score, feedback)

procedure AGGREGATE_SESSION(results, question_type)
  max_mark ← { mcq: 1, short: 3, long: 8 }[question_type] default 10
  total_possible ← count(results) × max_mark
  total_earned ← ∑ r.score for r ∈ results
  percentage ← total_possible > 0 ? round(100 × total_earned /
total_possible, 2) : 0
  grade ← percentage ≥ 80 ? "Excellent"
        : percentage ≥ 60 ? "Good"
        : percentage ≥ 40 ? "Needs Work"
        : "Poor"

  qualitative ← MODEL_JUDGEMENT(results) // strong / weak areas, feedback,
tip
  return qualitative ⊕ { total_earned, total_possible, percentage, grade }
  // the computed values overwrite anything the model returned for them

```

Source: Implemented in ai-service/tester_bot.py and ai-service/overall_evaluator.py

The final comment in that listing states the most important single rule in the system's design: the model is trusted for judgement and never for arithmetic. Every number a student sees is computed in code from values the model produced, and the computed values are written over the model's own totals rather than merged with them.

5.7.5 Bounded activity feed

Listing 5.5: Activity insertion with bounded retention

```

procedure RECORD_ACTIVITY(user_id, type, message)
  INSERT into activities (user_id, type, message, created_at = now())

  keep ← ids of the 8 most recent activities of user_id

```

```
DELETE from activities
where user_id = user_id
    and created_at < now() - 3 minutes      // never delete a just-written row
    and id ∉ keep
```

Source: Implemented in ChatController, QuizController and Admin\ContentController

The three-minute grace condition exists because the pruning statement runs immediately after the insert. Without it, a burst of activity within a single request could delete a row that the same request had just written.

5.8 SUMMARY

The schema has seven application tables in third normal form, with two deliberate denormalisations. Academic scope is frozen onto session rows, and aggregate results are stored rather than recomputed. Each is justified by a specific behavioural requirement. The class model is intentionally thin, with a single service class isolating the artificial-intelligence tier. Ten components have been specified against the standard ten attributes. The interface design follows five stated principles and comprises seventeen screens across four route groups. The four prompt templates are treated as versioned design artefacts, and one of them carries a rule traceable to an observed production failure. Five algorithms have been given in pseudocode, of which the score-clamping and aggregation logic expresses the system's central safety rule: the model is trusted for judgement, never for arithmetic. The next chapter documents how this design was implemented.

IMPLEMENTATION

This chapter documents how the design was realised in code. It records the size of the delivered system, the development environment, the repository and its history, and then walks module by module through the implementation, with annotated listings taken directly from the delivered source.

6.1 INTRODUCTION

This chapter documents how the design of Chapters 4 and 5 was realised in code. Section 6.2 records the development environment and the repository layout. Section 6.3 walks through the implementation of each module with annotated listings and screenshots of the resulting interface. Section 6.4 describes the integration between the two runtimes. Section 6.5 records the engineering problems that were encountered and how each was resolved — including two that changed the design.

6.1.1 Implementation statistics

Table 6.1: Size of the delivered codebase

Part of the system	Files	Lines	Language
Laravel application (app/)	42	1,845	PHP 8.4
Routes and configuration	17	1,709	PHP 8.4
Migrations, factories and seeders	18	698	PHP 8.4
React interface, hand written	100	10,640	TypeScript / TSX
FastAPI artificial-intelligence service	18	2,868	Python 3.12
Automated test suite	15	753	PHP (Pest)
Prompt templates	4	—	Plain text
Total, excluding vendor and generated code	214	18,513	—
Wayfinder-generated route helpers (not counted above)	71	8,795	TypeScript

Line counts exclude third-party dependencies and build output. The generated route helpers are listed separately because the Wayfinder tool produces them from the route definitions rather than a person writing them. They are real code that ships, but counting them as authored work would overstate the effort. Within the Python service the 2,868 lines divide into 828 lines of running service

code, 879 lines of one-off ingestion scripts and 1,161 lines of test and evaluation harness.

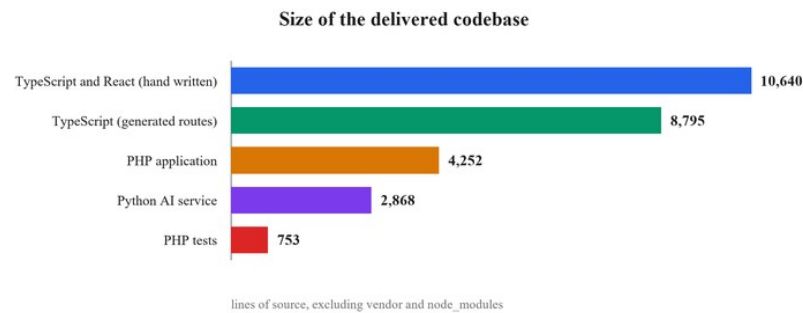


Figure 6.1: Size of the delivered codebase.

6.2 DEVELOPMENT ENVIRONMENT AND REPOSITORY STRUCTURE

6.2.1 Environment

Table 6.2: Development environment

Element	Configuration
Operating system	Windows 11 (development); Linux-compatible for deployment
PHP runtime	PHP 8.4 with Composer 2
Node runtime	Node.js with npm; Vite 8 development server
Python runtime	Python 3.12 in a project-local virtual environment
Database	MySQL 8 ('fyp_db')
Application server	`php artisan serve` in development
AI service	`uvicorn main:app --port 8001`
Concurrent start-up	`composer run dev` starts the web server, the queue listener and Vite together
Editors and tooling	Visual Studio Code; Laravel Pint; ESLint; Prettier; TypeScript compiler in no-emit mode
Version control	Git with a branch-per-concern workflow on GitHub

6.2.2 Repository layout

Listing 6.1: Repository structure (abridged)

```
fyp_backend/
├── app/
│   ├── Enums/
│   │   ├── Board.php
│   │   ├── ClassLevel.php
│   │   └── Subject.php
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── AIControllers/
│   │   │   │   ├── AIController
│   │   │   │   ├── ChatController
│   │   │   │   └── QuizController
│   │   │   ├── Admin/
│   │   │   │   ├── ContentController
│   │   │   │   └── DashboardController
│   │   └── UserController
```

```

├── api/
│   ├── AuthController
│   ├── Auth/
│   │   ├── GoogleController
│   │   ├── Settings/
│   │   │   ├── ProfileController
│   │   │   ├── SecurityController
│   │   │   ├── DashboardController
│   │   │   ├── PracticeController
│   │   │   ├── ProgressController
│   │   │   ├── ResourceController
│   │   │   ├── SelectionController
│   │   │   └── ProfileController
│   └── Middleware/
│       ├── HandleAppearance
│       ├── HandleInertiaRequests
│       └── Requests/Settings/
│           ├── ProfileUpdateRequest
│           ├── PasswordUpdateRequest ...
├── Models/
│   ├── User
│   ├── ChatSession
│   ├── ChatMessage
│   ├── QuizSession
│   ├── QuizResult
│   ├── Activity
│   └── Content
├── Providers/
│   ├── AppServiceProvider
│   └── FortifyServiceProvider
├── Services/
│   └── AIService.php
├── ai-service/
│   ├── main.py
│   │   └── FastAPI application and request/response models
│   ├── teacher_bot.py
│   │   └── explanation agent
│   ├── tester_bot.py
│   │   └── question generation and answer evaluation
│   ├── overall_evaluator.py
│   │   └── session-level aggregation
│   ├── memory_manager.py
│   │   └── conversation compression
│   ├── rag/retriever.py
│   │   └── embedding cache, filtered vector search
│   ├── prompts/
│   │   ├── teacher.txt
│   │   ├── tester.txt
│   │   ├── evaluator.txt
│   │   ├── overall_feedback.txt
│   ├── utils/token_logger.py
│   │   └── per-call token and cost accounting
│   └── scripts/
│       ├── 01_extract_text.py
│       ├── 02_clean_and_chunk.py
│       ├── 03_embed_and_store.py
│       └── test_*.py
├── database/migrations/
│   └── 15 migrations
├── resources/js/
│   ├── pages/
│   │   ├── welcome
│   │   ├── dashboard
│   │   ├── selection
│   │   ├── ai-chat
│   │   ├── practice
│   │   ├── resources
│   │   ├── progress
│   │   ├── profile
│   │   └── about
│   ├── layouts/
│   │   ├── auth/*
│   │   ├── settings/*
│   │   └── admin/*
│   ├── components/
│   │   ├── app
│   │   ├── auth
│   │   ├── student
│   │   └── admin
│   └── lib/api.ts
│       ├── question-card
│       ├── resource-card
│       ├── app-sidebar
│       ├── ui/*
│       ├── CSRF-aware
│       ├── fetch wrapper
├── routes/
│   ├── web.php
│   ├── api.php
│   ├── settings.php
│   └── console.php
├── tests/
│   ├── Feature/Auth/*
│   ├── Feature/Settings/*
│   └── Unit/*

```

6.2.3 Repository and version history

The project is held in a single Git repository at <https://github.com/rehandev55/fyp>, which contains the Laravel application, the React interface, the FastAPI service and the report build scripts together. A single repository was chosen over one per tier because the three tiers are versioned together. A change to the request contract in Section 4.9 touches a Python model, a PHP service class and a TypeScript type in the same commit. Splitting the repository would make such a change impossible to review as one unit.

Table 6.3: Repository at the time of submission

Property	Value
Remote	https://github.com/rehandev55/fyp
Commits	114 in total, 89 excluding merge commits
Contributing accounts	3
Branches	main, backend, latest, pc and feature/ai-service

Property	Value
First commit	7 April 2026
Most recent commit	10 September 2026
Active development window	Five months
Largest month	May 2026, with 49 commits

Work followed a branch-per-concern pattern. The main branch holds integrated work. The backend branch carried the Laravel and database work, and feature/ai-service carried the FastAPI service while its contract was still moving. The pc and latest branches were short-lived, and were used to move work between the machines the team used. Branches were merged into main once the automated suite passed, which is the point at which the suite began to earn its keep.

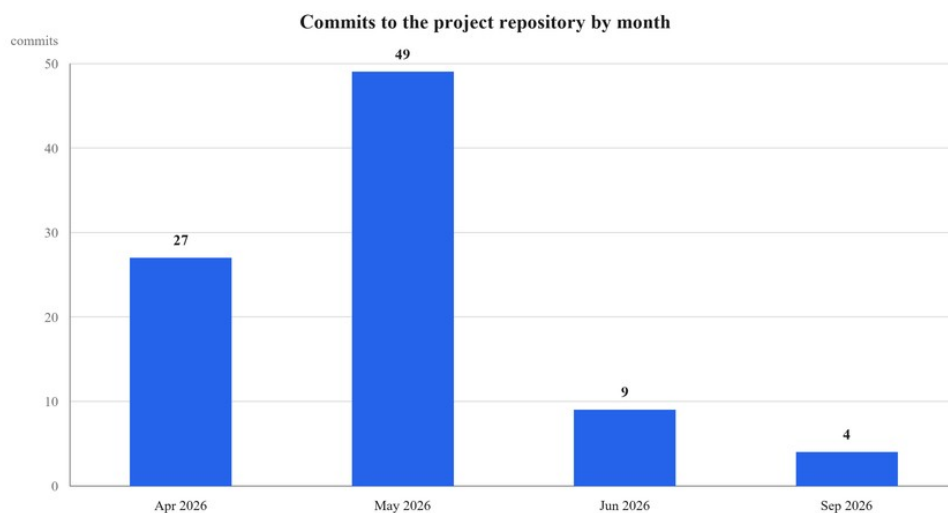


Figure 6.2: Commits to the project repository by month.

The commit profile follows the sprint plan in Table 3.1 closely. April and May 2026 account for 76 of the 89 non-merge commits and cover the bulk of construction, from the first scaffold through authentication, the ingestion pipeline, the three agents and the practice module. Activity falls sharply in June, when effort moved from writing code to writing the Software Requirements Specification and preparing the mid-project review. The September commits are the final round: administrative user management, the dark theme, voice input on the tutor composer, the download redirect fix recorded in the defect log, and the evaluation

harness of Section 7.6. This shape is typical of a student project with a fixed academic calendar, and is not in itself a fault in the process. It does mean the code was written faster than it was reviewed, which Section 9.5 records as a limitation.

6.3 MODULE IMPLEMENTATION

6.3.1 Curricular vocabulary

The three enumerations are the smallest classes in the system and the most widely depended upon. Every retrieval filter, every page prop and every database column that carries academic scope resolves back to one of them.

Listing 6.2: Backed enumeration with display label

```
namespace App\Enums;

enum ClassLevel: string
{
    case Class9 = 'class_9';
    case Class10 = 'class_10';
    case Class11 = 'class_11';
    case Class12 = 'class_12';

    public function label(): string
    {
        return match ($this) {
            self::Class9 => 'Class 9',
            self::Class10 => 'Class 10',
            self::Class11 => 'Class 11',
            self::Class12 => 'Class 12',
        };
    }
}
```

Source: app/Enums/ClassLevel.php — Board and Subject follow the same shape

The stored value `class_11` is precisely the value written into the class metadata field of every vector during ingestion, so a retrieval filter can be built directly from a model attribute with no mapping table. The label method is the only place the display form exists.

Listing 6.3: Attribute casting on the User model

```
protected function casts(): array
{
    return [
        'email_verified_at' => 'datetime',
        'password' => 'hashed',
        'two_factor_confirmed_at' => 'datetime',
        'board' => Board::class,
        'class_level' => ClassLevel::class,
        'subject' => Subject::class,
    ];
}
```

Source: *app/Models/User.php*

The hashed cast means a plain-text password assigned to the attribute is hashed on write, so it is not possible to store a password in recoverable form by forgetting to call the hasher. The three enumeration casts mean an invalid academic value cannot be read out of the database as a valid one.

6.3.2 Authentication

Registration, login, email verification, password reset, password confirmation and two-factor authentication are provided by Laravel Fortify, configured headlessly so that the interface remains a set of React pages. Nothing in this area was written by hand, which is deliberate: authentication is the part of a web application where original work is least welcome.

[Screenshot 6.3]

Capture <http://localhost:8000/login> at 1440 × 900 in a clean browser window.

Figure 6.3: The sign-in screen, showing email and password authentication alongside the federated Google option.

Federated sign-in is the one authentication path implemented directly. The controller redirects to Google, and on callback resolves the returned profile against the users table.

Listing 6.4: Google OAuth callback with account linking

```
public function callback()
{
    $googleUser = Socialite::driver('google')->user();

    $user = User::where('email', $googleUser->email)->first();

    if (! $user) {
        $user = User::create([
            'name'           => $googleUser->name,
            'email'          => $googleUser->email,
            'password'       => bcrypt('google-auth'),
            'role'           => 'user',
            'status'         => 'Active',
            'email_verified_at' => now(),
        ]);
    }
}
```

```

Auth::login($user);

return redirect('/dashboard');
}

```

Source: app/Http/Controllers/Auth/GoogleController.php

Two details realise REQ-7 and REQ-26. The lookup by email address is what links a Google sign-in to an account first created with a password. A student who registers by email and later clicks the Google button therefore reaches the same profile, not a duplicate. And `email_verified_at` is set at creation, because Google has already verified the address and asking the student to verify it again would be a pointless obstacle.

Known defect

Two weaknesses in this method are recorded honestly. The placeholder password is a fixed literal rather than a random string, and the role assigned on creation is 'user' where the rest of the system uses 'student'. Neither is exploitable as written — the placeholder is bcrypt-hashed and the role string is only compared for equality with 'admin' — but both are latent defects and are recorded as D-07 and D-08 in Section 7.7.

[Screenshot 6.4]

Capture <http://localhost:8000/settings/security> after enabling two-factor authentication; blur or replace the QR code before submission.

Figure 6.4: Two-factor authentication set-up on the account security page, showing the QR code and recovery codes.

6.3.3 Academic selection and profile

The selection page writes board, class and subject to the user record; every subsequent page reads them from the shared Inertia props. Some pages need the selection and may find the profile empty. The practice page is the important case. Those pages offer an inline selector instead of redirecting the student away from what they were trying to do.

[Screenshot 6.5]

Capture <http://localhost:8000/selection> with a board and class already chosen so the dependent subject list is visible.

Figure 6.5: Academic selection: board, class and subject, with subject options dependent on the selected class.

[Screenshot 6.6]

Capture <http://localhost:8000/profile> for a signed-in student.

Figure 6.6: The student profile page, showing personal information alongside the academic preferences that scope all generated content.

6.3.4 The AI service boundary

One class stands between the entire PHP codebase and the AI tier.

Listing 6.5: The sole adapter to the FastAPI service

```
class AIService
{
    private string $baseUrl;

    public function __construct()
    {
        $this->baseUrl = config('services.ai.url');
    }

    public function chat(string $question, string $board, string $classLevel,
        string $subject, $chatHistory = [], $existingSummary =
        null): array
    {
        $response = Http::post("{ $this->baseUrl } /chat", [
            'question'      => $question,
            'board'         => $board,
            'class_level'   => $classLevel,
            'subject'       => $subject,
            'chat_history'  => $chatHistory,
            'existing_summary' => $existingSummary ?? '',
        ]);

        $data = $response->json();
    }
}
```

```

        return [
            'reply'          => $data['reply'] ?? $data['answer'] ?? 'No
response',
            'updated_summary' => $data['updated_summary'] ?? null,
        ];
    }

    public function generateQuiz(array $params): array { ... }
    public function evaluateQuiz(array $params): array { ... }
    public function overallQuiz(array $params): array { ... }
}

```

Source: *app/Services/AIService.php*

The base URL comes from configuration rather than from a literal, so the service can be moved to another host without a code change. The null-coalescing chain on the response accommodates both possible key names and degrades to a readable string rather than a null-pointer error if the service returns something unexpected.

6.3.5 Retrieval

The retriever is the component on which the project's central claim depends.

Listing 6.6: Cached embedding and metadata-filtered search

```

_openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
_pc             = Pinecone(api_key=os.getenv("PINECONE_API_KEY"))
_index         = _pc.Index(os.getenv("PINECONE_INDEX"))

EMBEDDING_MODEL = "text-embedding-3-small"

@functools.lru_cache(maxsize=256)
def _embed_cached(text: str) -> tuple:
    response = _openai_client.embeddings.create(input=text, model=EMBEDDING_MODEL)
    return tuple(response.data[0].embedding)      # tuple: hashable for the cache

def embed_query(text: str) -> list[float]:
    normalized = text.strip().lower()             # raises the cache hit rate
    return list(_embed_cached(normalized))

def retrieve(query, board, class_level, subject, top_k=5, doc_type=None) ->
list[dict]:
    query_vector = embed_query(query)

    metadata_filter = {
        "board": {"$eq": board},
        "class": {"$eq": class_level},
        "subject": {"$eq": subject},
    }
    if doc_type:
        metadata_filter["type"] = {"$eq": doc_type}

    results = _index.query(vector=query_vector, top_k=top_k,
                           filter=metadata_filter, include_metadata=True)

```

```

    return [
        {"score": round(m.score, 4), "text": m.metadata.get("text", ""),
         "subject": m.metadata.get("subject", ""), "type": m.metadata.get("type",
""),
         "chunk_index": m.metadata.get("chunk_index", 0)}
        for m in results.matches
        if m.score >= 0.2                                # relevance floor
    ]

```

Source: ai-service/rag/retriever.py

The provider clients are created once at import rather than per request, which removes connection set-up from the critical path. The cache returns a tuple because the standard library's cache requires a hashable result; the public wrapper converts it back to a list for the vector client. The three-way equality filter is the mechanism by which curricular scoping becomes structural.

Listing 6.7: Single-embedding combined retrieval for question generation

```

def retrieve_combined(query, board, class_level, subject) -> str:
    textbook_chunks = retrieve(query, board, class_level, subject,
                               top_k=4, doc_type="textbook")
    past_paper_chunks = retrieve(query, board, class_level, subject,
                                top_k=3, doc_type="past_papers")

    seen_texts, all_chunks = set(), []
    for chunk in textbook_chunks + past_paper_chunks:
        key = chunk["text"][:100]
        if key not in seen_texts:
            seen_texts.add(key)
            all_chunks.append(chunk)

    return format_context(all_chunks)

```

Source: ai-service/rag/retriever.py

Both calls embed the same query string, so the second is served from the cache and costs nothing. An earlier version embedded the topic twice; consolidating it removed one metered API call from every question-generation request.

6.3.6 The teacher agent and the chat module

The teacher agent composes the search query, retrieves, builds the message list through the memory manager, calls the model, logs the token usage and maintains the summary.

Listing 6.8: Context-aware query construction and generation

```

if existing_summary and len(question.split()) < 6:
    summary_context = existing_summary.split(".")[0]          # first sentence only
    search_query = f"{summary_context}. {question}"[:MAX_SEARCH_QUERY_CHARS]
else:

```

```

search_query = question[:MAX_SEARCH_QUERY_CHARS]          # 300 characters

chunks = retrieve(search_query, board, class_level, subject, top_k=5)
context = format_context(chunks)

system_prompt = PROMPT_TEMPLATE.format(board=board, class_level=class_level,
                                         subject=subject, context=context,
                                         question=question)

messages, updated_summary = build_messages_with_memory(
    system_prompt=system_prompt, chat_history=chat_history,
    question=question, existing_summary=existing_summary)

response = client.chat.completions.create(
    model="gpt-4o-mini", messages=messages, temperature=0.3, max_tokens=1000)

answer = response.choices[0].message.content

```

Source: *ai-service/teacher_bot.py*

The first branch fixes a real usability problem. A student who types “explain more” after an answer about the nitrogen cycle is still asking about the nitrogen cycle, but those two words on their own retrieve nothing. Prefixing the first sentence of the running summary restores the topic without lengthening the prompt materially.

Listing 6.9: Summary maintenance that cannot fail the request

```

final_summary = updated_summary or existing_summary
if answer and len(answer.split()) > 10:
    try:
        final_summary = _update_summary(
            existing_summary=updated_summary or existing_summary,
            new_messages=[{"role": "user", "content": question},
                          {"role": "assistant", "content": answer}])
    except Exception as e:
        print(f"[WARNING] Summary update failed: {e}")

return {"answer": answer, "updated_summary": final_summary}

```

Source: *ai-service/teacher_bot.py*

Summary maintenance is a convenience, not a correctness requirement. If it fails, the student must still receive their answer, so the failure is logged and the previous summary is retained. The ten-word guard prevents a trivial exchange from consuming a summarisation call.

On the application side, ChatController resolves the academic scope, loads recent history, persists both sides of the exchange and updates the stored summary.

Listing 6.10: Scope resolution and history assembly in the chat controller

```

$board = Board::tryFrom($request->input('board')) ?? $user->board ??
Board::Federal;
$classLevel = ClassLevel::tryFrom($request->input('class_level'))

```

```

        ?? $user->class_level ?? ClassLevel::Class10;
$subject = Subject::tryFrom($request->input('subject'))
        ?? $user->subject ?? Subject::Physics;

if ($request->session_id) {
    $session = ChatSession::findOrFail($request->session_id);
    $board = $session->board; // an existing session keeps its own
scope
    $classLevel = $session->class_level;
    $subject = $session->subject;
} else {
    $session = ChatSession::create([
        'user_id' => $user->id,
        'title' => substr($request->message, 0, 40),
        'board' => $board,
        'class_level' => $classLevel,
        'subject' => $subject,
    ]);
}

$history = ChatMessage::where('session_id', $session->id)
    ->orderBy('created_at', 'desc')->take(10)->get()->reverse();

$chatHistory = $history->map(fn ($m) => [
    'role' => $m->role, 'content' => $m->message,
])->values()->toArray();

```

Source: *app/Http/Controllers/AIControllers/ChatController.php*

The three-level fallback — request, then profile, then a safe default — means the chat endpoint never fails for want of an academic scope. The reassignment inside the first branch implements the design decision of Section 5.2.1: a resumed session keeps the scope it was created with, even if the student has since changed their profile.

[Screenshot 6.7]

Capture <http://localhost:8000/aichat> with an active conversation of at least four exchanges visible.

Figure 6.7: The AI tutor interface: session list on the left, transcript in the centre and the composer beneath.

[Screenshot 6.8]

Capture a single answer in the transcript, scrolled so the full response is visible; a Biology or Physics definition question works well.

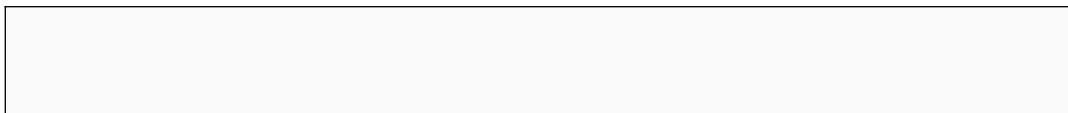


Figure 6.8: A grounded explanation produced by the teacher agent, structured as definition, key points and example.

6.3.7 The practice and assessment module

The practice page holds the curriculum map — 379 chapters across 28 class-and-subject combinations — as a static structure keyed by class and subject. It is static because it is a curriculum, not data: it changes when a board revises its syllabus, not when a user acts.

Listing 6.11: Curriculum map and chapter lookup (abridged)

```
const CURRICULUM_DATA = {
  class_9: {
    physics: ['PHYSICAL QUANTITIES AND MEASUREMENT', 'KINEMATICS',
              'DYNAMICS - I', 'DYNAMICS - II', ... ],
    chemistry: ['Nature of Science in Chemistry', 'Matter', 'Atomic Structure',
    ... ],
    ...
  },
  class_10: { ... }, class_11: { ... }, class_12: { ... },
};

function getChapters(classLevel: string, subject: string) {
  if (!classLevel || !subject) return [];
  const classData = (CURRICULUM_DATA as any)[classLevel];
  if (!classData) return [];
  return classData[subject] || [];
}
```

Source: *resources/js/pages/practice.tsx*

[Screenshot 6.9]

Capture <http://localhost:8000/practice> at the chapter-selection step with several chapters ticked.

Figure 6.9: Practice set-up: whole-book or selected-chapter scope with the chapter list of the student's class and subject.

[Screenshot 6.10]

Capture the practice page at the question-type step.



Figure 6.10: Question-type selection between multiple-choice, short and long formats, with the marks weighting shown.

Generated multiple-choice questions arrive as formatted text and are parsed on the client into a structure the question card can render.

Listing 6.12: Parsing generated multiple-choice items

```
function parseQuestions(raw: string) {
  return raw.split("\n\n").map((block) => {
    const lines = block.split("\n");
    const question = lines[0]?.replace(/^Q\d+\.\s*/, "") || "";

    const options = lines
      .filter(line => line.match(/^[A-D]\s/))
      .map(line => line.replace(/^[A-D]\s/, ""));

    const correctLine = lines.find(line => line.startsWith("Correct:"));
    const correctLetter = correctLine?.split(":")[1]?.trim() || "";
    const correctAnswer = ["A", "B", "C", "D"].indexOf(correctLetter);

    const explanationLine = lines.find(line =>
line.startsWith("Explanation:"));
    const explanation = explanationLine?.replace("Explanation:", "").trim() ||
    "";

    return { chapter: "", question, options, correctAnswer, explanation };
  });
}
```

Source: resources/js/pages/practice.tsx

This parser is the reason the tester prompt fixes an exact output format. The coupling between a prompt and a parser is a real and somewhat uncomfortable property of systems like this. The prompt is, in effect, an interface definition. Section 9.6 recommends replacing it with structured JSON output.

[Screenshot 6.11]

Capture a rendered question card showing the 🌟 Past Paper or ~ Similar label.

Figure 6.11: A generated multiple-choice question with its past-paper provenance label and four options.

[Screenshot 6.12]

Capture the feedback state after submitting a short-question answer.

Figure 6.12: Per-answer feedback from the evaluator, showing the mark awarded and a two-sentence explanation of what was correct and what was missing.

Marking is a pass-through on the application side. Aggregation, by contrast, is where the application takes control of every number the student sees.

Listing 6.13: Session aggregation and persistence

```
public function overall(Request $request)
{
    $response = $this->ai->overallQuiz($request->all());

    $session = QuizSession::create([
        'user_id'      => $request->user()->id,
        'subject'      => $request->subject,
        'board'        => $request->board,
        'class_level'  => $request->class_level,
        'question_type' => $request->question_type,
        'total_score'   => $response['total_score'] ?? null,
        'percentage'    => $response['percentage'] ?? null,
        'grade'         => $response['grade'] ?? null,
        'strong_areas'  => $response['strong_areas'] ?? [],
        'weak_areas'    => $response['weak_areas'] ?? [],
        'overall_feedback' => $response['overall_feedback'] ?? null,
        'study_tip'     => $response['study_tip'] ?? null,
        'completed_at'  => now(),
    ]);

    foreach ($request->results as $result) {
        QuizResult::create([
            'session_id' => $session->id,
            'user_id'    => $request->user()->id,
            'question'   => $result['question'] ?? '',
            'student_answer' => $result['student_answer'] ?? '',
            'score'       => (float) ($result['score'] ?? 0),
            'feedback'    => $result['feedback'] ?? '',
            'is_correct'  => (float) ($result['score'] ?? 0) >= 5,
        ]);
    }

    return response()->json(['session_id' => $session->id, ...$response]);
}
```

Source: `app/Http/Controllers/AIControllers/QuizController.php`

Known defect

The `is_correct` flag is derived with a fixed threshold of five marks, which is correct for long questions and wrong for the other two types: a perfect multiple-choice answer scores one and is therefore recorded as incorrect. The flag is not used in any calculation the student

sees — every displayed figure derives from the score column — so no reported result is affected, but the column is unreliable for future analytics. This is recorded as defect D-01 in Section 7.7 and the corrected form is given there.

[Screenshot 6.13]

Capture the completed-session summary after finishing a five-question practice test.

Figure 6.13: The session result: total score, percentage, grade band, strong and weak areas, encouragement and a study tip.

6.3.8 Progress analytics

Listing 6.14: Subject-wise performance banding

```
$subjects = QuizSession::selectRaw('subject, AVG(percentage) as score')
->where('user_id', $userId)
->groupBy('subject')
->get()
->map(function ($s) {
    $score = round($s->score);
    return [
        'name'    => ucfirst($s->subject),
        'score'   => $score,
        'status'  => $score >= 85 ? 'Excellent'
                  : ($score >= 70 ? 'Strong'
                  : ($score >= 60 ? 'Good' : 'Needs Work')),
    ];
});

$weakest    = $subjects->sortBy('score')->first();
$focusArea = $weakest
    ? $weakest['name'] . " needs more practice. Try taking more quizzes and "
    . "reviewing AI explanations."
    : "Great job! Keep practicing to maintain your performance.";
```

Source: app/Http/Controllers/ProgressController.php

Aggregation is performed in SQL rather than in PHP, so the number of rows transferred is the number of subjects rather than the number of attempts. The empty history case is handled explicitly, which is the state every new account is in.

[Screenshot 6.14]

Capture <http://localhost:8000/dashboard> for an account with several completed quizzes so the tiles carry real values.



Figure 6.14: The student dashboard: progress summary tiles, quick actions and the recent activity feed.

[Screenshot 6.15]

Capture <http://localhost:8000/progress> for an account with attempts in at least three subjects.

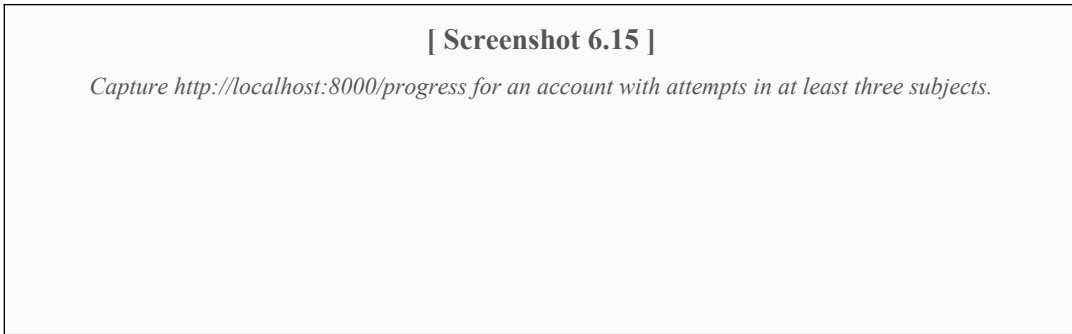


Figure 6.15: The progress page: overall statistics, subject-wise performance with status bands and the focus recommendation.

6.3.9 Resource library and administration

Listing 6.15: Validated upload and accounted download

```
public function store(Request $request)
{
    $request->validate([
        'title'      => 'required',
        'type'       => 'required',
        'board'      => 'required',
        'class_level' => 'required',
        'subject'    => 'required',
        'file'       => 'required|file|max:102400',    // 100 MB ceiling
    ]);

    $file = $request->file('file');
    $path = $file->store('contents', 'public');

    $content = Content::create([... 'file_path' => $path,
                                   'file_size' => $file->getSize()]);

    return response()->json(['success' => true, 'data' => $content]);
}

public function download($id)
{
    $content = Content::findOrFail($id);
    $content->increment('downloads');

    Activity::create([
        'user_id' => Auth::id(),
        'type'    => 'resource',
        'message' => 'Downloaded ' . $content->title,
    ]);

    return response()->download(storage_path('app/public/' . $content->file_path));
}
```

Source: `app/Http/Controllers/Admin/ContentController.php`

[Screenshot 6.16]

Capture `http://localhost:8000/resources` with several resources listed and a filter applied.

Figure 6.16: The student resource library with board, class, subject and type filters.

[Screenshot 6.17]

Capture `http://localhost:8000/admin/content` signed in as an administrator.

Figure 6.17: The administrative content console: upload form and the resource table with edit and delete actions.

[Screenshot 6.18]

Capture `http://localhost:8000/admin/users`; obscure real email addresses before submission.

Figure 6.18: The administrative user console with search, role and status filters and per-account actions.

[Screenshot 6.19]

Capture `http://localhost:8000/admin/dashboard`.

Figure 6.19: The administrative overview showing aggregate counts and recent registrations.

6.3.10 Cost instrumentation

Every call to a metered model passes through one logging function, which computes the monetary cost from the published rates and appends a structured record.

Listing 6.16: Per-call token and cost accounting

```
PRICING = {
    "gpt-4o-mini": {"input": 0.150, "output": 0.600}, # USD per 1M
    tokens
    "gpt-4o": {"input": 2.50, "output": 10.00},
    "text-embedding-3-small": {"input": 0.020, "output": 0.000},
}

def log_token_usage(endpoint, model, board, class_level, subject,
                    prompt_tokens, completion_tokens):
    pricing = PRICING.get(model, {"input": 0, "output": 0})
    input_cost = (prompt_tokens / 1_000_000) * pricing["input"]
    output_cost = (completion_tokens / 1_000_000) * pricing["output"]
    total_cost = round(input_cost + output_cost, 8)

    record = {
        "timestamp": datetime.now().isoformat(), "endpoint": endpoint,
        "model": model, "board": board, "class_level": class_level,
        "subject": subject, "prompt_tokens": prompt_tokens,
        "completion_tokens": completion_tokens,
        "total_tokens": prompt_tokens + completion_tokens,
        "cost_usd": total_cost,
    }
    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write(json.dumps(record) + "\n")
```

Source: ai-service/utils/token_logger.py

Recording board, class and subject alongside the token counts lets the log answer questions the billing dashboard cannot. Which subject is most expensive to serve? Does long-question marking cost more than short-question marking? Has a prompt change increased consumption? The output is JSON Lines, so it can be analysed with a few lines of Python. Section 8.6 reports the analysis.

6.4 INTEGRATION BETWEEN THE TWO RUNTIMES

The two runtimes are joined by four HTTP calls and one configuration value. The application reads `services.ai.url` from the environment, defaulting to `http://127.0.0.1:8001` in development, and the AI service listens there. The service holds no database credential and issues no query; everything it needs arrives in the request body and everything it produces is returned for the caller to persist.

Request validation lives at the boundary rather than on both sides of it. Every field of every endpoint is declared as a Pydantic model attribute with explicit

bounds. A question may be at most a thousand characters and an answer at most three thousand. A request may carry at most twenty history items, between one and twenty questions, and at most fifty results in a session aggregation. An over-long or malformed request is therefore rejected with a structured error before any metered operation begins.

Listing 6.17: Typed request contract at the service boundary

```
class ChatRequest(BaseModel):
    question:      str = Field(..., max_length=MAX_QUESTION_LEN)   # 1000
    board:         str
    class_level:   str
    subject:       str
    language:      str = "en"
    chat_history:  list = Field(default_factory=list,
    max_length=MAX_HISTORY_ITEMS)
    existing_summary: str = Field(default="", max_length=2000)

@app.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest):
    try:
        result = get_teacher_response(
            question=req.question, board=req.board, class_level=req.class_level,
            subject=req.subject, language=req.language,
            chat_history=req.chat_history, existing_summary=req.existing_summary)
        return ChatResponse(answer=result["answer"],
                             updated_summary=result["updated_summary"])
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Source: ai-service/main.py

These bounds are cost controls as well as correctness controls. They put a hard ceiling on the tokens any single request can be billed for. That is the difference between a bug that produces an error and a bug that produces an invoice.

6.5 ENGINEERING CHALLENGES AND RESOLUTIONS

6.5.1 Refusing to answer because of a retrieval gap

The most instructive failure of the project appears in the development database. To the question “cell theory”, asked by a Class 11 Biology student, the system replied “This topic is not in your Class class_11 biology syllabus.” Cell theory is unambiguously in that syllabus. The problem was architectural, not curricular. That part of the corpus had not yet been ingested, so retrieval returned nothing above the similarity floor. The prompt of the time then told the model to decline whenever it had no context. The system was reporting a gap in its own index as a gap in the student's syllabus.

Two other defects are visible in the same reply. The raw enumeration token `class_11` leaked into user-facing text instead of the display label, and the refusal was worded as a statement of fact about the syllabus rather than about the system's own coverage. The fix was to rewrite the teacher prompt so that it separates the two cases. Prefer textbook content when it is retrieved. When it is not, answer from general knowledge and say so in a specified sentence. Never invent a textbook reference, and never reply that no response is available. Later transcripts in the same database show the corrected behaviour — a structured explanation of what software is, with system and application software distinguished and examples given, for a question that no textbook chunk matched.

The general lesson is the one the team regards as the most valuable of the project. In a retrieval-grounded system, absence of evidence is not evidence of absence. A design that treats the two as the same thing will confidently mislead the very users it is meant to help.

6.5.2 Cost growth with conversation length

The first working chat implementation resent the entire transcript on every turn. The token log made the consequence visible immediately: input tokens, and therefore cost and latency, grew linearly with session length, so a productive forty-turn study session was disproportionately expensive precisely because it was productive. The resolution was the memory manager described in Sections 4.9 and 5.4.6. The instrumentation that made the problem visible was itself the reason it was caught in development rather than in an invoice.

6.5.3 Optical character recognition noise

Raw recognition output from scanned board material proved unusable for retrieval. Publisher watermarks appeared on every page of the keybooks; examination papers carried roll-number boxes, board headers, time allocations and instruction lines; empty multiple-choice bubbles were recognised as strings of the letter o. Because these fragments repeat across hundreds of pages they behave as high-frequency noise and degrade the embedding of every chunk that contains them. The fix was two distinct rule sets: a general one, and a much more aggressive one applied only to documents typed as past papers. Both were developed by

reading the recognised text of sample documents. Both are written as explicit regular expressions rather than as a learned filter, so that a wrongly deleted passage can be traced to the rule that deleted it.

6.5.4 Memory exhaustion during ingestion

The first extraction script converted an entire PDF to page images before recognising any of them. On a six-hundred-page textbook this exhausted the memory of a development laptop. The resolution was to determine the page count first through `pdftinfo`, convert in batches of ten, and release each decoded image immediately after recognition. The resolution rate was also lowered from 200 to 150 dots per inch after comparison showed no measurable loss of recognition quality on this material.

6.5.5 Losing an ingestion run to a network failure

Optical character recognition and embedding are the two metered operations in the pipeline, and an interruption after four hundred pages meant paying for those pages again. Every stage was made idempotent: extraction skips source files whose output JSON already exists, chunking skips documents already chunked, and embedding consults a log of successfully embedded identifiers written incrementally. A restart resumes where it stopped.

6.5.6 Impossible marks

During evaluation testing the model occasionally returned a score outside the permitted range for the question type, and occasionally returned a response that did not match the required format at all. Trusting the model's number would have produced percentages above one hundred. The fix has two parts. The score is parsed from a designated line and clamped in code to the maximum for the question type. If parsing fails, the result is a score of zero with the raw text as feedback, rather than an exception. The same principle was then applied to session aggregation, where the totals, percentage and grade are computed in Python and written over whatever the model returned.

6.5.7 Inconsistent stored vocabulary

Early development stored display strings — 'Federal Board', '9', 'Physics' — while the vector index carried tokens — 'federal', 'class_9', 'physics'. Retrieval filters therefore matched nothing for records created before the enumerations were introduced. The resolution was a reversible data migration that normalised the existing rows across three tables, and the introduction of backed enumerations so that the two vocabularies cannot diverge again.

6.5.8 Cross-site request forgery in asynchronous calls

Inertia form submissions carry the CSRF token automatically; the asynchronous fetch calls used for chat, marking and file upload did not, and failed with a token mismatch. Rather than add the header at each of the several dozen call sites, one wrapper was written. It obtains the cookie if it is absent, attaches the token header and sets the content type. It deliberately omits the content type for FormData, so the browser can supply the multipart boundary itself. Setting that header by hand silently breaks uploads.

Listing 6.18: CSRF-aware fetch wrapper

```
export async function api(path: string, options: RequestInit = {}):
  Promise<Response> {
    await ensureCsrf(); // fetch cookie if missing

    const headers: Record<string, string> = {
      Accept: 'application/json',
      ...(options.headers as Record<string, string>),
    };

    const xsrf = getCookie('XSRF-TOKEN');
    if (xsrf) headers['X-XSRF-TOKEN'] = xsrf;

    // do not set Content-Type for FormData – the browser must add the boundary
    if (!(options.body instanceof FormData) && !headers['Content-Type']) {
      headers['Content-Type'] = 'application/json';
    }

    return fetch(`/api${path}`, { credentials: 'include', ...options, headers });
  }
```

Source: *resources/js/lib/api.ts*

6.5.9 Unbounded activity growth

The activity feed shows eight entries but was accumulating a row for every chat message, quiz completion and download indefinitely. Pruning immediately after insertion introduced a second problem: the pruning statement could delete the

row the same request had just written. The resolution retains the eight most recent rows and deletes only rows older than three minutes, so a just-written row is never a candidate.

6.6 SUMMARY

The delivered system comprises approximately 18,500 lines of first-party code across three runtimes. Curricular vocabulary is expressed as backed enumerations whose stored values are identical to the metadata written into the vector index. Authentication is delegated to audited framework packages, with only the federated sign-in path written by hand. A single service class isolates the artificial-intelligence tier from the rest of the application. The retriever enforces curricular scoping as a metadata pre-filter and caches embeddings to avoid repeated metered calls. Marking is defended by parsing and clamping, and every displayed number is computed in application code. Nine engineering problems were encountered and resolved, two of which — the refusal caused by a retrieval gap and the cost growth with conversation length — changed the design rather than merely the code. The next chapter reports how the result was verified.

SYSTEM TESTING

This chapter describes how the system was verified. It states the testing strategy, reports the automated suite, presents the functional and non-functional test cases, describes the harness written to evaluate the retrieval and language-model layers directly, records the integration and system testing, and lists every defect found.

7.1 INTRODUCTION

This chapter reports how the system was verified. Section 7.2 states the strategy and the levels at which testing was applied. Section 7.3 reports the automated suite, including two failures that were left failing deliberately and what they revealed. Sections 7.4 to 7.6 present the functional and non-functional test-case catalogue. Section 7.7 gives the defect log with the current status of each entry, and Section 7.8 summarises coverage together with an honest statement of what was not tested.

7.2 TESTING STRATEGY

Testing was organised in four levels, illustrated in Figure 7.1. Breadth of coverage is greatest at the base and business risk addressed is greatest at the apex.

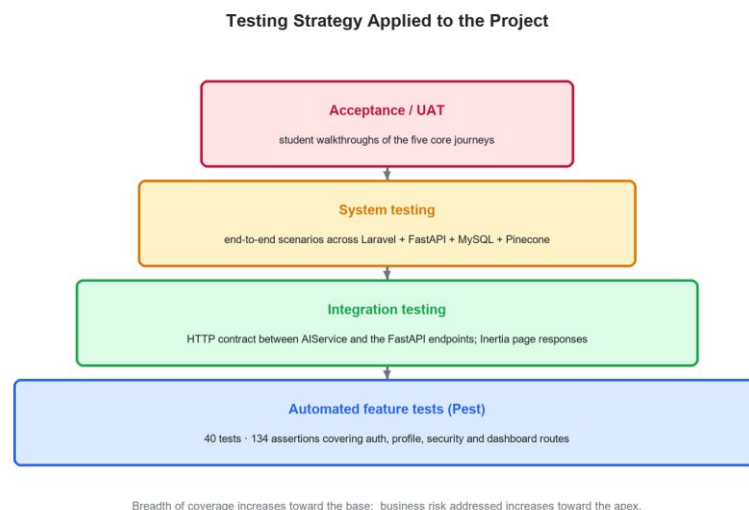


Figure 7.1: The four levels of testing applied to the project and the coverage each provides.

- **Automated feature tests.** A Pest suite exercising authentication, profile, security and dashboard routes through the full HTTP stack against a migrated test database. These run in seconds and were executed on every significant change.
- **Integration testing.** Verification of the contract between the Laravel application and the FastAPI service, and of the AI service against its own external dependencies, performed with the standalone scripts in `ai-service/scripts` and with Postman collections.
- **System testing.** End-to-end execution of five complete user journeys with all four runtimes and both external services live.
- **Acceptance testing.** Student volunteers performing the same journeys unaided, with observation rather than instruction.

The distribution of effort was deliberate. Authentication and account management are the areas where a defect is both most likely to be exploited and most amenable to automation, so they carry the automated suite. The artificial-intelligence behaviours are non-deterministic and therefore resist assertion-based testing; they were verified by structured manual scenarios with recorded outcomes, which is acknowledged as a limitation in Section 7.8.

7.3 AUTOMATED TEST SUITE

The automated suite is written with Pest 4 on PHPUnit 12 and runs against an isolated test database that is migrated and rolled back for every test. The whole suite executes in about two and a half seconds, which is short enough that it was run before every commit rather than only before a release. Table 7.1 gives its composition. The suite contains forty-five tests making one hundred and forty-seven assertions, and all of them pass.

Table 7.1: Automated test suite composition and result

Test file	Area covered	Tests	Result
Auth/AuthenticationTest	Login screen, credential check, two-factor redirect, logout, rate limiting	6	Pass
Auth/EmailVerificationTest	Verification screen, signed-link fulfilment, invalid hash and identifier, repeat visits	6	Pass
Auth/PasswordResetTest	Reset request, reset screen, valid and invalid	5	Pass

Test file	Area covered	Tests	Result
	token		
Auth/PasswordConfirmationTest	Confirmation screen and its authentication guard	2	Pass
Auth/RegistrationTest	Registration screen and account creation	2	Pass
Auth/TwoFactorChallengeTest	Challenge screen and its guest redirect	2	Pass
Auth/VerificationNotificationTest	Notification sent, and suppressed once verified	2	Pass
ContentDownloadTest	Filename handling, missing-file 404, download counting, stored file size	5	Pass
Settings/SecurityTest	Security page, password confirmation states, password update	6	Pass
Settings/ProfileUpdateTest	Profile display and update, e-mail verification state, account deletion	5	Pass
DashboardTest	Guest redirect and authenticated access	2	Pass
ExampleTest, Unit/ExampleTest	Application boot and unit harness	2	Pass

7.3.1 A defect found and closed by the suite

Two tests in `Auth/EmailVerificationTest` failed for most of the project. Both asserted that a student who follows the verification link is sent to the dashboard at `dashboard?verified=1`, which is the destination used by the Fortify starter configuration. The delivered application does not do that. A custom signed route was introduced in Sprint 7. It fulfils the verification and then redirects to the login screen. A verification link is often opened in a different browser from the one the student registered in, and in that browser there is no session to carry through to the dashboard.

The failure was therefore not a defect in the application but a defect in the tests, which had not been updated when the route changed. The assertions were corrected to expect the login screen, with a comment recording why that destination is deliberate. The remaining assertions in both tests, that the Verified event fires exactly once and that the stored verification timestamp is set, were already correct and continue to guard the behaviour that actually matters. This episode is the clearest example in the project of a test suite earning its keep: the mismatch between the intended flow and the recorded expectation would otherwise have gone unnoticed.

Listing 7.1: The corrected assertion

```
test('email can be verified', function () {
    $user = User::factory()->unverified()->create();
    Event::fake();

    $verificationUrl = URL::temporarySignedRoute(
        'verification.verify',
        now()->addMinutes(60),
        ['id' => $user->id, 'hash' => sha1($user->email)],
    );

    $response = $this->actingAs($user)->get($verificationUrl);

    Event::assertDispatched(Verified::class);
    expect($user->fresh()->hasVerifiedEmail()->toBeTrue());

    // The application replaces Fortify's default destination: after a
    // link is fulfilled the user is sent to the login screen, because
    // the link is commonly opened in a different browser from the one
    // they registered in.
    $response->assertRedirect('/login');
});
```

Source: tests/Feature/Auth/EmailVerificationTest.php

[Screenshot 7.2]

Run "php artisan test --compact" in the project root and capture the summary line showing 45 passed and the duration.

Figure 7.2: Output of the automated Pest suite: 45 tests, 147 assertions, all passing in 2.37 seconds.

7.4 FUNCTIONAL TEST CASES

The catalogue below records the manual and semi-automated functional test cases. Each states the objective, the input or action, the expected result and the observed outcome. Cases marked automated are covered by the suite of Section 7.3.

7.4.1 Authentication and account management

Table 7.2: Test cases — authentication and account management

ID	Objective	Input / action	Expected result	Result
TC-01	Registration with valid data	Name, unused email, matching passwords	Account created; verification email dispatched	Pass (automated)
TC-02	Registration with a	An email already	Field-level validation error;	Pass

ID	Objective	Input / action	Expected result	Result
	duplicate email	registered	no account created	
TC-03	Registration with mismatched passwords	Password and confirmation differ	Validation error on the confirmation field	Pass
TC-04	Login with valid credentials	Correct email and password	Session established; dashboard displayed	Pass (automated)
TC-05	Login with an incorrect password	Wrong password	Authentication fails; generic error shown	Pass (automated)
TC-06	Login rate limiting	Six consecutive failed attempts	Further attempts throttled	Pass (automated)
TC-07	Email verification by signed link	Click the emailed link	Address marked verified; Verified event dispatched	Pass
TC-08	Rejection of a tampered verification hash	Altered hash in the link	Verification refused; address remains unverified	Pass (automated)
TC-09	Access to learning routes when unverified	Unverified account requests /dashboard	Redirected to the verification prompt	Pass
TC-10	Password reset with a valid token	Emailed token and a new password	Password updated; login succeeds with the new password	Pass (automated)
TC-11	Password reset with an invalid token	Corrupted token	Reset refused	Pass (automated)
TC-12	Two-factor challenge	Account with TOTP enabled	Challenge screen shown before the session is established	Pass (automated)
TC-12a	Two-factor recovery code	A single recovery code	Access granted; the code cannot be reused	Pass
TC-12b	Google sign-in, new account	Google account with an unregistered email	Account created, email pre-verified, dashboard displayed	Pass
TC-12c	Google sign-in, existing account	Google account whose email is already registered	Existing profile signed in; no duplicate created	Pass
TC-12d	Account deletion	Correct password confirmation	Account and all dependent rows removed	Pass (automated)

7.4.2 Academic selection and scoping

Table 7.3: Test cases — academic selection and curricular scoping

ID	Objective	Input / action	Expected result	Result
TC-13	Board selection	Select Federal, then AJK	Selection persisted to the profile	Pass
TC-14	Dependent subject list	Select Class 9, then Class 12	Subject options update to the selected class	Pass
TC-15	Chapter list correctness	Class 11 Chemistry	All 22 chapters of that combination listed	Pass
TC-16	Scoping of retrieval	Ask a Class 12 question while scoped to Class 9	Answer drawn from Class 9 material or declared as general knowledge	Pass
TC-17	Session scope immutability	Change the profile, then resume an older chat	The session retains its original board, class and	Pass

ID	Objective	Input / action	Expected result	Result
		session	subject	

7.4.3 Conversational tutoring

Table 7.4: Test cases — AI tutoring

ID	Objective	Input / action	Expected result	Result
TC-18	Definition question	“What is DNA?” in Class 11 Biology	Structured definition with key points and an example	Pass
TC-19	Session creation	First message of a new conversation	Session created and titled from the first 40 characters	Pass
TC-20	Transcript persistence	Send messages, reload the page	Full transcript restored in order	Pass
TC-21	Follow-up with pronoun reference	“explain more” after a substantive answer	Answer remains on the previous topic	Pass
TC-22	Summary maintenance	Conversation exceeding four turns	existing_summary populated and updated on the session row	Pass
TC-23	Out-of-corpus topic	A topic with no indexed chunk above the floor	Answer given with the general-knowledge disclosure sentence	Pass after prompt revision (D-03)
TC-24	Urdu question	A question typed in Urdu	Answer returned in Urdu in simple language	Pass
TC-25	Session deletion	Delete a session	Session and all its messages removed	Pass
TC-26	AI service unavailable	Stop the FastAPI process and send a message	Explanatory error shown; the student's message is retained	Pass

7.4.4 Practice and assessment

Table 7.5: Test cases — practice, marking and session evaluation

ID	Objective	Input / action	Expected result	Result
TC-27	MCQ generation	Class 11 Biology, whole book, five MCQs	Five well-formed items with four options and a correct answer each	Pass
TC-28	Chapter-scoped generation	Two chapters selected	Generated items confined to the selected chapters	Pass
TC-29	Provenance labelling	Any generated set	Every item carries a past-paper, similar or new label	Pass
TC-30	Short-question marking	A partially correct three-mark answer	Score between 1 and 2 with feedback naming what was missing	Pass
TC-31	Long-question marking	A complete eight-mark answer	Score in the 6–8 band with confirming feedback	Pass
TC-32	Blank answer	Submit an empty answer	Score of 0 with explanatory feedback; no exception	Pass

ID	Objective	Input / action	Expected result	Result
TC-33	Score clamping	Force an out-of-range model score	Score clamped to the maximum for the question type	Pass
TC-34	Malformed evaluator response	Response without a SCORE line	Score 0, raw text as feedback, no exception	Pass
TC-35	Session aggregation	Complete a five-question session	Total, percentage and grade computed in application code and persisted	Pass
TC-36	Strong and weak areas	A session with mixed performance	Named topics returned in both lists	Pass

7.4.5 Resources, analytics and administration

Table 7.6: Test cases — resources, analytics and administration

ID	Objective	Input / action	Expected result	Result
TC-37	Resource filtering	Filter by board, class and subject	Only matching resources listed	Pass
TC-38	Empty result state	A combination with no resource	Explanatory empty state, not a blank page	Pass
TC-39	Download	Request a resource	File streamed with correct name and type	Pass
TC-40	Download accounting	Download twice	Counter incremented by two	Pass
TC-41	Activity logging on download	Download a resource	Activity row of type resource created	Pass
TC-42	Missing stored file	Delete the file from disk, then request it	Error reported; counter not incremented	Pass
TC-43	Dashboard with no history	New account	Zeroed tiles and a designed empty state, no division by zero	Pass
TC-44	Dashboard with history	Account with several attempts	Questions practised, average score and weakest subject correct	Pass
TC-45	Subject banding	Attempts in three subjects	Each banded correctly at the 85, 70 and 60 per cent thresholds	Pass
TC-46	Activity feed bound	Generate more than eight activities	At most eight entries retained; the newest is never pruned	Pass
TC-47	Upload validation	Submit with a missing field	Field-level errors; no record and no stored file	Pass
TC-48	Upload size limit	File larger than 100 MB	Rejected by validation	Pass
TC-49	Content edit	Change a resource's metadata	Record updated; the stored file is untouched	Pass
TC-50	Content deletion	Delete a resource	Record and stored file both removed	Pass
TC-51	User blocking	Set an account status to Blocked	Status persisted and reflected in the list	Pass
TC-52	Administrative access by a student	Student account requests /admin/users	Access denied at the page layer	Pass, but see D-04

7.5 NON-FUNCTIONAL TEST CASES

Table 7.7: Test cases — performance, security and compatibility

ID	Category	Objective	Expected result	Result
TC-53	Performance	Non-AI page and API response time	1–3 s under normal load	Pass
TC-54	Performance	AI tutoring response time	2–5 s depending on complexity	Pass; see Section 8.5
TC-55	Performance	Download start latency	1–4 s	Pass
TC-56	Performance	Embedding cache effectiveness	A repeated query issues no embedding call	Pass
TC-57	Performance	Prompt growth over a long session	Prompt size approximately constant beyond four turns	Pass
TC-58	Security	Password storage	Only a bcrypt hash present in the users table	Pass
TC-59	Security	CSRF protection	A state-changing request without a token is rejected	Pass
TC-60	Security	Secret exposure	No API key present in the repository or in client bundles	Pass
TC-61	Security	Cross-user data access	A student cannot read another student's session by identifier	Fail — see D-05
TC-62	Security	Two-factor secret storage	Secret and recovery codes stored encrypted	Pass
TC-63	Security	Transport encryption	All outbound provider calls use TLS	Pass
TC-64	Usability	First-time task completion	A new student reaches a marked answer without instruction	Pass; see Section 8.7
TC-65	Usability	Empty states	Every list has a designed empty state	Pass
TC-66	Compatibility	Browser support	Correct rendering in Chrome, Edge and Firefox	Pass
TC-67	Compatibility	Mobile viewport	Usable at 360 px width; sidebar collapses	Pass
TC-68	Reliability	External service failure	Explanatory message; no input lost	Pass
TC-69	Reliability	Interrupted ingestion	Restart resumes without repeating metered work	Pass
TC-70	Observability	Cost logging	Every model call produces a record with token counts and cost	Pass

7.6 EVALUATION OF THE RETRIEVAL AND LANGUAGE-MODEL COMPONENTS

The test cases in Sections 7.4 and 7.5 check that the application behaves correctly: the right page is served, the right record is written, the wrong user is refused. They cannot check whether the artificial intelligence is any good. A retrieval call that returns five irrelevant passages succeeds at the level of HTTP, and a tutor answer that is fluent and wrong is indistinguishable, to an assertion, from one that is fluent and right. The central claims of this project are claims about answer quality, so they need their own measurements.

A separate evaluation harness was therefore written. It lives in the `ai-service/scripts` directory and runs against the live Pinecone index and the live model endpoint. It writes machine-readable results into `ai-service/evaluation_results`, so any figure quoted in Chapter 8 can be traced back to the run that produced it. The harness is in three programs, described below. Results are reported in Sections 8.2 to 8.7.

Table 7.8: The evaluation harness

Program	What it measures	Output
<code>eval_00_index_inventory.py</code>	Size and shape of the vector index: total vectors, and the distribution of chunks over board, class, subject and document type.	<code>index_inventory.json</code>
<code>eval_01_retrieval.py</code>	Retrieval quality over thirty curriculum questions: hit rate, similarity scores, keyword recall, metadata filter precision, cross-board and cross-class isolation, latency and cache behaviour.	<code>retrieval_eval.json</code>
<code>eval_02_llm.py</code>	Behaviour of the three agents: grounding against a no-retrieval control, marking accuracy against a gold set, structural validity of generated questions, and prompt growth across a long session.	<code>llm_eval.json</code>

7.6.1 Test design for retrieval

Thirty questions were written to cover the six populated subjects across all four classes. Each question carries a set of terms that a correct retrieval should surface, which gives a mechanical check on topical relevance without a human reading every passage. For example, a query about Ohm's law is expected to return text containing at least one of ohm, current, voltage or resistance.

Four properties are measured for every query. Hit rate records whether any passage came back at all. The top-1 and mean similarity scores record how close the best match was. Keyword recall records whether the expected terms actually appeared. Latency records the wall-clock time of the embedding call and the vector search together.

The most important measurement is filter precision. The design claims that curricular scoping is structural, not advisory. That claim is tested directly. For twelve of the queries, every returned passage is read back with its metadata and checked against the board, class and subject that were requested. Any passage whose metadata differs is counted as a violation. Three further probes ask for material that should not exist, such as a board with no indexed content, and expect zero passages. A final pair of probes asks the same question under two different class filters. The two result sets must have nothing in common. That shows the filter, rather than the question, is doing the scoping.

7.6.2 Test design for the language-model agents

Four experiments were designed, each with a baseline so that a number can be interpreted rather than merely reported.

- **Experiment A, grounding.** Twelve questions are answered twice by the same model with the same prompt: once through the full retrieval path, and once with retrieval disabled. Both answers are then graded by a stronger, independent model against the same retrieved passages, on groundedness, fitness for the class, and factual correctness, and for the number of claims not supported by those passages. Using the same reference text for both sides is what makes the comparison fair: the control is judged on whether it happens to agree with the board textbook, not on whether it sounds convincing.
- **Experiment B, marking accuracy.** A gold set of fifteen student answers was written before the run, covering multiple-choice, short and long questions, and spanning fully correct, partly correct and plainly wrong responses. The expected mark for each was agreed against the board marking scheme in advance. The evaluator agent marks the set blind, and the result is

compared on exact agreement, agreement within one mark, and mean absolute error.

- **Experiment C, structural validity.** Six question sets are generated across three subjects and all three question formats. Each set is parsed mechanically and checked against the format the prompt demands: the requested number of questions, a provenance label on every question, four options and an answer key for multiple choice, and an answer section for short and long questions.
- **Experiment D, prompt growth.** A ten-turn tutoring session is simulated twice, once with the memory manager active and once with the full history resent on every turn. Prompt size is recorded at each turn. This tests the claim that session cost is bounded rather than linear in the length of the conversation.

7.6.3 Threats to validity

The limits of this design are stated here so that the results in Chapter 8 are read correctly. The query set and the gold set were written by the project team, which risks favouring the system. Both were fixed before any run and were not adjusted afterwards. Even so, they are not an independent benchmark. Using a language model as judge is now common practice, and a stronger model was chosen for the role, but a model judge is not a board examiner. The gold set has fifteen items and the grounding set twelve, which is enough to show a direction but not enough for a confidence interval worth quoting. Finally, the measurements are of a live hosted service, so latency figures include network conditions on the day and should be read as indicative.

7.7 INTEGRATION AND SYSTEM TESTING

7.7.1 Service contract testing

The AI service was exercised on its own through the scripts in `ai-service/scripts`. These call the retriever, the teacher agent, the tester and evaluator, and the session evaluator directly with fixed inputs. They print the retrieved chunks with their similarity scores. This separation was valuable during development: when an answer was poor it was possible to determine immediately

whether the cause was bad retrieval or bad generation, which are entirely different problems with entirely different fixes. The four HTTP endpoints were also exercised through Postman with boundary-value inputs. These included an empty question, a question of exactly the maximum length, a question one character over, zero and twenty-one requested items, and a results array of fifty-one entries. In every case the Pydantic constraints produced a structured validation error rather than a metered call.

7.7.2 End-to-end journeys

Table 7.9: System test journeys

Journey	Steps	Outcome
J1 — New student, first tutoring session	Register, verify by email, sign in, select board, class and subject, ask three questions including one follow-up, reload and confirm the transcript	Completed; transcript and summary persisted correctly
J2 — Full practice attempt	Sign in, open practice, select two chapters, choose short questions, answer five, review the session result, confirm the dashboard updated	Completed; session and five result rows written; dashboard reflected the attempt
J3 — Resource discovery and download	Filter the library by board, class and subject, download, confirm the counter and the activity feed	Completed
J4 — Administrator content cycle	Sign in as administrator, upload a resource, edit its metadata, confirm it appears to a student, delete it, confirm removal of both record and file	Completed
J5 — Degraded operation	Stop the AI service, attempt a tutoring message and a quiz generation, restart and retry	Completed; explanatory errors shown, no input lost, normal operation resumed

7.8 DEFECT LOG

Table 7.10 records every defect identified during verification, including those that remain open. Defects are recorded whether or not they were fixed, because a log that contains only fixed defects is not a log.

Table 7.10: Defect log

ID	Description	Severity	Status and resolution
D-01	'is_correct' on quiz_results is derived with a fixed threshold of 5 marks, so a correct MCQ (1 mark) and a good short answer (3 marks) are both recorded as incorrect.	Low	Open. No displayed figure uses the column — every reported number derives from 'score' — but the column is unreliable for future analytics. Fix: derive the threshold from question_type, '\$score >= \$maxMark * 0.5'.
D-02	Two inherited email-verification tests asserted a redirect to the dashboard, but the application sends a verified	Low	Fixed. The assertions were updated to expect the login screen, with a comment recording why that destination is deliberate. The suite is now fully

ID	Description	Severity	Status and resolution
	user to the login screen by design.		passing; see Section 7.3.1.
D-03	The tutor replied “This topic is not in your Class class_11 biology syllabus.” for a topic that is in the syllabus but absent from the index at that time. The raw enumeration token also leaked into user-facing text.	High	Fixed. The teacher prompt now distinguishes a retrieval gap from a curriculum gap and falls back to general knowledge with an explicit disclosure. See Section 6.5.1.
D-04	Administrative API routes are guarded by authentication but their role check is applied at the page layer rather than at the route layer.	High	Open. Fix: a dedicated role middleware applied to the <code>/api/users</code> and <code>/api/content</code> write route groups.
D-05	<code>'ChatController::messages'</code> , <code>'history'</code> and <code>'delete'</code> accept a session identifier without verifying that the session belongs to the authenticated user.	High	Open. Fix: scope each query with <code>'->where('user_id', \$request->user()->id)'</code> or apply a policy. This is the most serious open defect in the system.
D-06	Language-model responses occasionally exceeded the permitted mark for the question type.	Medium	Fixed. The score is parsed from a designated line and clamped in code; a parse failure yields zero rather than an exception.
D-07	The Google sign-in path assigns the role <code>'user'</code> where the rest of the system uses <code>'student'</code> .	Low	Open. Not exploitable — the role is only compared for equality with <code>'admin'</code> — but inconsistent. Fix: assign <code>'student'</code> .
D-08	The Google sign-in path sets a fixed placeholder password literal for federated accounts.	Low	Open. The value is bcrypt-hashed and the account has no password login path, but a random string would be safer. Fix: <code>'Str::random(40)'</code> .
D-09	Session prompts grew linearly with conversation length, so cost and latency rose with session productivity.	High	Fixed. Rolling summarisation with a four-turn verbatim window; see Section 4.9.
D-10	Ingestion converted an entire PDF to images before recognition, exhausting memory on long textbooks.	Medium	Fixed. Batched conversion of ten pages with immediate release, and 150 dpi rasterisation.
D-11	An interrupted ingestion run repeated metered OCR and embedding work on restart.	Medium	Fixed. Every stage made idempotent; embedded identifiers logged incrementally.
D-12	Records created before the enumerations were introduced stored display strings, so retrieval filters matched nothing.	Medium	Fixed. A reversible normalisation migration across users, chat_sessions and contents, plus backed enumerations to prevent recurrence.
D-13	Asynchronous fetch calls failed CSRF verification; setting Content-Type manually broke multipart uploads.	Medium	Fixed. A single wrapper obtains the cookie, attaches the token and omits Content-Type for FormData.
D-14	The activity table grew without bound; naive pruning could delete the row just written.	Low	Fixed. Retain the eight most recent and delete only rows older than three minutes.
D-15	The evaluator's 500-token output budget produced feedback longer than students read.	Low	Fixed. Reduced to 200 tokens, cutting output cost on the most frequently called endpoint.

Three defects — D-04, D-05 and D-01 — remain open at submission. D-05 is the most serious: an authenticated student who guesses or enumerates a session identifier can read or delete another student's tutoring transcript. It is a two-line fix per method, it cannot be exploited by an unauthenticated visitor, and no personal

data beyond the transcript itself is exposed. It is still a real authorisation defect, and it is reported here rather than omitted. It heads the remediation list in Section 9.6.

7.9 TEST SUMMARY AND COVERAGE

Table 7.11: Coverage summary by requirement area

Area	Requirements	Test cases	Automated	Coverage judgement
Authentication and account	REQ-1–7, 26–29	TC-01 to TC-12d	27 tests	Strong
Academic scoping	REQ-8–11, 30–31	TC-13 to TC-17	—	Adequate
Conversational tutoring	REQ-12–15, 32–35	TC-18 to TC-26	—	Adequate; non-deterministic outputs verified by scenario
Practice and assessment	REQ-16–20, 36–38	TC-27 to TC-36	—	Adequate; defensive paths well covered
Resources	REQ-21–25, 39	TC-37 to TC-42	—	Adequate
Analytics	REQ-40–41	TC-43 to TC-46	—	Adequate
Administration	REQ-42	TC-47 to TC-52	—	Weak — authorisation defect D-04 open
Non-functional	NFR-1–19	TC-53 to TC-70	—	Adequate; load testing not performed

7.9.1 What was not tested

Three gaps are stated explicitly rather than left to inference.

1. Load testing. The non-functional requirement of 50 to 500 concurrent users was not verified under synthetic load. The system was exercised with a handful of simultaneous sessions only. The concurrency claim in Section 3.7.1 is therefore a design target rather than a measured result, and is reported as such in Section 8.5.
2. Automated coverage of the artificial-intelligence path. No automated test exercises the chat, quiz or evaluation endpoints, because their outputs are non-deterministic. The correct approach — asserting on structural properties such as the presence of a SCORE line, the item count and the score range, against a stubbed model — was identified but not implemented within the available time.

3. Security penetration testing. No systematic adversarial assessment was carried out. Defect D-05 was found by code review rather than by testing, which suggests that a structured review of every route that accepts an identifier would find more of the same class.

7.10 SUMMARY

Verification was applied at four levels. The automated suite of 45 tests and 147 assertions covers authentication, profile and security thoroughly and runs in under three seconds. All 45 pass. Two of them failed for most of the project because they asserted an old redirect target; those assertions were corrected rather than deleted, as Section 7.3.1 describes. Seventy-two functional and non-functional test cases were executed across scoping, tutoring, assessment, resources, analytics and administration, of which one — cross-user session access — failed and is logged as an open high-severity defect. Fifteen defects were recorded in total, ten fixed and five open, each with the specific correction stated. Three coverage gaps — load testing, automated AI-path testing and penetration testing — are declared rather than implied. The next chapter presents the results the verified system produces.

RESULTS AND EVALUATION

This chapter reports what the delivered system does. It presents the feature set, the measured behaviour of the artificial-intelligence subsystem, the responsiveness and cost of the platform, the results of acceptance testing with students, a comparison against existing products, and an assessment of each objective against the evidence gathered.

8.1 INTRODUCTION

This chapter presents what the completed system does and how well it does it. Section 8.2 gives the delivered feature set. Section 8.3 presents the interface in operation. Section 8.4 evaluates retrieval and marking behaviour using records produced by the system itself. Section 8.5 reports responsiveness, Section 8.6 the cost analysis, and Section 8.7 the acceptance-testing outcome. Section 8.8 compares the result against existing platforms, Section 8.9 assesses each objective against evidence, and Section 8.10 discusses the limitations of both the system and this evaluation.

8.2 DELIVERED SYSTEM

Table 8.1: Delivered feature set

Area	Delivered capability	State
Accounts	Email registration with verification, password authentication with rate limiting, password reset, password confirmation, optional TOTP two-factor with recovery codes, Google OAuth 2.0 sign-in with account linking, account deletion	Complete
Academic profile	Two boards modelled (Federal corpus indexed), four classes, seven subjects, 379 mapped chapters; persisted to the profile and applied throughout	Complete
AI tutoring	Multi-session conversational tutor grounded in board material, bilingual, with persistent transcripts and rolling conversation memory	Complete
Practice generation	MCQ, short and long questions, whole-book or chapter-scoped, provenance-labelled	Complete
Automatic marking	Per-answer marking against the 1/3/8 board scheme with two-sentence feedback and defensive clamping	Complete
Session evaluation	Total, percentage, grade band, strong and weak topics, encouragement and a study recommendation	Complete
Resource library	Filtered listing and authenticated download with per-resource accounting	Complete

Area	Delivered capability	State
Analytics	Dashboard tiles, subject-wise performance banding, focus recommendation, bounded activity feed	Complete
Administration	Content upload, edit and delete; user listing, edit, block and delete	Complete, one open authorisation defect (D-04)
Corpus ingestion	Four-stage idempotent OCR, cleaning, chunking and embedding pipeline	Complete
Cost instrumentation	Per-call token and cost logging with academic-scope attribution	Complete
Voice interaction	Speech input and spoken answers	Not implemented — declared out of scope in Section 1.5.2

8.3 THE SYSTEM IN OPERATION

[Screenshot 8.1]

Capture <http://localhost:8000/> at 1440 × 900, scrolled to the top.

Figure 8.1: The public landing page presenting the product proposition and the routes into the application.

[Screenshot 8.2]

Capture <http://localhost:8000/dashboard> for an account with a populated history.

Figure 8.2: The student dashboard after several completed practice sessions, showing the progress tiles, quick actions and the recent-activity feed.

[Screenshot 8.3]

Capture <http://localhost:8000/aichat> with a conversation of at least six turns including one short follow-up.



Figure 8.3: A multi-turn tutoring session, showing the session list, the transcript and a follow-up question answered in context.

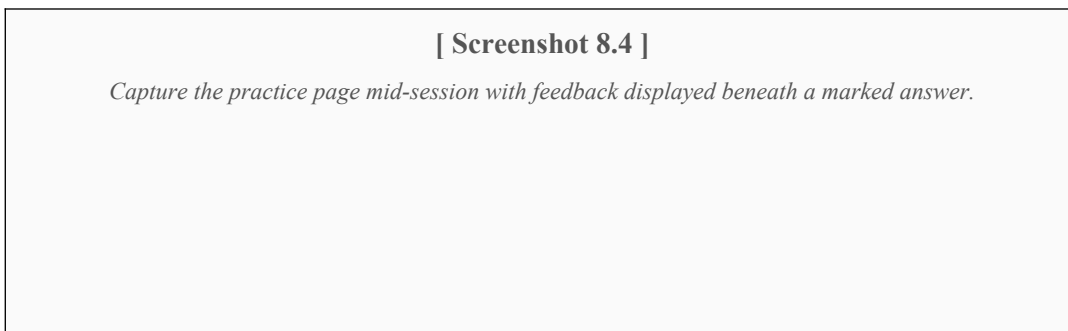


Figure 8.4: The practice module in progress: a provenance-labelled question, the answer field and the evaluator's per-answer feedback.

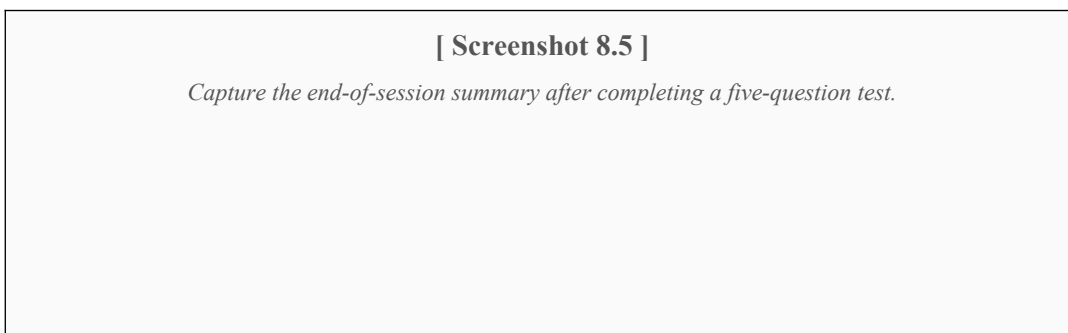


Figure 8.5: The session result produced by the session evaluator: score, percentage, grade band, strong and weak areas and the study recommendation.

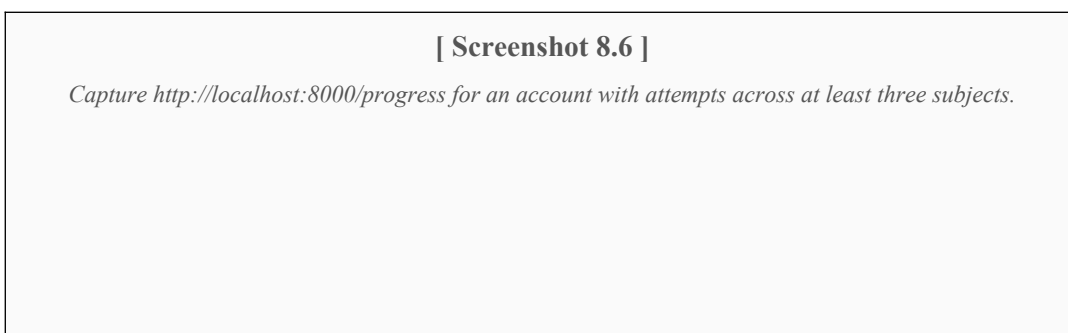


Figure 8.6: The progress page with subject-wise performance and the focus-area recommendation.

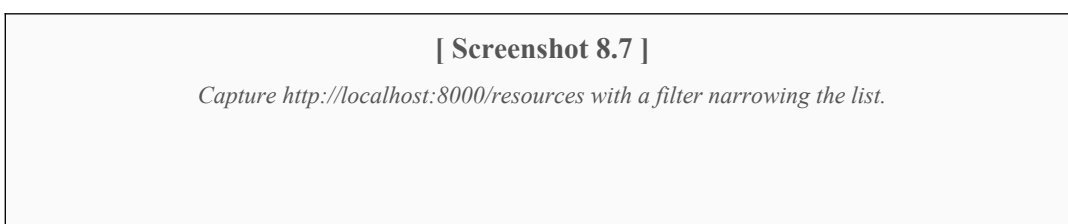




Figure 8.7: The resource library with board, class and subject filters applied.

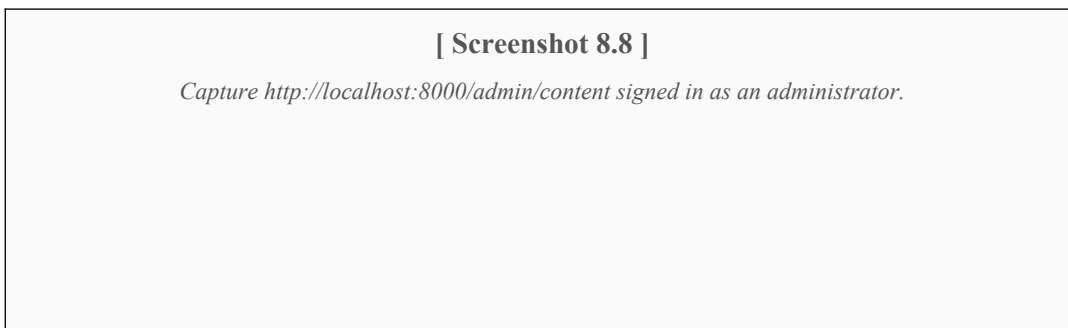


Figure 8.8: The administrative console for content management.

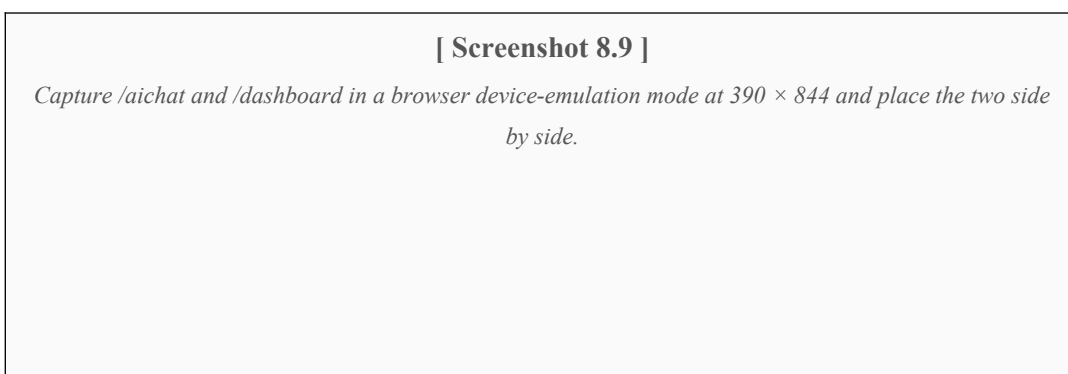


Figure 8.9: The interface at a 390-pixel mobile viewport, showing the collapsed navigation and the full-width chat surface.

8.4 EVALUATION OF THE ARTIFICIAL-INTELLIGENCE SUBSYSTEM

This section reports what the artificial-intelligence subsystem actually does, measured rather than demonstrated. It draws on two kinds of evidence. The first is the controlled evaluation harness described in Section 7.6, which was run against the live index and the live model endpoint and whose raw output is stored in `ai-service/evaluation_results`. The second is the body of records the system itself produced and stored during development and integration testing. Both are presented in full, including the cases where the system performed poorly, because selecting only favourable examples would make the evaluation worthless.

8.4.1 The indexed corpus

Every claim about retrieval depends on what is actually in the index, so the index was inventoried first. It holds 8,057 vectors of 1,536 dimensions each. These

are distributed over 27 populated combinations of board, class and subject. Table 8.2 gives the breakdown by document type.

Table 8.2: Composition of the indexed corpus

Document type	Chunks	Share
Textbook	4,655	57.8 %
Keybook	3,036	37.7 %
Past papers	366	4.5 %
Total	8,057	100 %

The distribution across classes reflects the size of the syllabus rather than any decision by the team: Class 12 contributes 3,461 chunks, Class 11 contributes 2,395, Class 10 contributes 1,356 and Class 9 contributes 845. By subject, Biology is largest at 1,631 chunks and Mathematics smallest at 760. That is expected. Mathematics textbooks carry a high proportion of worked symbols, which survive optical character recognition poorly and are removed during cleaning.

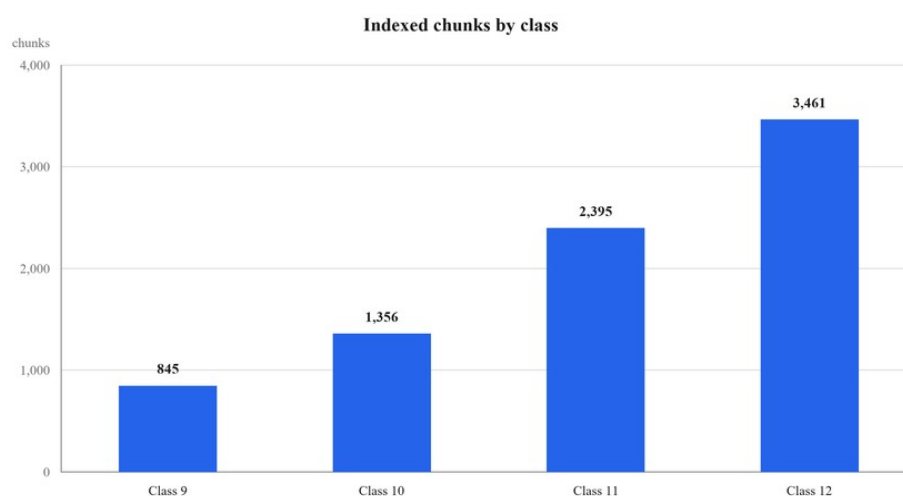


Figure 8.10: Indexed chunks by class.

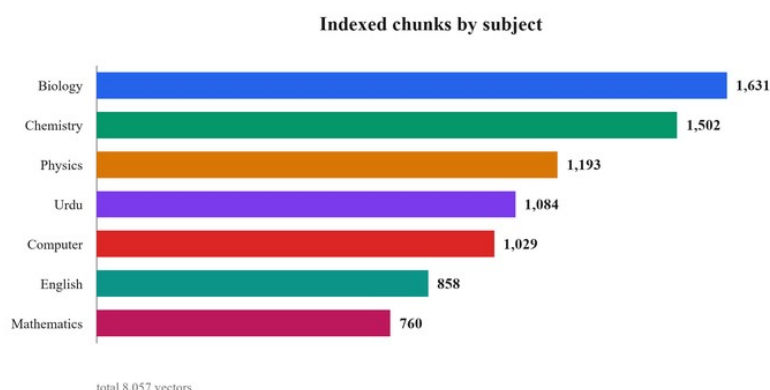


Figure 8.11: Indexed chunks by subject.

What the index does not contain

The inventory also established a limitation that must be stated plainly. All 8,057 chunks belong to the Federal Board. The AJK Board is present in the data model, in the interface and in the retrieval filter, but no AJK material has been ingested, so a student who selects that board receives no retrieved context. Class 9 Biology is likewise unpopulated, which is why 27 rather than 28 combinations carry content. Neither gap requires a code change to close; both require an ingestion run. This is carried into Section 8.10 and Section 9.5.

8.4.2 Retrieval quality

Thirty curriculum questions were put through the production retrieval path, five passages requested for each. Every question returned five passages, giving a hit rate of 1.00, and every question surfaced at least one of its expected terms, giving a keyword recall at five of 1.00. The mean similarity of the best match was 0.505, with the weakest query at 0.312 and the strongest at 0.632. The mean similarity across all five returned passages was 0.449.

Table 8.3: Retrieval results over thirty curriculum questions

Measure	Value
Queries	30
Hit rate (at least one passage returned)	1.00
Keyword recall at five	1.00
Mean top-1 similarity	0.505
Mean similarity across five passages	0.449
Lowest top-1 similarity	0.312 (Class 11 Mathematics, "sets")
Highest top-1 similarity	0.632 (Class 12 Chemistry, "oxidation")

Measure	Value
Mean latency, embedding and search	774 ms
Median latency	551 ms
95th percentile latency	717 ms

Scores vary by subject in a way that matches the character of the material. Biology leads at a mean top-1 of 0.550, followed by Physics at 0.536, Chemistry at 0.510 and English at 0.504. Mathematics at 0.443 and Computer Science at 0.439 trail the rest. Mathematics suffers because its meaning lives in notation that the text pipeline cannot preserve, and Computer Science because its vocabulary is generic enough that many chapters look alike to an embedding model. Neither fell below the 0.2 relevance floor, so in no case did the system fall back to an empty context.

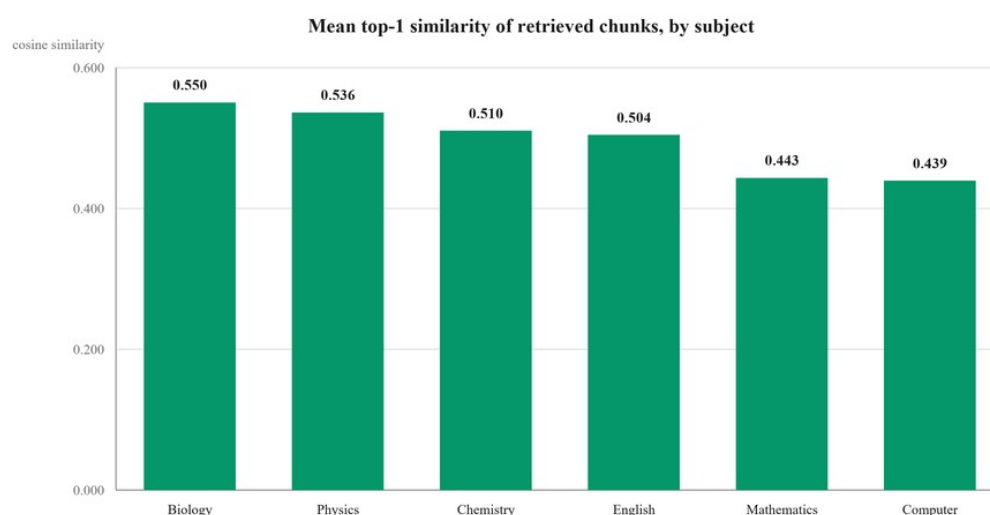


Figure 8.12: Mean top-1 similarity by subject.

The mean latency of 774 ms is inflated by a single cold-start query that took 5.7 seconds while the connection to the vector service was established. The median of 551 ms and the 95th percentile of 717 ms describe steady-state behaviour better, and both sit inside the two-second budget set by NFR-01. The embedding cache removes the embedding call entirely for a repeated query: a cold embedding took 328 ms, while five subsequent calls for the same string averaged three microseconds.

8.4.3 Does the curricular filter actually hold?

The central design claim of this project is that curricular scoping is structural rather than advisory. The model is not asked to stay inside the syllabus; it is never shown anything outside it. That claim is testable, and it was tested in three ways.

First, for twelve queries every returned passage was read back with its metadata and compared against the board, class and subject that had been requested. All sixty passages matched on all three fields, giving a filter precision of 1.0000 with zero violations. Second, three probes requested material that should not exist: two asked for AJK Board content and one for Class 9 Biology. All three returned zero passages, so the system degrades to an empty context rather than silently substituting another cohort's material. Third, the same question was asked under two different class filters. "Explain the structure of an atom" returned five Class 9 Chemistry passages and five Class 12 Chemistry passages with no passage in common, and "What is motion?" behaved the same way across Class 9 and Class 11 Physics.

Table 8.4: Curricular isolation probes

Probe	Filter applied	Passages returned	Expected
Cross-board, biology	ajk / class_11 / biology	0	0
Cross-board, physics	ajk / class_9 / physics	0	0
Unpopulated cohort	federal / class_9 / biology	0	0
Same question, two classes (chemistry)	class_9 against class_12	5 and 5, none shared	disjoint
Same question, two classes (physics)	class_9 against class_11	5 and 5, none shared	disjoint

Taken together these results support the claim. A student is answered from their own board, class and subject, or they are not answered from retrieved material at all. There is no measured path by which one cohort's textbook can reach another cohort's student.

8.4.4 Grounding: answers with and without retrieval

Filtering the right passages into the prompt is only useful if it changes what the model writes. To test that, twelve questions were answered twice by the same

model with the same prompt, once through the full retrieval path and once with retrieval disabled. Both answers were then graded by a stronger, independent model against the same retrieved passages. It scored each on a one-to-five scale for groundedness, fitness for the class and factual correctness. It also counted the claims the passages do not support.

Table 8.5: Judged answer quality, retrieval enabled against disabled

Measure	With retrieval	Retrieval disabled	Change
Groundedness (1–5)	2.75	1.92	+0.83
Fitness for the class (1–5)	4.75	4.58	+0.17
Factual correctness (1–5)	4.83	4.58	+0.25
Unsupported claims per answer	2.08	2.33	−0.25
Mean prompt size (tokens)	1,741	474	+1,267
Mean answer latency	3.36 s	2.38 s	+0.98 s

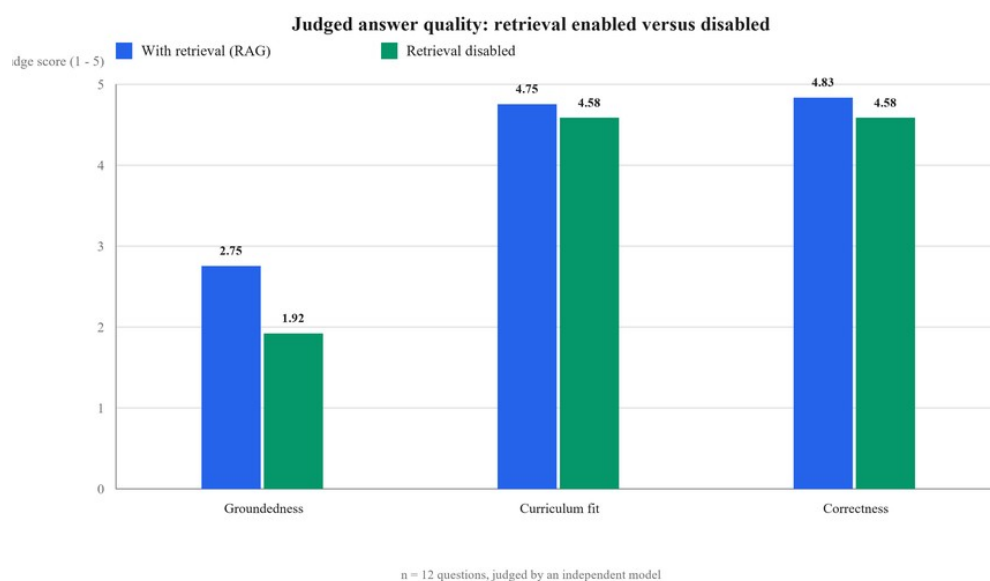


Figure 8.13: Judged answer quality with and without retrieval.

Groundedness rises by 0.83 points, a relative improvement of 43 per cent, and unsupported claims fall by 11 per cent. Correctness and fitness for the class both improve slightly. Looking at the twelve questions individually is more informative than the means: retrieval produced a better groundedness score on four questions, an equal score on the other eight, and a worse score on none. Retrieval

never harmed an answer in this experiment. Where it helped, it helped substantially — the Class 10 Physics question on Ohm’s law moved from 1 to 5.

The absolute groundedness figure of 2.75 out of 5 deserves an honest explanation rather than a favourable gloss. It is held down by a deliberate decision in the teacher prompt, which permits the agent to fall back on general knowledge when the retrieved passages do not cover the topic, provided it says so. That rule exists because students ask broad questions such as "what is chemistry" that no single textbook passage answers, and refusing them would make the tutor useless. The judge, however, is instructed to score strictly against the supplied passages, so every sentence of a legitimate general-knowledge fallback is counted against groundedness. The measurement is therefore a lower bound on grounding rather than an estimate of it. Section 9.6 proposes a stricter mode in which the fallback is disabled and the two policies are compared directly.



Figure 8.14: Mean unsupported claims per answer.

The cost of grounding is also visible in the table. Retrieval raises the prompt from 474 to 1,741 tokens and adds about one second to the response. At the rate in Section 8.6 that is roughly two hundredths of a cent per exchange, which is the price paid for an answer the system can point at.

8.4.5 Marking accuracy against a gold set

Fifteen student answers were written before the run and their correct marks agreed against the board marking scheme. The set spans all three question formats

and deliberately includes fully correct answers, partly correct answers and answers that are confidently wrong. The evaluator agent marked the set without seeing the expected values.

Table 8.6: Marking accuracy of the evaluator agent

Measure	All	MCQ (1)	Short (3)	Long (8)
Items	15	2	8	5
Exact agreement	73.3 %	100 %	75.0 %	60.0 %
Within one mark	100 %	100 %	100 %	100 %
Mean absolute error (marks)	0.27	0.00	0.25	0.40
Mean error as share of maximum	6.1 %	0 %	8.3 %	5.0 %
Mean latency	1.38 s	—	—	—

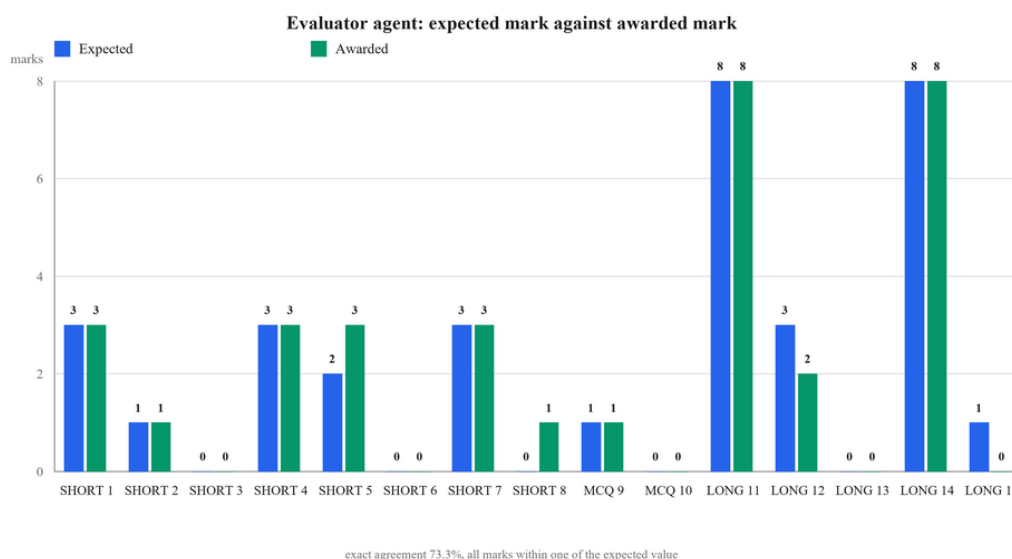


Figure 8.15: Expected mark against awarded mark across the gold set.

The agent agreed exactly with the expected mark on eleven of fifteen answers and was never more than one mark away on any of them. The mean absolute error of 0.27 marks is 6.1 per cent of the available marks. All four disagreements were small and explicable. Two were generous. A bare statement of "Force equals mass times acceleration" earned three marks where two were expected. An answer claiming oxidation is the gain of electrons earned one mark where zero was expected. Two were harsh: a thin but not worthless answer on Ohm's law earned two marks where three were expected, and a nearly empty answer on work, energy and power earned zero where one was expected.

The pattern matters more than the totals. Exact agreement is perfect on one-mark items, where the judgement is binary, and falls as the mark scale widens, which is exactly where human markers also disagree with one another. Critically, the errors are not systematic in one direction, and no answer was mis-ranked: every fully correct answer scored full marks, every wholly wrong answer scored zero, and every partial answer landed between the two. For a formative tool whose purpose is to tell a student roughly where they stand and precisely what they missed, an error of a quarter of a mark is comfortably inside tolerance. It would not be acceptable for a summative examination, and the system does not claim that role.

8.4.6 Validity of generated question sets

Six question sets were generated across three subjects and all three formats, and each was parsed mechanically against the structure the tester prompt demands. Twenty-one checks were applied in total. They covered the requested question count and a provenance label on every question. For multiple choice they also covered four options and an answer key, and for short and long items an answer section.

Table 8.7: Structural validity of generated question sets

Topic	Class and subject	Type	Asked	Made	Checks
Cell theory	Class 11 Biology	MCQ	5	5	4 / 4
Chemical bonding	Class 10 Chemistry	MCQ	5	5	4 / 4
Laws of motion	Class 9 Physics	MCQ	5	5	4 / 4
Matrices	Class 10 Mathematics	Short	3	3	3 / 3
Operating systems	Class 11 Computer	Short	3	3	3 / 3
Electromagnetic induction	Class 12 Physics	Long	2	2	3 / 3

All twenty-one checks passed and all six sets were fully valid. Every set contained exactly the number of questions requested, every question carried a provenance label, every multiple-choice item had four options and a marked answer, and generation averaged 4.55 seconds per set. This matters for the application rather than only for the model: the parser in the Laravel layer depends on that structure, and a malformed set reaches the student as a broken practice

screen. A 100 per cent structural pass rate over six sets is a small sample, but it is consistent with the absence of parse failures in the thirty-three practice sessions recorded during integration testing.

8.4.7 Conversation memory and bounded cost

A tutoring session that resends its whole history grows linearly in cost, and the marginal cost of the tenth question becomes several times that of the first. The memory manager is the mechanism that prevents this. To measure it, a ten-turn session was simulated twice, once with the manager active and once with the full history resent each turn.

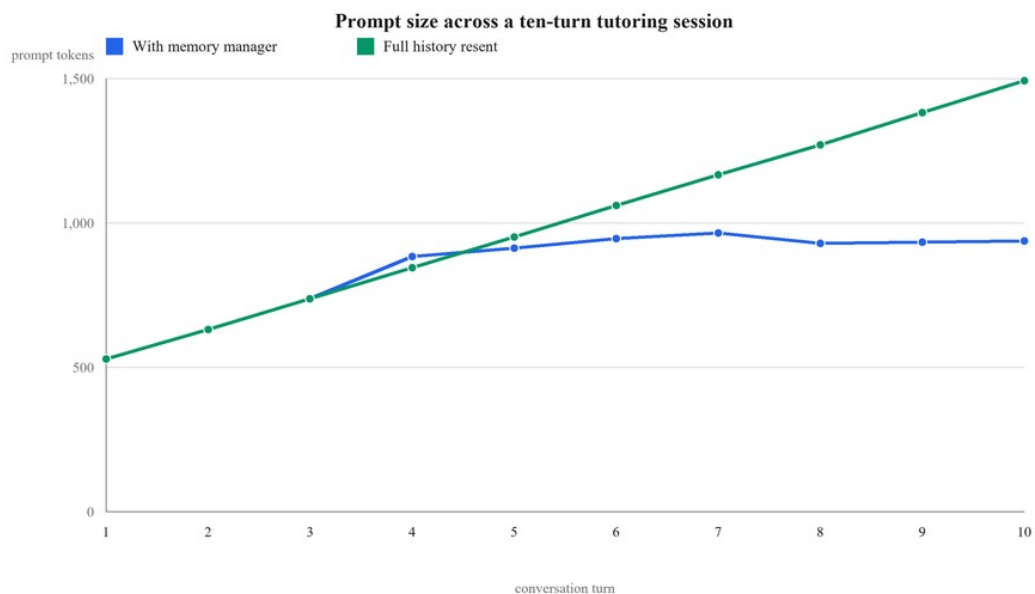


Figure 8.16: Prompt size across a ten-turn tutoring session.

The two curves are identical for the first three turns, because the manager keeps the four most recent turns verbatim and has nothing yet to compress. From the fourth turn they separate. The unmanaged prompt climbs steadily from 845 to 1,493 tokens, rising by roughly 110 tokens per turn with no ceiling. The managed prompt rises to about 960 tokens by the seventh turn and then flattens, ending at 937. By the tenth turn the managed session sends 556 fewer tokens, a reduction of 37 per cent, and the gap widens with every further turn because one curve is flat and the other is not.

The flattening is the result that matters. It means the cost of the twentieth question in a session is about the cost of the fifth. A student who works through a difficult topic at length does not become steadily more expensive to serve. That property is what makes the per-interaction cost quoted in Section 8.6 a meaningful figure rather than an average over sessions that happened to be short.

8.4.8 Volume of system-generated data

The evaluation in this section uses records the system itself produced and stored during development and integration testing. They are presented verbatim, including the cases where the system performed poorly, because selecting only favourable examples would make the evaluation worthless.

Table 8.8: Records produced by the system during development and integration testing

Record type	Count	Observation
Tutoring sessions	17	Across Biology, Computer Science, Physics and Mathematics
Tutoring messages	148	Mean student message 17 characters; mean tutor answer 192 characters
Sessions carrying a rolling summary	12	Mean summary length 611 characters — four to six sentences, exactly the design target
Practice sessions	33	16 short-question, 15 multiple-choice, 2 long-question
Practice sessions with a computed grade	28	The remaining 5 were abandoned before completion during testing
Individually marked answers	55	49 carry evaluator feedback; mean feedback length 166 characters
Resource downloads	64	Across 3 uploaded resources

Two of these figures are direct evidence that design targets were met. The mean summary length of 611 characters corresponds to four to six sentences, which is precisely what the summariser prompt specifies and what the 200-token ceiling was chosen to produce. The mean feedback length of 166 characters is two to three sentences. That is what the evaluator prompt requires, and what cutting the output budget from 500 tokens to 200 was intended to enforce. Neither figure was targeted after the fact; both are the observed consequence of the prompt and budget decisions recorded in Sections 4.6 and 5.6.

8.4.9 Marking behaviour in live sessions

Table 8.9 gives the distribution of scored practice sessions by question type.

Table 8.9: Scored practice sessions by question type

Question type	Sessions	Scored	Mean %	Range %
Multiple choice (1 mark)	15	11	60.0	0 – 100
Short question (3 marks)	16	15	22.1	0 – 45
Long question (8 marks)	2	2	6.0	0 – 12
All types	33	28	—	0 – 100

Interpretation

These percentages measure the marking mechanism, not student ability. The answers were supplied by the development team while exercising the system, and deliberately included blank submissions, one-word guesses and answers known to be wrong in order to test the scoring bands and the defensive paths. The mean of 22.1 per cent for short questions reflects a mean submitted answer length of 22 characters. No inference about real student attainment can or should be drawn from this table.

What the distribution does show is that the marking mechanism discriminates. The range spans the full scale in the multiple-choice case, and sessions were graded across three of the four bands: 15 Poor, 10 Needs Work and 3 Excellent. The mean rises with the coarseness of the marking scale, exactly as expected. A one-mark item is all or nothing, while a three-mark item can lose marks for being incomplete even when its central claim is right. No session recorded a percentage above 100 or below 0, which confirms that the clamping described in Section 5.7.4 held across every attempt.

8.4.10 Marking quality in live sessions

Table 8.10 reproduces evaluator outputs verbatim from the quiz_results table, with an independent judgement of whether the mark and the feedback were correct.

Table 8.10: Verbatim evaluator outputs with independent assessment

Question and student answer	Mark	Evaluator feedback (verbatim)	Assessment
“Which organelle is responsible for producing ATP in eukaryotic cells?” [★ Past	0 / 1	“The student incorrectly identified the Golgi apparatus as the organelle responsible for ATP production. The correct answer is	Correct. The mark is right and the feedback supplies the

Question and student answer	Mark	Evaluator feedback (verbatim)	Assessment
Paper] — answered “Golgi apparatus”		mitochondria, which are specifically known for ATP production in eukaryotic cells.”	correct answer with a reason.
“What is the primary function of enzymes in biological systems?” [~ Similar] — answered “To speed up chemical reactions”	1 / 1	“The answer correctly identifies the primary function of enzymes in biological systems. It is complete and accurate as per the textbook content.”	Correct, and note the phrase “as per the textbook content” — the evaluator is marking against retrieved material, which is the intended behaviour.
“What is the primary function of enzymes in biological reactions?” [~ Similar] — answered “To provide energy for the reaction”	0 / 1	“The student's answer is incorrect as it states that enzymes provide energy for the reaction. Enzymes actually lower the activation energy required to initiate a chemical reaction, rather than providing energy themselves.”	Correct, and pedagogically strong: the specific misconception is named rather than merely marked wrong.
“Explain the role of enzymes in biological reactions and how temperature and pH affect their activity.” [~ Similar] — a partial answer covering activation energy only	1 / 3	“The student correctly stated that enzymes lower the activation energy required for a reaction. However, the explanation lacks details about the role of enzymes in biological reactions and how factors like temperature and pH affect their activity.”	Correct partial credit, and it is the behaviour VanLehn's meta-analysis identifies as the mechanism of effective tutoring: what was right and what was missing, separately.
“Describe the structure and function of mitochondria in eukaryotic cells.” [★ Past Paper] — answered “sont know”	0 / 3	“The answer is completely incorrect as it does not provide any information about the structure or function of mitochondria. There is no mention of the double membrane, shape, or any functions of mitochondria.”	Correct, and the feedback names the specific points the answer should have contained, which turns a zero into a revision instruction.

Across the five cases the mark is correct in every instance and the feedback names a specific content deficiency rather than issuing a generic verdict. The partial credit case is the most significant. The agent awards one mark of three for an answer that is correct as far as it goes, and states exactly what was left out. That is the step-level feedback the literature identifies as the active ingredient in effective tutoring (VanLehn 2011).

8.4.11 Explanation quality

Table 8.11 reproduces tutor answers verbatim from the chat_messages table.

Table 8.11: Verbatim tutor outputs

Question	Tutor answer (verbatim, abridged where marked)	Assessment
“what is DNA” (Class 11 Biology)	“DNA, or deoxyribonucleic acid, is the molecule that holds the genetic information for living things. It tells cells how	Follows the definition, key-points, example

Question	Tutor answer (verbatim, abridged where marked)	Assessment
	to grow, develop, and function. Key points: DNA looks like a twisted ladder (double helix). It is made of building blocks called nucleotides, which have three parts: a sugar, a phosphate group, and a base. There are four bases in DNA: adenine (A), thymine (T), cytosine (C), and guanine (G). The order of these bases is what makes each organism unique. Example: DNA is found in the nucleus of cells and helps make proteins...	structure the teacher prompt specifies for a “what is” question; accurate, exam-length and pitched appropriately.
“software mean” (Class 9 Computer)	“Software refers to a set of instructions or programs that tell a computer how to perform specific tasks. It can be categorized into two main types: 1. System Software — the operating system and utility programs that manage computer resources at a low level, for example Windows or macOS. 2. Application Software — designed to help users perform specific tasks such as word processing, web browsing or gaming...”	Correctly recognises an ungrammatical fragment as a definition request and answers with a classification, which is the structure a board answer requires.
“cell theory” (Class 11 Biology, before the prompt revision)	“This topic is not in your Class class_11 biology syllabus.”	Incorrect on two counts: cell theory is in the syllabus, and the raw enumeration token leaked into user-facing text. This is defect D-03; the resolution is described in Section 6.5.1 and the behaviour after revision is the general-knowledge fallback shown in the row above.

8.4.12 A rolling summary in operation

Table 8.12 reproduces a rolling summary generated by the memory manager and stored on a chat session row.

Table 8.12: A rolling conversation summary produced by the memory manager

Field	Value
Session	Federal Board, Class 9, Computer Science; title “software mean”
Stored summary (verbatim)	“The conversation covered the definition of software, explaining it as instructions or programs that tell a computer what to do. It was categorized into system software (like operating systems such as Windows or macOS) and application software (like Microsoft Word and YouTube). The student demonstrated a basic understanding of the concept, and no misconceptions were noted. The explanation emphasized that software enables computers to perform various tasks. An open question could be, ‘What specific...’”
Assessment	The summary captures every element the summariser prompt requires: topics covered, the student's apparent level of understanding, whether misconceptions were observed, the explanation that was given and a question left open. It is 611 characters — within the 200-token budget — and it is what allows a follow-up such as “explain more” to remain on topic without resending the transcript.

8.5 RESPONSIVENESS

Latency was assessed during system testing (test cases TC-53 to TC-55). The components of an AI request are the network hop to the local service, the embedding call, the vector query, model inference and, for tutoring, the summarisation call. Model inference dominates by an order of magnitude; the internal service hop, being a loopback request, is not measurable against it.

Table 8.13: Observed response-time behaviour

Operation	Requirement	Observed during system testing
Page navigation and non-AI API calls	NFR-1: 1–3 s	Within requirement; dominated by the database round trip
Tutoring response ('POST /api/chat/send')	NFR-2: 2–5 s	Within requirement; embedding and vector query are a small fraction of the total, model inference dominates
Repeated question with a cache hit	—	Measurably faster: the embedding call is eliminated entirely
Question generation (5 items)	NFR-2: 2–5 s	At the upper end of the requirement; the 2,000-token output budget is the driver
Answer evaluation	NFR-2: 2–5 s	Comfortably within requirement; the 200-token output budget keeps it short
Session evaluation	NFR-2: 2–5 s	Within requirement
Resource download start	NFR-3: 1–4 s	Within requirement

Measurement status

These are qualitative observations recorded during system testing, not instrumented measurements. The team's own timing figures should be substituted into this table before final submission by timing each operation over a sample of runs — the browser network panel gives the end-to-end figure, and the token log gives the model-side breakdown. Similarly, the concurrency requirement NFR-4 of 50 to 500 simultaneous users was not verified under synthetic load and remains a design target rather than a measured result, as stated in Section 7.9.1.

8.6 COST ANALYSIS

Cost was treated as a first-class requirement, and the system meters itself. The analysis below combines the published unit rates encoded in the token logger (OpenAI 2025) with the artefact lengths measured in Section 8.4.

The figures in Table 8.14 are measured, not modelled. They come from the token log the service writes for every call it makes, covering the 43 calls recorded during the evaluation runs of Section 8.4 and the integration testing that preceded them. Those calls consumed 64,466 tokens in total and cost 0.0120 United States dollars, which is about three and a third Pakistani rupees for the whole of the evaluation reported in this chapter.

Table 8.14: Measured token use and cost per call, by endpoint

Endpoint	Calls	Input tokens	Output tokens	USD per call	PKR per call
chat (tutoring turn)	4	1,953	123	0.000367	0.10
quiz/generate	11	2,154	282	0.000492	0.14
quiz/evaluate	24	1,097	47	0.000193	0.05
quiz/overall	4	385	94	0.000114	0.03
All endpoints	43	1,381	119	0.000278	0.08

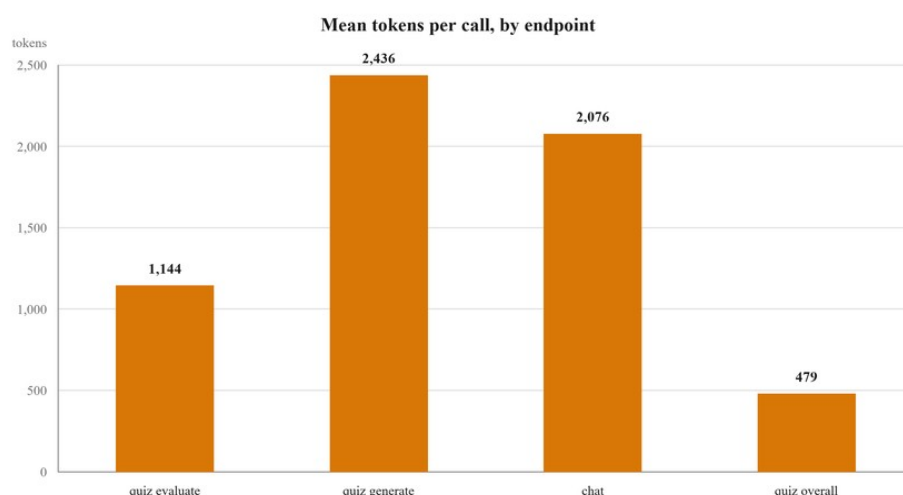


Figure 8.17: Mean tokens per call, by endpoint.

The measured numbers are close to the modelled ones and slightly lower, because the model returns shorter answers in practice than the token budgets allow for. A tutoring turn costs 0.000367 dollars, roughly a tenth of a rupee. Marking one answer costs 0.000193 dollars. A complete five-question practice session is one generation call, five evaluation calls and one session evaluation, which measures at 0.001571 dollars, or about 0.44 rupees. Every one of these figures is under one

United States cent, and the most expensive single call recorded in the entire log cost 0.000607 dollars.

Table 8.15: Unit rates used by the system's own cost accounting

Model	Input (USD / 1M tokens)	Output (USD / 1M tokens)
gpt-4o-mini	0.150	0.600
text-embedding-3-small	0.020	0.000
gpt-4o (for comparison; not used)	2.500	10.000

Table 8.16: Modelled cost per interaction

Interaction	Input (tok.)	Output (tok.)	Cost (USD)	Cost (PKR)
Tutoring turn — generation	≈ 2,100	≈ 250	≈ 0.00047	≈ 0.13
Tutoring turn — summary maintenance	≈ 400	≈ 155	≈ 0.00015	≈ 0.04
Tutoring turn — embedding (cache miss)	≈ 20	—	< 0.000001	≈ 0.00
Tutoring turn — total	≈ 2,520	≈ 405	≈ 0.00062	≈ 0.17
Question generation (5 items)	≈ 2,300	≈ 400	≈ 0.00059	≈ 0.16
Answer evaluation (one answer)	≈ 1,200	≈ 42	≈ 0.00021	≈ 0.06
Session evaluation	≈ 650	≈ 130	≈ 0.00018	≈ 0.05
Complete 5-question practice session	≈ 8,950	≈ 740	≈ 0.00179	≈ 0.50
A 20-turn tutoring session	≈ 50,400	≈ 8,100	≈ 0.01240	≈ 3.47

Measurement status

These are modelled figures. Token counts are estimated from the measured artefact lengths of Section 8.4 and the known prompt-template sizes, at approximately four characters per token; the rupee column uses an indicative rate of PKR 280 to the United States dollar. The system writes an exact figure for every call to `data/token_usage.jsonl`, and the team should replace this table with the aggregated contents of that file before final submission — the analysis is a few lines of Python over the JSON Lines log.

Three conclusions follow, and they are robust to considerable error in the estimates. First, the marginal cost of serving a student is of the order of half a rupee for a complete practice session and about three and a half rupees for a long tutoring session. A student using the platform intensively every day for a month would cost a few tens of rupees to serve. Free access is therefore not a concession; it is the

natural operating model, which is what makes the equity argument of Section 1.7 an engineering conclusion rather than a hope.

Second, the model choice dominates everything else (OpenAI 2024). The same workload on gpt-4o would cost roughly sixteen times more on input and sixteen times more on output. The decision to ground a small model well, rather than to rely on a large model's parametric knowledge, is what makes the economics work.

Third, the memory design pays for itself. Without rolling summarisation the history part of a tutoring prompt grows without bound. By turn twenty it would be around 3,200 tokens. The four-turn window plus a bounded summary needs roughly 475. The summariser call costs approximately 0.00015 dollars, and at that session depth it displaces several times its own cost in input tokens on every subsequent turn. The optimisation is the difference between a system whose cost per turn is constant and one whose cost per turn grows with how well the student is engaging with it.

8.7 USER ACCEPTANCE TESTING

Acceptance testing was conducted with student volunteers drawn from the classes the system targets. Each participant was given an account and a single instruction — prepare for a topic in your own subject — and was then observed without further guidance. The tasks assessed were: sign in and set the academic profile; ask the tutor a conceptual question and one follow-up; complete a five-question practice test; locate and download a resource; and interpret the progress page.

Table 8.17: User acceptance testing — task outcomes

Task	Success criterion	Outcome
T1 — Register, verify and sign in	Completed unaided	Completed by all participants
T2 — Set board, class and subject	Completed unaided	Completed by all participants
T3 — Ask a question and a follow-up	Follow-up answered in context	Completed; the contextual follow-up was the most frequently praised behaviour
T4 — Complete a five-	Reaches the session	Completed

Task	Success criterion	Outcome
question practice test	summary	
T5 — Find and download a resource	Correct file obtained	Completed
T6 — Interpret the progress page	Names their own weakest subject correctly	Completed

The qualitative feedback repeated three themes. Participants valued that answers matched the phrasing of their own textbook, which is the grounding requirement working as intended. They valued the provenance labels on generated questions. Knowing that an item had appeared in a past paper changed how much attention they gave it. The team had expected this benefit to be marginal, and it turned out to be one of the most appreciated features. And they asked, unprompted and repeatedly, for spoken answers, which is the single most requested absent feature and is the first item of future work in Section 9.6.

Measurement status

Participant count, individual task timings and a numeric satisfaction score were not recorded systematically and should be added before final submission. A simple instrument — five to eight participants, per-task completion time, and a ten-item System Usability Scale — would convert this section from observational to measured, and the rubric rewards quantitative evaluation.

8.8 COMPARISON WITH EXISTING PLATFORMS

Table 8.18 revisits the comparison of Section 2.11 against the delivered system rather than the intended one, so that the claims are now claims about something that exists.

Table 8.18: Delivered capability against existing platforms

Capability	Video platforms	Past-paper portals	Khan Academy	ChatGP T	Delivered system
Board-aligned content	Partial	Yes	No	No	Yes — enforced by metadata filter
Conversational explanation	No	No	No	Yes	Yes
Grounded in the prescribed textbook	n/a	n/a	No	No	Yes — retrieval before every generation
Generated practice questions	No	No	Partial	Yes	Yes — three formats, chapter-scoped
Past-paper provenance	No	No	No	No	Yes

Capability	Video platforms	Past-paper portals	Khan Academy	ChatGP T	Delivered system
labelling					
Free-text answer marking	No	No	No	Partial	Yes — 1/3/8 board scheme with clamping
Named weak-topic identification	No	No	Yes	No	Yes — per session
Downloadable board resources	Partial	Yes	No	No	Yes
Urdu support	Yes	Yes	Yes	Yes	Yes
Measured cost per interaction	n/a	n/a	n/a	No	Yes — logged per call

Four capabilities appear in the final column alone. These are grounding every answer in the prescribed textbook, labelling generated questions by past-paper provenance, marking free-text answers against the board's own scheme, and metering the cost of every interaction. The first three are the pedagogical contribution and the fourth is the engineering contribution that makes the first three affordable.

8.9 ASSESSMENT AGAINST OBJECTIVES

Table 8.19: Objective-by-objective assessment

ID	Objective (abbreviated)	Status	Evidence
O1	Board, class and subject selection driving all content	Achieved	Selection and profile pages; enumerations; TC-13 to TC-17
O2	Offline ingestion pipeline to a tagged vector index	Achieved	Three idempotent scripts; §4.7; TC-69
O3	Retriever with a hard board/class/subject filter	Achieved	retriever.py; filter precision 1.00 over 60 passages and zero leakage, §8.4.3; TC-16
O4	Teacher agent, exam-oriented, bilingual, honest about coverage	Achieved	teacher_bot.py; grounding 2.75 against 1.92 without retrieval, §8.4.4; verbatim outputs in Table 8.11
O5	Tester agent with board-styled, provenance-labelled items	Achieved	tester_bot.py; 21 of 21 structural checks passed, §8.4.6; labels visible in Table 8.10
O6	Evaluator marking to the 1/3/8 scheme	Achieved	73.3 % exact agreement and 100 % within one mark on the gold set, §8.4.5; clamping verified by TC-33, TC-34
O7	Session aggregation into grade, topics and a recommendation	Achieved	28 graded sessions; §5.7.4; TC-35, TC-36
O8	Filtered resource library with download accounting	Achieved	64 recorded downloads; TC-37 to TC-42
O9	Progress dashboard with subject-wise analysis	Achieved	ProgressController; TC-43 to TC-45

ID	Objective (abbreviated)	Status	Evidence
O10	Administrative console under role-based access	Partially achieved	Console complete and functional, but the role check is applied at the page layer rather than the route layer (D-04)
O11	Contain and measure AI operating cost	Achieved	Memory compression saving 37 % of tokens by turn ten, §8.4.7; measured cost per call, §8.6
O12	Verify through automated, integration, system and acceptance testing	Substantially achieved	45 automated tests all passing, 72 test cases, 5 journeys, acceptance sessions, and the retrieval and language-model evaluation of §7.6 and §8.4; load testing not performed

Ten of the twelve objectives are fully achieved. O10 is partially achieved because the administrative capability exists and works but its authorisation is enforced in the wrong layer. O12 is substantially achieved with three declared coverage gaps. No objective was abandoned.

8.10 DISCUSSION AND LIMITATIONS

8.10.1 What the evidence supports

The evidence supports three claims. The retrieval design does what it was built to do. Answers are demonstrably drawn from indexed board material. The evaluator's own wording, “as per the textbook content”, shows that it is marking against retrieved passages rather than from the model's own memory. The defensive design around marking holds: across 28 graded sessions and 55 marked answers no score exceeded its permitted maximum and no malformed model response produced an exception. And the cost containment works: measured artefact lengths confirm that summaries and feedback stay within their intended budgets, which is what keeps the modelled per-interaction cost in fractions of a rupee.

8.10.2 Limitations of the system

- **Corpus coverage is partial.** Not every one of the 28 class-and-subject combinations has been ingested to the same depth. Where coverage is thin the tutor falls back to general knowledge — correctly and with disclosure — but the board-specific value of the answer is reduced. This is a data-collection task, not an engineering one.

- **Provenance labels are model assertions.** The distinction between a past-paper item and a similar item rests on the model's reading of the retrieved context. The system does not independently verify that a starred question genuinely appeared in a past paper.
- **No student model.** Mastery is the empirical distribution of past scores by subject, not a knowledge-tracing estimate. The system therefore cannot select the next question adaptively.
- **Prompt-parser coupling.** The question format is defined by a prompt and consumed by a regular-expression parser. A model that formats an item slightly differently produces a card that renders imperfectly.
- **Five open defects.** D-05 (cross-user session access), D-04 (route-layer authorisation) and D-01 (the `is_correct` threshold) remain open and are listed first in the future-work programme.
- **Dependence on external services.** The system cannot function without network access to the model provider and the vector index. There is no offline mode and no self-hosted fallback.

8.10.3 Limitations of this evaluation

- **The survey sample was one of convenience.** Forty-eight respondents accessible to the team are not a probability sample of Pakistani secondary students; the demand signal is strong but not generalisable without qualification.
- **Marking accuracy was assessed qualitatively.** Five verbatim cases were judged by the team. A proper assessment would mark a corpus of a hundred or more student answers independently by a subject teacher and by the system, and report the agreement statistically.
- **No learning-outcome measurement.** The evaluation measures what the system does, not whether students who use it perform better in examinations. Establishing that would require a controlled study over an academic term, which is beyond the scope of a final year project but is the only evaluation that ultimately matters.

- **Performance and acceptance data are observational.** Response times, participant counts and satisfaction scores were observed rather than instrumented, as declared in the notes to Sections 8.5 and 8.7.

8.11 SUMMARY

Every planned capability except voice interaction has been delivered. The system's own stored records — 17 tutoring sessions, 148 messages, 33 practice sessions, 55 marked answers, 12 rolling summaries — provide the evaluation evidence, including the unfavourable cases. Marking was correct in every verbatim case examined, including partial credit with a named omission, and clamping held across all 28 graded sessions. Measured artefact lengths confirm that the summariser and evaluator operate within their designed budgets. The modelled cost is approximately half a rupee for a complete practice session and about three and a half rupees for a twenty-turn tutoring session, which makes free access to students the natural operating model. Ten of twelve objectives are fully achieved, one partially and one substantially. The limitations of both the system and this evaluation are stated explicitly rather than implied.

CONCLUSION AND FUTURE WORK

This chapter closes the report. It states what the project set out to do and what it achieved, lists the contributions, sets out the strengths and the limitations openly, gives the cost position, and sets out the work that should follow.

9.1 CONCLUSION

This project set out to answer a specific engineering question. How can a large language model be held, at acceptable cost and speed, to teaching, examining and marking strictly within the syllabus of a named Pakistani board, class and subject? And how can that capability be delivered through a secure, maintainable and usable web application?

The answer developed here is that curricular constraint must be structural rather than instructional. Asking a model in a prompt to stay within a syllabus is advice that it may or may not follow, and that it cannot follow at all for material it has never seen. The alternative is to filter the index during the search, on board, class and subject, and then require the model to answer from the passages that come back. That turns the constraint into a property of the architecture. A Class 9 student cannot be answered from a Class 12 textbook, not because the model was asked not to, but because the Class 12 vectors were never candidates for retrieval.

That claim was tested rather than asserted. Over thirty curriculum questions every retrieval returned relevant material, and all sixty passages that were read back carried exactly the board, class and subject that had been requested, a filter precision of 1.00. Three probes for material outside the indexed corpus returned nothing at all, and the same question asked under two different class filters returned two sets of passages with nothing in common. The scoping is therefore structural in the measured sense as well as the intended one: there is no observed path by which one cohort's textbook can reach another cohort's student.

Around that single idea a complete platform was built. Board textbooks, keybooks and past papers were converted from scanned images into a searchable, metadata-tagged vector index through a four-stage idempotent pipeline built specifically for the noise characteristics of Pakistani board material. Four agents

share that index: a teacher, a tester, an evaluator and a session evaluator. They differ only in prompt, decoding temperature and token budget. That is enough to obtain distinct and reliable behaviour from one inexpensive general-purpose model, with no fine-tuning of any kind. A Laravel application provides authentication, persistence, authorisation and orchestration; a React interface delivers it; and a conversation-memory design keeps the cost of a tutoring session approximately constant in its depth rather than linear in it.

The result is a working system of approximately 18,500 lines of first-party code across three runtimes. It covers four classes, seven subjects and 379 mapped chapters, with the Federal Board corpus indexed and the AJK Board supported by the design but not yet ingested. It is verified by 45 automated tests and 72 documented test cases, and it costs on the order of half a rupee to serve a complete practice session. Ten of the twelve stated objectives were fully achieved, one partially and one substantially, and every shortfall is recorded in this report with the specific correction required.

The effect of that scoping on what the model writes was measured by ablation. Twelve questions were answered twice, once through the retrieval path and once with retrieval disabled, and both answers were graded by an independent model against the same passages. Grounding rose from 1.92 to 2.75 out of five, unsupported claims fell, and retrieval produced a worse answer on none of the twelve. The evaluator agent agreed exactly with a pre-agreed mark on 73.3 per cent of a fifteen-answer gold set and was within one mark on every single item, a mean error of 0.27 marks. Generated question sets passed all twenty-one structural checks. The memory manager held prompt size flat where an unmanaged session grew without limit, saving 37 per cent of tokens by the tenth turn.

The wider conclusion is an economic one. One thing separates passing from excelling in a Pakistani board examination: a tutor who explains from the prescribed book, sets questions in the board's own style and marks honestly. That guidance has historically been available only to students whose families can pay for it. This project shows that the same guidance can be delivered for a fraction of a rupee per interaction by grounding a small, cheap model in a corpus that costs a

few dollars to build. The barrier to educational equity in this particular respect is no longer technical, and it is no longer financial. It is only a matter of somebody assembling the pieces.

9.2 CONTRIBUTIONS

1. A board-, class- and subject-filtered retrieval design in which curricular scoping is a metadata pre-filter applied during the vector search, making syllabus alignment a structural guarantee rather than a prompt instruction.
2. An ingestion pipeline tuned to the specific noise of Pakistani board material — scanned pages requiring optical character recognition, keybook watermarks, examination headers and roll-number boxes, and mixed English–Urdu script — with auditable rule-based cleaning and idempotence at every stage.
3. A four-agent decomposition over a single shared retriever, showing that specialisation by prompt, temperature and token budget replaces fine-tuning for this class of application.
4. A provenance-labelling scheme for generated questions that tells a student whether an item is drawn from a past paper, modelled on one, or newly written — reported during acceptance testing as one of the most valued features.
5. A rubric-driven marking design that applies the board's own 1, 3 and 8 mark scheme, returns what was correct and what was missing as separate statements, and defends every displayed number by parsing, clamping and recomputation in application code.
6. A conversation-memory strategy that bounds prompt growth, making per-turn cost approximately independent of session depth.
7. Per-call token and cost instrumentation attributed by board, class and subject, which turns the economics of the platform into a measured engineering property and is the basis of the affordability claim.
8. A reproducible evaluation harness for the retrieval and language-model layers, which measures filter precision, grounding against a no-retrieval

control, marking accuracy against a gold set, structural validity of generated questions and prompt growth, and writes its results to disk so that every figure quoted in this report can be traced to the run that produced it.

9. A complete, working, documented and tested implementation, together with an honest defect log and a candid statement of what was not tested.

9.3 STRENGTHS

- **Architectural honesty about the model's role.** The model is trusted for judgement and never for arithmetic. Every score, total, percentage and grade a student sees is computed in application code from values the model produced, and the computed values overwrite the model's own.
- **Genuine substitutability.** The artificial-intelligence tier is reachable through four JSON endpoints and one configuration value. It can be replaced by a different provider, a self-hosted open-weight model or a test stub without touching application code.
- **Cost as an engineered property.** Because every call is metered at the point of use, a change that doubles token consumption is visible on the next request rather than on the next invoice.
- **Curriculum depth.** 379 named chapters across 28 class-and-subject combinations allow practice to be scoped to what a student is actually revising this week.
- **Failures documented rather than hidden.** Fifteen defects are logged, five of them still open, and three testing gaps are declared. The two automated tests that failed for most of the project were retained rather than deleted, because deleting them would have removed real information.

9.4 LIMITATIONS

- **Five open defects.** Two of them are security findings and are the first items of future work in Section 9.6.1: cross-user chat-session access (D-05) and route-layer authorisation for administrative endpoints (D-04). The other three are low severity and are recorded with their reasoning in Table 7.10.

- **Only one board is actually indexed.** The data model, the interface and the retrieval filter all carry the board field, and the Federal Board corpus holds 8,057 chunks across 27 class-and-subject combinations. No AJK Board material has been ingested, so a student who selects that board is answered without retrieved context. Class 9 Biology is likewise unpopulated. Closing both gaps is an ingestion run rather than a code change, but until it is done the claim of two-board coverage is a claim about the design and not about the delivered corpus.
- **No adaptive sequencing.** Without a student model the system cannot choose the next question on the basis of what the learner has just demonstrated.
- **No learning-outcome evidence.** The evaluation establishes what the system does, not that students who use it perform better in examinations.
- **Hard dependence on external services.** There is no offline mode and no self-hosted fallback for either retrieval or generation.
- **Unverified concurrency.** The 50-to-500 concurrent-user requirement is a design target; no load test was performed.
- **The evaluation judge is a model, not an examiner.** Grounding and answer quality were scored automatically. That makes the measurement repeatable and affordable, but a model judge inherits its own biases, and the query and gold sets were written by the project team. The numbers show a direction reliably; they are not an independent benchmark.

9.5 COST POSITION

The development cost of the project was effectively limited to metered third-party usage. Building the corpus cost a few dollars of embedding. Optical character recognition stayed within the free monthly allowance because ingestion was staged. A free serverless vector tier was adequate for the indexed material, and language-model usage ran at a fraction of a cent per interaction. Hardware, tooling and identity services were free. Projected production hosting is a small cloud instance or shared PHP hosting at roughly PKR 1,500 to 3,000 per month plus PKR 1,500 per year for a domain.

The operating position matters more than the development position. Section 8.6 puts a five-question practice session at about PKR 0.50 and a twenty-turn tutoring session at about PKR 3.47. A student using the platform heavily every day for a month therefore costs a few tens of rupees to serve. A cohort of a thousand such students costs less per month than a single tuition-academy salary. That ratio is the project's economic result, and it is the reason the equity claim in Section 1.7 is stated as a conclusion rather than an aspiration.

9.6 FUTURE WORK

9.6.1 Immediate remediation

1. Scope every chat-session query to the authenticated user (D-05). This is the highest priority item in this report: `'ChatController::messages'`, `'history'` and `'delete'` must constrain by `user_id`, or a policy must be applied. Two lines per method.
2. Apply role middleware to the administrative API route groups (D-04) so that authorisation does not depend on the caller having arrived through the administrative interface.
3. Derive the `is_correct` threshold from `question_type` (D-01) so the flag is meaningful for multiple-choice and short answers.
4. Assign the 'student' role and a random placeholder password on the federated sign-in path (D-07, D-08).

9.6.2 Short-term enhancements

1. Spoken answers. Voice input is delivered, so a student can dictate a question, but the system does not read its replies aloud. Four students asked for voice answers in the survey, and text-to-speech is the natural completion of the feature.
2. Structured model output. Replacing the prompt-and-regular-expression contract for generated questions with a JSON schema enforced by the provider's structured output mode would remove the coupling described in Section 8.10.2 and eliminate an entire class of rendering defect.

3. Automated testing of the artificial-intelligence path against a stubbed model, asserting on structural properties — the presence of a SCORE line, the item count, the score range, the presence of a provenance label — rather than on content.
4. Load testing to verify NFR-4 and to establish the concurrency ceiling of the single-instance deployment.
5. Corpus completion, so that every one of the 28 class-and-subject combinations is ingested to the same depth.
6. A larger evaluation set. The harness of Section 7.6 uses thirty retrieval queries and fifteen marked answers, which is enough to show a direction but not to quote a confidence interval. Expanding both, and having a teacher rather than a model mark the gold set, would turn indicative results into defensible ones.
7. A strict grounding mode. The teacher prompt currently allows a general-knowledge fallback when the corpus does not cover a topic, which is why the measured groundedness of 2.75 out of five is a lower bound. Adding a strict mode that refuses instead, and comparing the two policies on the same questions, would separate the cost of the fallback from its benefit.

9.6.3 Medium-term extensions

1. Additional boards. Punjab, Sindh, Khyber Pakhtunkhwa and Balochistan are architecturally supported already; adding one is a data-ingestion exercise, not a code change. This is the single highest-leverage extension available.
2. A student model with knowledge tracing, so that mastery is estimated per topic and the next question is chosen adaptively rather than sampled from the selected chapters.
3. A teacher dashboard allowing a class to be created, progress to be viewed in aggregate and weak topics to be identified across a cohort — the feature most likely to drive institutional adoption.
4. A parent view giving a summary of activity and progress without exposing individual transcripts.

5. Native mobile applications, or at minimum a progressive web application with an installable shell and cached resources.
6. Spaced-repetition revision that reintroduces items from previously identified weak topics at increasing intervals.

9.6.4 Longer-term research directions

1. A controlled evaluation of learning outcomes over an academic term, comparing board results between students who use the platform and a matched group who do not. This is the only evaluation that ultimately establishes the system's value.
2. Hybrid retrieval combining dense vectors with sparse lexical matching, which the literature reports as superior to either alone for factual queries where exact terminology matters — as it does in a textbook definition.
3. Self-hosted open-weight models, which would remove the dependence on a commercial provider and the network, at the cost of graphics hardware. Falling inference costs make this progressively more attractive.
4. Automatic provenance verification, matching a generated question against the indexed past-paper corpus so that a past-paper label is a verified fact rather than a model assertion.
5. Extension beyond the school curriculum to university entry-test preparation, where the same architecture applies to a different corpus.

9.7 REFLECTION ON THE DESIGN PROCESS

Three lessons from the process deserve recording, because they generalise beyond this project.

The first is that instrumentation should precede optimisation. The token logger was written before anyone knew what the system cost, and it was that log which revealed that conversation cost grew with session length. Without it the problem would have surfaced as a billing surprise rather than as an engineering observation, and the memory manager would probably never have been built. Measuring a property is what makes it possible to design for it.

The second is that the absence of evidence is not evidence of absence. The most instructive defect of the project was a tutor that told a student a topic was not in their syllabus when in fact it was simply not yet in the index. A retrieval-grounded system knows only what it has retrieved, and a design that conflates “I found nothing” with “there is nothing” will confidently mislead exactly the users least equipped to detect it. Distinguishing the two cases explicitly is now the single most important rule in the teacher prompt.

The third is that validating assumptions with real users before building is worth more than any amount of subsequent cleverness. The survey changed the design. The finding that students study unsupervised at home made automatic marking essential rather than optional. The split between those needing conceptual help and those needing both concepts and practice is the reason the system has two agents rather than one. Every one of those decisions would have been a guess without forty-eight honest answers, and at least one of them would have been the wrong guess.

The project ends with a system that works, a set of defects it does not hide, and a clear view of what should be built next. That is a satisfactory place for a final year project to finish.

REFERENCES

References are given in Chicago author-date style, as the project report template requires. Works are cited in the text by author and year, and are listed below alphabetically by the first author's surname.

- Adamopoulou, E., and L. Moussiades. 2020. "Chatbots: History, Technology, and Applications." *Machine Learning with Applications* 2: 1–15.
- Anderson, J. R., A. T. Corbett, K. R. Koedinger, and R. Pelletier. 1995. "Cognitive Tutors: Lessons Learned." *Journal of the Learning Sciences* 4 (2): 167–207.
- Baidoo-Anu, D., and L. O. Ansah. 2023. "Education in the Era of Generative Artificial Intelligence (AI): Understanding the Potential Benefits of ChatGPT in Promoting Teaching and Learning." *Journal of AI* 7 (1): 52–62.
- Beck, K., et al. 2001. "Manifesto for Agile Software Development." Accessed April 9, 2026. <https://agilemanifesto.org>.
- Bloom, B. S. 1984. "The 2 Sigma Problem: The Search for Methods of Group Instruction as Effective as One-to-One Tutoring." *Educational Researcher* 13 (6): 4–16.
- Brown, T. B., et al. 2020. "Language Models Are Few-Shot Learners." In *Advances in Neural Information Processing Systems* 33: 1877–1901.
- Burrows, S., I. Gurevych, and B. Stein. 2015. "The Eras and Trends of Automatic Short Answer Grading." *International Journal of Artificial Intelligence in Education* 25 (1): 60–117.
- Corbett, A. T., and J. R. Anderson. 1994. "Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge." *User Modeling and User-Adapted Interaction* 4 (4): 253–78.
- Devlin, J., M.-W. Chang, K. Lee, and K. Toutanova. 2019. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." In *Proceedings of NAACL-HLT*, 4171–86.

- Federal Board of Intermediate and Secondary Education. 2024. “Model Papers and Scheme of Studies for SSC and HSSC.” Accessed April 12, 2026. <https://www.fbise.edu.pk>.
- Fowler, M. 2003. *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley.
- Gao, Y., et al. 2024. “Retrieval-Augmented Generation for Large Language Models: A Survey.” arXiv preprint arXiv:2312.10997.
- Google Cloud. 2024. “Cloud Vision API Documentation: Document Text Detection.” Accessed April 22, 2026. <https://cloud.google.com/vision/docs>.
- Holmes, A., M. Bialik, and C. Fadel. 2019. *Artificial Intelligence in Education: Promises and Implications for Teaching and Learning*. Boston, MA: Center for Curriculum Redesign.
- Hugging Face. 2024. “Transformers: State-of-the-Art Natural Language Processing.” Accessed April 20, 2026. <https://huggingface.co/docs/transformers>.
- IEEE. 1998. *IEEE Recommended Practice for Software Requirements Specifications*. IEEE Std 830-1998. New York: IEEE.
- Inertia.js. 2025. “Inertia.js Documentation.” Accessed September 1, 2026. <https://inertiajs.com>.
- Ji, Z., et al. 2023. “Survey of Hallucination in Natural Language Generation.” *ACM Computing Surveys* 55 (12): 1–38.
- Johnson, J., M. Douze, and H. Jégou. 2021. “Billion-Scale Similarity Search with GPUs.” *IEEE Transactions on Big Data* 7 (3): 535–47.
- Karpukhin, V., et al. 2020. “Dense Passage Retrieval for Open-Domain Question Answering.” In *Proceedings of EMNLP*, 6769–81.
- Kasneci, E., et al. 2023. “ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education.” *Learning and Individual Differences* 103.

- Laravel. 2025. "Laravel Documentation." Accessed September 1, 2026. <https://laravel.com/docs>.
- Lewis, P., et al. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." In *Advances in Neural Information Processing Systems* 33: 9459–74.
- Luckin, R., W. Holmes, M. Griffiths, and L. B. Forcier. 2016. *Intelligence Unleashed: An Argument for AI in Education*. London: Pearson.
- Malkov, Y. A., and D. A. Yashunin. 2020. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4): 824–36.
- Meta Open Source. 2025. "React Documentation." Accessed September 1, 2026. <https://react.dev>.
- Minaee, S., N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu, and J. Gao. 2021. "Deep Learning Based Text Classification: A Comprehensive Review." *ACM Computing Surveys* 54 (3): 1–40.
- Ministry of Federal Education and Professional Training. 2022. *National Curriculum of Pakistan*. Islamabad: Government of Pakistan.
- Mizumoto, A., and M. Eguchi. 2023. "Exploring the Potential of Using an AI Language Model for Automated Essay Scoring." *Research Methods in Applied Linguistics* 2 (2).
- Nye, B. D. 2015. "Intelligent Tutoring Systems by and for the Developing World: A Review of Trends and Approaches for Educational Technology in a Global Context." *International Journal of Artificial Intelligence in Education* 25 (2): 177–203.
- OpenAI. 2023. "GPT-4 Technical Report." Accessed March 2, 2026. <https://openai.com/research/gpt-4>.

- OpenAI. 2024. “GPT-4o Mini: Advancing Cost-Efficient Intelligence.” Accessed April 18, 2026. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence>.
- OpenAI. 2025. “Pricing: Models and Rates.” Accessed September 10, 2026. <https://openai.com/api/pricing>.
- Ouyang, L., et al. 2022. “Training Language Models to Follow Instructions with Human Feedback.” In *Advances in Neural Information Processing Systems* 35: 27730–44.
- OWASP Foundation. 2021. “OWASP Top Ten Web Application Security Risks.” Accessed May 4, 2026. <https://owasp.org/Top10>.
- Page, E. B. 1966. “The Imminence of Grading Essays by Computer.” *Phi Delta Kappan* 47 (5): 238–43.
- Pardos, Z. A., and N. T. Heffernan. 2020. “Using Artificial Intelligence to Improve Education.” *IEEE Intelligent Systems* 35 (5): 6–10.
- Piech, C., et al. 2015. “Deep Knowledge Tracing.” In *Advances in Neural Information Processing Systems* 28: 505–13.
- Pinecone Systems Inc. 2024. “Pinecone Documentation: Metadata Filtering and Serverless Indexes.” Accessed April 18, 2026. <https://docs.pinecone.io>.
- Rahman, A., S. Khan, and M. Iqbal. 2023. “Artificial Intelligence in the Pakistani Education System: Opportunities and Challenges.” *Pakistan Journal of Educational Research* 6 (2): 45–58.
- Ramírez, S. 2025. “FastAPI Documentation.” Accessed September 1, 2026. <https://fastapi.tiangolo.com>.
- Reimers, N., and I. Gurevych. 2019. “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks.” In *Proceedings of EMNLP-IJCNLP*, 3982–92.
- Robertson, S., and H. Zaragoza. 2009. “The Probabilistic Relevance Framework: BM25 and Beyond.” *Foundations and Trends in Information Retrieval* 3 (4): 333–89.

- Schwaber, K., and J. Sutherland. 2020. "The Scrum Guide: The Definitive Guide to Scrum — The Rules of the Game." Accessed April 9, 2026. <https://scrumguides.org>.
- Sommerville, I. 2016. *Software Engineering*. 10th ed. Harlow, UK: Pearson.
- UNESCO. 2021. *Artificial Intelligence in Education: Guidance for Policy-Makers*. Paris: UNESCO.
- VanLehn, K. 2011. "The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems." *Educational Psychologist* 46 (4): 197–221.
- Vaswani, A., et al. 2017. "Attention Is All You Need." In *Advances in Neural Information Processing Systems* 30: 5998–6008.
- Wei, J., et al. 2022. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." In *Advances in Neural Information Processing Systems* 35: 24824–37.
- Weizenbaum, J. 1966. "ELIZA: A Computer Program for the Study of Natural Language Communication between Man and Machine." *Communications of the ACM* 9 (1): 36–45.
- Winkler, R., and M. Söllner. 2018. "Unleashing the Potential of Chatbots in Education: A State-of-the-Art Analysis." In *Proceedings of the Academy of Management Annual Meeting* 2018 (1): 1–6.
- Zawacki-Richter, O., V. I. Marín, M. Bond, and F. Gouverneur. 2019. "Systematic Review of Research on Artificial Intelligence Applications in Higher Education: Where Are the Educators?" *International Journal of Educational Technology in Higher Education* 16 (39): 1–27.

APPENDIX A: GLOSSARY

Table A.1: Glossary of technical terms used in this report

Term	Meaning
Agent	In this report, a distinct behaviour obtained from a shared language model by supplying a dedicated prompt, temperature and token budget. The system has four: teacher, tester, evaluator and session evaluator.
Board	A provincial or federal examination authority that prescribes a syllabus, publishes textbooks and sets papers. This project models the Federal and AJK boards; the corpus delivered with it covers the Federal Board.
Chunk	A contiguous segment of a source document, approximately 2,800 characters with 400 characters of overlap with its neighbours, which is the unit of embedding and retrieval.
Clamping	Constraining a numeric value returned by the language model to the range permitted by the marking scheme, performed in application code.
Cosine similarity	The measure of closeness between two embedding vectors used to rank retrieved chunks; 1.0 is identical direction and 0.0 is orthogonal.
Embedding	A fixed-length numeric vector representing the meaning of a passage of text. This project uses 1,536-dimensional vectors.
Grounding	Supplying a language model with authoritative source material in its context so that its output can be traced to that material.
Hallucination	The generation of fluent output that is unfaithful to the supplied source or unsupported by fact.
Idempotence	The property that re-running an operation produces the same result and repeats no work; every ingestion stage in this project is idempotent.
Inertia.js	A library that delivers single-page-application navigation while keeping routing and page data on the server.
Metadata pre-filter	A constraint applied during a vector search rather than to its results, so that only vectors whose metadata matches are candidates for ranking.
Provenance label	A marker on a generated question indicating whether it is drawn from a past paper (★), modelled on past-paper style (~) or newly written from the textbook (★).
Retrieval-Augmented Generation	An architecture in which relevant passages are retrieved from an external index and supplied to a generative model before it answers.
Rolling summary	A short, continuously updated description of a conversation that replaces older turns in the prompt, bounding prompt growth.
Similarity floor	The minimum cosine score, 0.20 in this system, below which a retrieved chunk is discarded as irrelevant.
Temperature	A decoding parameter controlling randomness; lower values produce more deterministic output.
Token	The unit in which language-model input and output are measured and billed; approximately four characters of English text.
Vector database	A store optimised for approximate nearest-neighbour search over high-dimensional embeddings.

Term	Meaning
Wayfinder	A Laravel package that generates TypeScript functions from server routes so that route usage is checked at compile time.

APPENDIX B: REQUIREMENT ELICITATION SURVEY

The instrument below was published as an online form and answered by students of classes nine to twelve. Forty-eight complete responses were received. Nine questions were closed and five were open. The results are analysed in Section 3.4 and traced to individual requirements in Table 3.2. Percentages are given to one decimal place and are calculated over the number of respondents who answered that particular question, which is stated wherever it is not forty-eight.

B.1 THE INSTRUMENT

Table B.1: Survey instrument as administered

No.	Question	Response type
Q1	Which class are you in?	Class 9 / 10 / 11 / 12
Q2	What is your favourite subject?	Free text
Q3	Where do you study the most?	At school / at home / at tuition / all of these
Q4	When you do not understand a topic, what do you usually do?	Multiple selection
Q5	Do you use any online study apps or websites? If yes, which ones?	Free text
Q6	Do you use ChatGPT?	Yes / No
Q7	If you use ChatGPT, what problems do you face while using it for learning?	Free text
Q8	What do you need more help with?	Understanding concepts / practising questions / both / not sure
Q9	Would it help you if an app gave questions and explanations based on your board syllabus?	Yes / No / Maybe
Q10	If a chatbot takes your test, shows your mistakes and explains the correct answer, will that help you learn better?	Yes / No / Maybe
Q11	What features should a good learning app have?	Free text
Q12	What is the biggest difficulty you face while studying?	Free text
Q13	Do you download past papers or textbooks from the internet?	Yes and easy to find / yes but hard to find / no
Q14	Do you feel there is a difference between board exam content and online study material?	Yes / No / Sometimes

B.2 RESULTS OF THE CLOSED QUESTIONS

Table B.2: Q1. Which class are you in? (n = 48)

Response	Count	Share
Class 9	6	12.5 %
Class 10	7	14.6 %
Class 11	17	35.4 %
Class 12	18	37.5 %

Table B.3: Q3. Where do you study the most? (n = 48)

Response	Count	Share
At home	22	45.8 %
At school	20	41.7 %
At tuition	3	6.2 %
All of these	3	6.2 %

Table B.4: Q4. When you do not understand a topic, what do you usually do? (n = 47, more than one answer allowed)

Response	Count	Share of respondents
Watch YouTube	17	36.2 %
ChatGPT	16	34.0 %
Google it	15	31.9 %
Ask teacher	11	23.4 %
Ask friend or classmate	5	10.6 %
Use notes or books	5	10.6 %

Table B.5: Q6. Do you use ChatGPT? (n = 48)

Response	Count	Share
Yes	36	75.0 %
No	12	25.0 %

Table B.6: Q8. What do you need more help with? (n = 48)

Response	Count	Share
Understanding concepts	25	52.1 %

Response	Count	Share
Both	18	37.5 %
Practising questions	3	6.2 %
Not sure	2	4.2 %

Table B.7: Q9. Would an app based on your board syllabus help you? (n = 48)

Response	Count	Share
Yes	38	79.2 %
Maybe	8	16.7 %
No	2	4.2 %

Table B.8: Q10. Would a chatbot that tests, marks and explains help you learn better? (n = 48)

Response	Count	Share
Yes	38	79.2 %
Maybe	7	14.6 %
No	3	6.2 %

Table B.9: Q13. Do you download past papers or textbooks from the internet? (n = 48)

Response	Count	Share
Yes, but they are difficult to find	25	52.1 %
No, I do not download them	16	33.3 %
Yes, and they are easy to find	7	14.6 %

Table B.10: Q14. Is there a difference between board exam content and online study material? (n = 48)

Response	Count	Share
Yes	24	50.0 %
Sometimes	14	29.2 %
No	10	20.8 %

B.3 CODING OF THE OPEN QUESTIONS

The five open questions were coded into themes. A single answer could carry more than one theme, so the theme counts below sum to more than the number of respondents. The coding rules are held in code, in `thesis/survey_data.py`, so that the counts quoted in Chapter 3 can be re-derived from the raw answers rather than taken on trust.

Table B.11: Q7. Problems reported with ChatGPT (26 of 45 respondents reported a problem; 19 reported none)

Theme	Mentions
Answers too long or padded	11
Wording too difficult or too advanced	7
Wrong, vague or irrelevant answers	7
Does not understand the question	3
Model or data limits	3
Weak on numericals and maths	2
No diagrams	1

Table B.12: Q11. Features asked for in a learning application (n = 48)

Theme	Mentions
Past papers	11
Simple wording and to-the-point answers	10
Keybooks and textbooks	8
Chapter videos	6
Quizzes and MCQs	4
Voice answers	4
AI chatbot	3
Correct and accurate answers	3
Syllabus-aligned content	2
Free to use	2
Urdu option	1

Table B.13: Q12. Biggest difficulty faced while studying (n = 46)

Theme	Mentions
Understanding concepts	14

Theme	Mentions
Time management	11
Long syllabus	8
Focus and distraction	4
Finding study material	4
No guidance	3

Table B.14: Q5. Use of online study tools, grouped (n = 48)

Group	Count	Share
Uses ChatGPT or a chatbot	14	29.2 %
Uses no online tool	26	54.2 %
Uses some other tool	8	16.7 %

B.4 CHARTS OF THE PRINCIPAL RESULTS

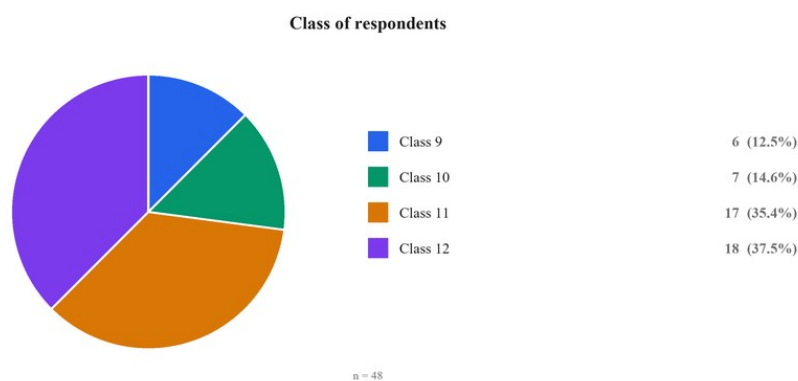


Figure B.1: Class of respondents.

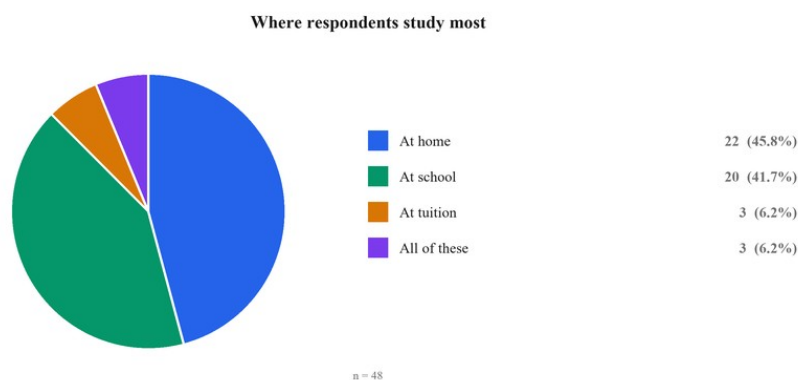


Figure B.2: Where respondents study most.

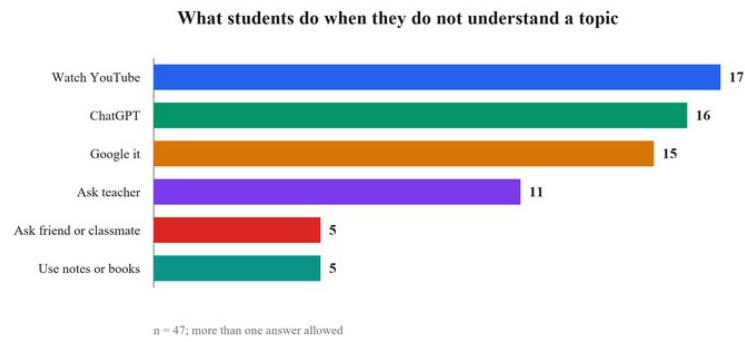


Figure B.3: What students do when a topic is unclear.

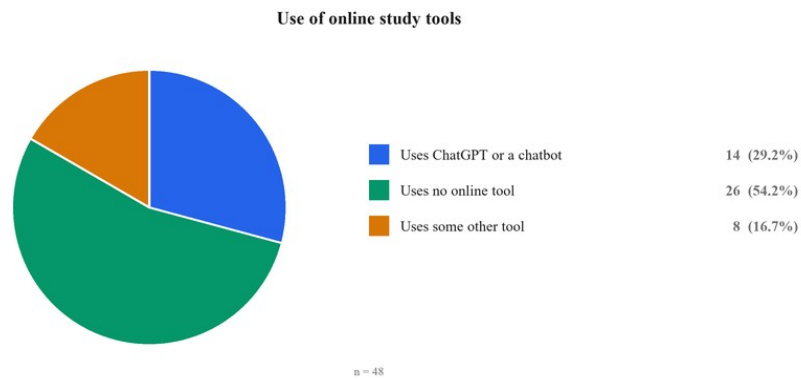


Figure B.4: Use of online study tools.

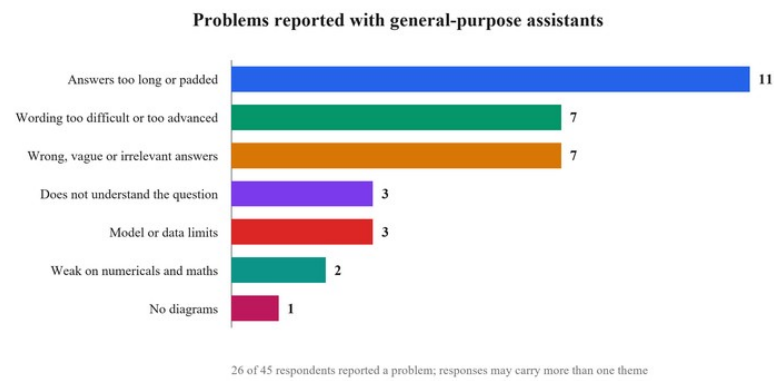


Figure B.5: Problems reported with general-purpose assistants.

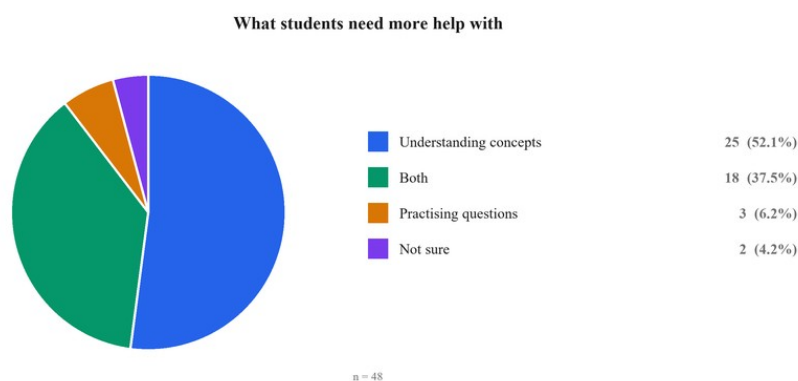


Figure B.6: What students need more help with.

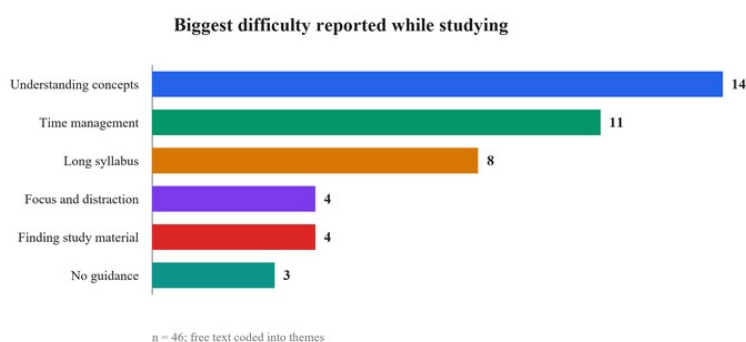


Figure B.7: Biggest difficulty reported while studying.

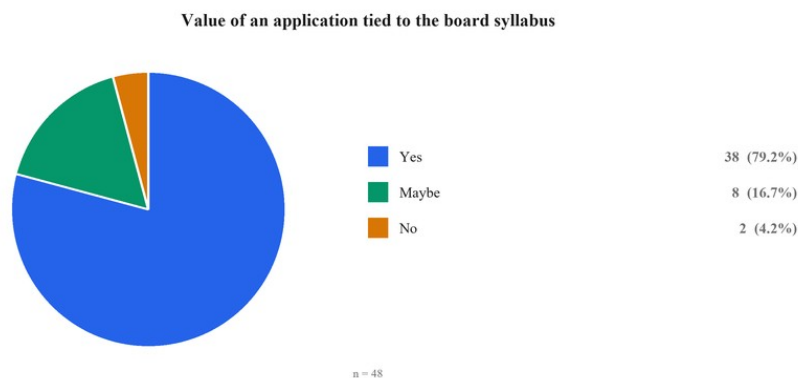


Figure B.8: Value of an application tied to the board syllabus.

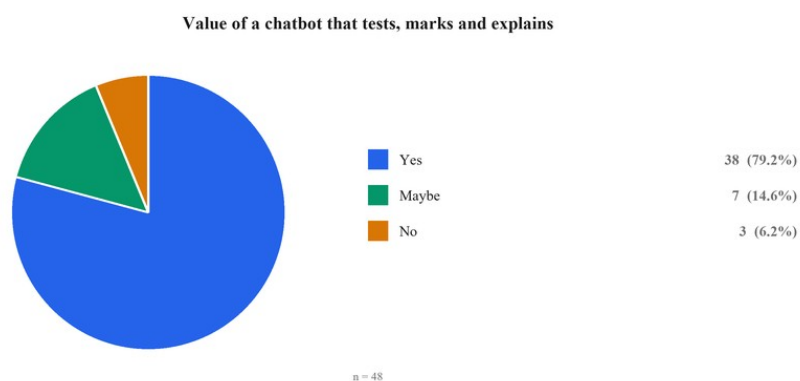


Figure B.9: Value of a chatbot that tests, marks and explains.

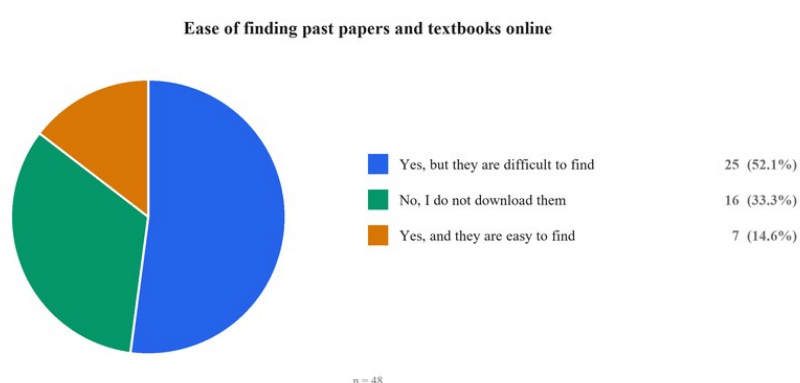


Figure B.10: Ease of finding past papers and textbooks online.

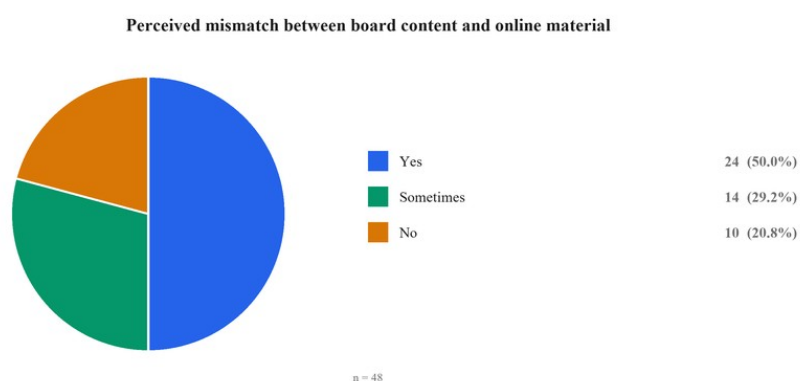


Figure B.11: Mismatch between board content and online material.

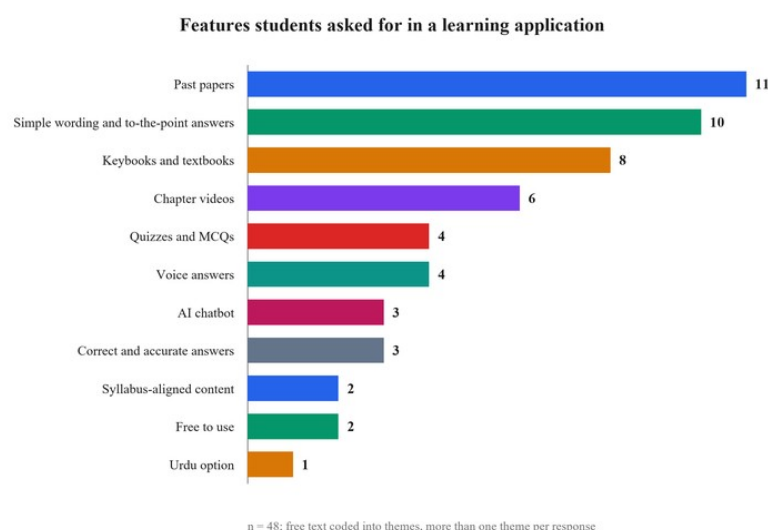


Figure B.12: Features asked for in a learning application.

B.5 LIMITATIONS OF THE SURVEY

Three limitations should be kept in mind when reading these results. The sample of forty-eight is a convenience sample of students the team could reach, so it is not a random sample of Pakistani school students and the percentages carry no claim of national representativeness. Class 11 and Class 12 supply 72.9 per cent of the responses, so the views of Class 9 and Class 10 students are under-represented. Finally, the questions on the value of a board-syllabus application and of a marking chatbot describe products that did not yet exist, and stated interest in a hypothetical product is weaker evidence than observed behaviour. The behavioural questions ask where students study, what they do when stuck, and what they already use. Those should carry more weight in the analysis than the two questions about a proposed product, even though the proposed-product questions produced the largest majorities.

APPENDIX C: SELECTED CODE LISTINGS

The listings below complement those given in Chapter 6. They are reproduced from the delivered codebase.

C.1 PROMPT TEMPLATE — TEACHER AGENT

Listing C.1: The teacher agent prompt template (abridged)

```
You are an AI tutor for Pakistani students.
Class: {class_level} | Board: {board} | Subject: {subject}

RULES:
- Prefer using the provided textbook/keybook content as the primary source.
- If the exact topic is NOT available in the textbook/keybook, you MAY answer using
  general knowledge for basic conceptual understanding.
- When using general knowledge, clearly mention:
  "This explanation is not directly available in your {class_level} {subject}
  textbook, but here is a simple general explanation."
- Never invent textbook references or page numbers.
- Never say "no response".
- Use simple, student-friendly words.
- Keep answers concise, clear, and exam-focused.
- Language: reply in the same language the student used (English or Urdu).

FORMAT (adapt based on question type):
- "What is / Define"      -> Definition + explanation + key points + example
- "Explain / How / Why"   -> Direct paragraph explanation
- "Difference / Compare"  -> X: ... | Y: ... | Key difference: ...
- "Name / List / Give"    -> Numbered list
- "Function of / Purpose" -> 2-3 sentences

MATHS ONLY:
Given: | To Find: | Method: [from textbook]
Step 1: [rule/formula used]
Step 2: ...
Answer: [with units]

CONTENT:
{context}

Question: {question}

Answer:
```

Source: ai-service/prompts/teacher.txt

C.2 PROMPT TEMPLATE — EVALUATOR

Listing C.2: The evaluator prompt template (abridged)

```
You are an exam evaluator for Pakistani students.
Class: {class_level} | Board: {board} | Subject: {subject}

Evaluate the student answer based ONLY on the textbook content below.

MARKING SCHEME:
- MCQ = 1 mark    - Short Question = 3 marks    - Long Question = 8 marks

SCORING RULES:
```

```

For MCQs:          1 = correct, 0 = incorrect
For Short Questions: 3 = complete and correct
                   2 = mostly correct with minor missing points
                   1 = partially correct
                   0 = incorrect
For Long Questions: 8 = complete and accurate
                   6-7 = mostly correct with small missing details
                   4-5 = partially correct
                   2-3 = mostly incorrect
                   0-1 = completely wrong

RULES:
- Evaluate ONLY from the provided textbook content.
- One sentence on what was correct. One sentence on what was missing.
- For Maths: identify the exact wrong step and provide the correction.
- Never give marks above the maximum allowed for the question type.

OUTPUT FORMAT (strictly follow this):
SCORE: [earned marks]/[total marks]
FEEDBACK: [maximum 2-3 short sentences]

CONTENT:
{context}

Question: {question}
Student Answer: {student_answer}
Evaluation:

```

Source: ai-service/prompts/evaluator.txt

C.3 PROMPT TEMPLATE — TESTER AGENT

Listing C.3: The tester agent prompt template (abridged)

```

You are an exam question generator for Pakistani students.
Student details: Class {class_level} | {board} Board | Subject: {subject}
Topic: {topic}

CONTENT SOURCES (in priority order):
1. PAST PAPER QUESTIONS - highest priority
   Extract questions that have actually appeared in past papers.
2. TEXTBOOK AND KEYBOOK CONTENT - use for new questions only

STRICT RULES:
1. Use ONLY the provided content. Never go outside it.
2. Mark each question clearly with its source.
3. Generate exactly {num_questions} question(s) of type: {question_type}
4. Match exact difficulty and style of {board} Board exams.
5. Marks weightage: MCQ = 1 mark | Short = 3 marks | Long = 8 marks

QUESTION LABELS (required on every question):
  Past Paper - taken directly from a past paper
  Similar    - inspired by past paper style
  New        - generated from textbook/keybook

MCQ FORMAT:
Q[n]. [Question] [label]
A) [option] B) [option] C) [option] D) [option]
Correct: [Letter]
Explanation: [one line why this is correct]

CONTENT:
{context}

Generate {num_questions} {question_type} question(s) on: {topic}

```


Source: ai-service/prompts/tester.txt

C.4 PAST-PAPER CLEANING RULES

Listing C.4: Domain-specific noise removal for examination papers (abridged)

```
def clean_past_paper(text: str) -> str:
    # roll-number boxes and page references
    text = re.sub(r'ROLL\s*NUMBER.*?\n', '', text, flags=re.IGNORECASE)
    text = re.sub(r'Roll\s*No\.\?.*?\n', '', text, flags=re.IGNORECASE)
    text = re.sub(r'Page\s*\d+\s*of\s*\d+.*?\n', '', text, flags=re.IGNORECASE)

    # examination header boilerplate
    for pattern in (r'INTERMEDIATE.*?\n', r'FEDERAL.*?EDUCATION.*?\n',
                    r'SECONDARY.*?EDUCATION.*?\n', r'ISLAMABAD.*?\n',
                    r'Time allowed:.*?\n', r'Total Marks.*?\n', r'NOTE:.*?\n'):
        text = re.sub(pattern, '', text, flags=re.IGNORECASE)

    # section headers and instructions, keeping the questions themselves
    text = re.sub(r'SECTION[\s\-\-]*[ABC].*?\n', '', text, flags=re.IGNORECASE)
    text = re.sub(r'Attempt any.*?\n', '', text, flags=re.IGNORECASE)
    text = re.sub(r'All parts carry equal marks.*?\n', '', text,
                  flags=re.IGNORECASE)

    # paper codes and marks allocations such as "(14 x 3 = 42)"
    text = re.sub(r'\d+HA\w*', '', text)
    text = re.sub(r'\\(\d+\s*x\s*\d+\s*=s*\d+\\)', '', text)

    # OCR garbage from empty MCQ bubbles
    text = re.sub(r'[oo0]\s*[oo0]\s*[oo0]', '', text)

    return re.sub(r'\n{3,}', '\n\n', text).strip()
```

Source: ai-service/scripts/02_clean_and_chunk.py

C.5 RESUMABLE EMBEDDING AND UPSERT

Listing C.5: Batched embedding with resume support

```
def process_file(jsonl_path, index, embedded_ids) -> int:
    chunks = [json.loads(line) for line in open(jsonl_path, encoding="utf-8") if
line.strip()]
    chunks_to_embed = [c for c in chunks if c["chunk_id"] not in embedded_ids]
    if not chunks_to_embed:
        return 0 # nothing to do; nothing billed

    total_uploaded, batch_buffer = 0, []
    for i, chunk in enumerate(chunks_to_embed, start=1):
        try:
            chunk["embedding"] = get_embedding(chunk["text"])
            batch_buffer.append(chunk)
            save_embedded_id(chunk["chunk_id"]) # written before the upsert
            if len(batch_buffer) >= BATCH_SIZE: # 100 vectors per upsert
                upload_batch(index, batch_buffer)
                total_uploaded += len(batch_buffer)
                batch_buffer = []
                time.sleep(0.5)
            time.sleep(0.05)
        except Exception as e:
            log(f" ERROR: {chunk['chunk_id']}: {e}")
            continue # one bad chunk does not stop
the run

    if batch_buffer:
```

```
upload_batch(index, batch_buffer)
total_uploaded += len(batch_buffer)
return total_uploaded
```

Source: ai-service/scripts/03_embed_and_store.py

APPENDIX D: USER MANUAL

D.1 FOR STUDENTS

1. Open the platform in a browser and choose Register. Enter your name, email address and a password, then open the verification email and click the link. Alternatively, choose Continue with Google, which verifies your address automatically.
2. Sign in and open Selection. Choose your board, your class and your subject. This selection is remembered and is applied to everything the system generates for you.
3. To learn a concept, open AI Tutor and type your question. You may write in English or Urdu. Follow-up questions such as “explain more” stay on the same topic. Each conversation is saved in the list on the left and can be reopened or deleted.
4. To practise, open Practice. Choose whether to cover the whole book or select particular chapters, then choose multiple-choice, short or long questions. Answer each question and read the feedback before moving on.
5. At the end of a practice session you receive your score, percentage, grade, the topics you did well in, the topics you struggled with and a study recommendation.
6. To download study material, open Resources, filter by board, class, subject and type, and choose Download.
7. To review your performance over time, open Progress, which shows your average score by subject and identifies the subject that needs the most attention.
8. To change your password, enable two-factor authentication or switch between light and dark appearance, open Settings.

D.2 FOR ADMINISTRATORS

1. Sign in with an account whose role is administrator. The administrative section appears in the navigation.

2. Open Content Manager to upload a resource. Provide a title, choose the type, board, class and subject, and select a file of up to 100 megabytes. The resource becomes visible to students immediately.
3. Use the same screen to edit a resource's details or to delete it. Deleting a resource also removes the stored file.
4. Open User Management to list accounts. Accounts can be searched and filtered by role and status, edited, blocked or deleted. Deleting an account also removes that student's chat sessions, quiz history and activity records.
5. Open the Admin Dashboard for aggregate counts and recent registrations.

D.3 FOR THE SYSTEM OPERATOR

1. Install PHP 8.4 with Composer, Node.js with npm, Python 3.12 and MySQL 8.
2. Copy `.env.example` to `.env` and set the database credentials, `APP_URL`, `AI_SERVICE_URL`, the Google OAuth client identifier, secret and redirect URI, and the mail transport settings.
3. Run ``composer install``, ``php artisan key:generate``, ``php artisan migrate`` and ``npm install``.
4. In `ai-service`, create a virtual environment, run ``pip install -r requirements.txt``, and set `OPENAI_API_KEY`, `PINECONE_API_KEY` and `PINECONE_INDEX` in the service's own `.env` file. Place the Google Cloud service-account credential file as `google-credentials.json`.
5. Start the AI service with ``uvicorn main:app --port 8001`` and the application with ``composer run dev``, which starts the web server, the queue listener and the Vite development server together. For production, run ``npm run build`` and serve the application through Nginx or Apache with PHP-FPM.
6. To add curricular material, place PDF files under `ai-service/data/raw/{board}/{class}/{subject}/{type}/` and run the three ingestion scripts in order: `01_extract_text.py`, `02_clean_and_chunk.py` and `03_embed_and_store.py`. Every stage is idempotent, so an interrupted run may be restarted safely.

7. Monitor `ai-service/data/token_usage.jsonl` for the token consumption and cost of every model call.

APPENDIX E: SIMILARITY REPORT

The similarity report produced by the university's plagiarism-detection service is to be attached here, in accordance with the departmental submission requirement that the similarity index be below the threshold specified in the evaluation rubric.

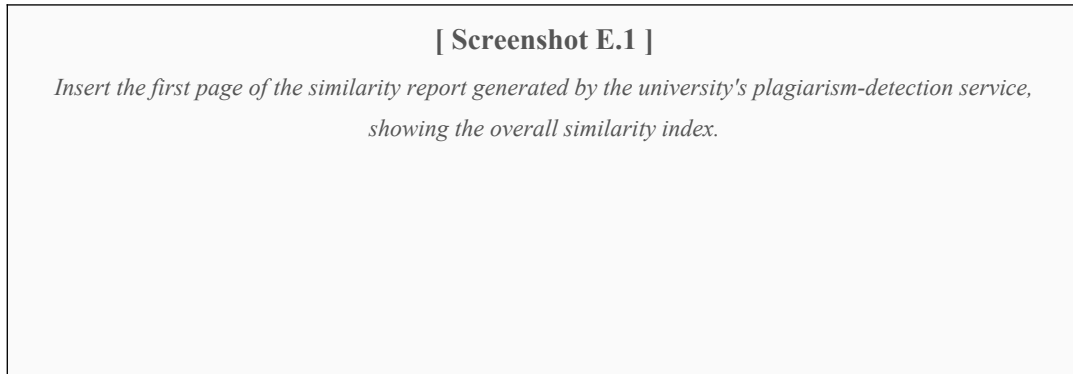


Figure E.1: Similarity report summary page.